

LAB MANUAL

Course: AIC365-Natural Language Processing



Department of Computer Science

Java Learning Procedure

- 1) Stage **J** (Journey inside-out the concept)
- 2) Stage **a₁** (Apply the learned)
- 3) Stage **V** (Verify the accuracy)
- 4) Stage **a₂** (Assess your work)

Table of Contents

Lab #	Topics Covered	Page #
Lab # 01	Python Essentials for NLP	
Lab # 02	Data Acquisition	
Lab # 03	Regex, Tokenization & Sentence Segmentation	
Lab # 04	Text Normalization, Cleaning & Minimum Edit Distance	
Lab # 05	N-gram Language Models & Smoothing	
Lab # 06	Text Classification: Naive Bayes & Logistic Regression	
Lab # 07	POS Tagging & Named Entity Recognition	
Mid Term Examination		
Lab # 08	Topic Modeling: LDA & LSA	
Lab # 09	Vector Semantics & Word Embeddings	
Lab # 10	Visualizing & Evaluating Embeddings	
Lab # 11	Neural Networks for Text: RNN, LSTM, GRU	
Lab # 12	Sequence-to-Sequence, Attention & the Transformer	
Lab # 13	Large Language Models: Prompting, Generation, Evaluation & Retrieval-Augmented Generation	
Terminal Examination		

Lab 1: Python Essentials for NLP

Statement Purpose:

This lab has two jobs. First, it makes sure every student can actually *write* Python comfortably enough to manipulate text. Second, it gives enough NumPy/pandas fluency to treat text as structured data. This is the foundation every later lab depends on.

Activity Outcomes:

This lab teaches you the following topics:

- Core Python data structures for text: strings, lists, dictionaries, sets, tuples, and why each is the natural fit for a specific text-processing job.
- String methods essential to NLP
- Comprehensions (list/dict/set) as the idiomatic way to transform text collections
- Functions and `lambda`, and `map()` / `filter()` / `sorted(key=...)` for text pipelines
- `collections.Counter` for frequency counting which is the basis of almost every stats-based NLP technique later in the course
- File handling and the `json` module for reading structured corpora, plus UTF-8 text handling
- NumPy essentials and pandas essentials

Instructor Note:

As pre-lab reading, assign Chapter 1 ("Introduction") of *Speech and Language Processing* (Jurafsky & Martin, 3rd ed.) for the field-level overview of what NLP is and why it's hard, and Chapter 2 ("Data Collection and Pipelines") of *Practical Natural Language Processing* (Vajjala et al.) for how a real NLP pipeline is structured end to end.

1) Stage J (Journey)

Before any NLP technique can be applied, a student needs fluency in the language used to implement it. This lab establishes that foundation on two levels. First, it builds comfort with Python's core data structures and idioms for manipulating text: strings, lists, dictionaries, comprehensions, and functional tools such as `map()`, `filter()`, and `sorted(key=...)`. Second, it introduces NumPy and pandas, the two libraries that let text be treated as structured, numerical data rather than as opaque strings. Every later lab in this course, from tokenization to embeddings to Transformer-based models, assumes this fluency as a prerequisite. The lab's ultimate goal is conceptual: text is fundamentally a sequence of discrete symbols, and virtually every NLP method covered later is a different strategy for converting that symbolic sequence into numbers a model can process.

Symbolic Text and the Motivation for Structured Representations

Natural language, as it exists on a page or a screen, is a sequence of discrete symbols: characters, words, punctuation marks. A computational model, however, operates exclusively on numerical values. This mismatch is the single motivating problem behind essentially every method taught in this course. Python's core data structures give a first, coarse-grained way to organize discrete symbols before any numerical transformation happens: a `string` represents raw symbolic text, a `list` represents an ordered sequence of tokens or sentences, a `dictionary` represents a mapping (for example, word to frequency, or word to index), and a `set` represents a deduplicated vocabulary. Choosing the correct structure for a given text-processing task is not a stylistic preference. A `dict` gives average $O(1)$ lookup for vocabulary membership checks, while a `list` would require $O(n)$ scanning; this distinction becomes materially important once vocabulary sizes reach the tens of thousands of unique tokens typical of real corpora.

Comprehensions (`list`, `dict`, and `set`) are the idiomatic Python mechanism for applying a transformation or filter across a text collection in a single, readable expression. Conceptually, a comprehension such as `[transform(x) for x in tokens if condition(x)]` is a compact fusion of a `map` operation and a `filter` operation, and this fusion pattern recurs throughout NLP preprocessing: normalizing case, stripping punctuation, removing stopwords, and building vocabulary-to-index mappings are all instances of the same map-filter structure applied to a token sequence.

Frequency Counting as the Statistical Basis of NLP

Many classical NLP techniques, from simple keyword extraction to n-gram language models, ultimately reduce to counting how often symbols or symbol sequences occur. `collections.Counter` is a specialized `dict` subclass built exactly for this purpose: it maps each distinct item in an iterable to its occurrence count, and exposes `.most_common(n)` to retrieve the n highest-frequency items directly, without requiring the student to manually sort a dictionary by value.

This frequency-counting operation is the conceptual seed of a broader idea covered later in the course: the **distributional hypothesis**, which holds that a word's meaning can be inferred from the statistical pattern of its co-occurrences. Before that idea can be formalized mathematically, students need to be comfortable with the basic operation of counting occurrences accurately, which requires first normalizing text (lowercasing, punctuation stripping) so that "Data" and "data," are correctly counted as the same token rather than as two distinct ones.

Vector Semantics: From Symbol Sequences to Numerical Similarity

The transition from symbolic text to numerical representation is formalized through the vector. A **vector** is an ordered array of numbers, and once two pieces of text are each represented as a vector, geometric operations become available to compare them. The **dot product** of two vectors **a** and **b** of equal dimension n is defined as:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i$$

where a_i and b_i are the i -th components of each vector. The dot product alone is sensitive to vector magnitude, which is problematic when comparing texts of different lengths, since a longer document naturally produces larger raw counts. **Cosine similarity** corrects for this by normalizing the dot product against the product of the vectors' magnitudes (their **norms**):

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|, \|\mathbf{b}\|}, \quad \|\mathbf{a}\| = \sqrt{\sum_{i=1}^n a_i^2}$$

Here $\|\mathbf{a}\|$ and $\|\mathbf{b}\|$ are the Euclidean norms (magnitudes) of each vector, and θ is the angle between them in vector space. The resulting value ranges from -1 to 1 for general vectors, and from 0 to 1 for non-negative vectors such as word-count vectors, with values closer to 1 indicating higher similarity. This single formula is the mathematical bedrock for nearly every later similarity-based technique in the course, including embedding comparison and retrieval scoring in RAG pipelines.

NumPy's `np.dot()` and `np.linalg.norm()` implement these two operations directly, which is why vector fluency in NumPy is introduced before any NLP-specific library.

Note: The intuition to anchor here: cosine similarity measures the *direction* two vectors point in, not their *length*. Two documents with identical word-usage proportions but very different lengths will still produce a cosine similarity close to 1 .

Tabular Organization of Text Corpora with pandas

Once a corpus consists of more than a handful of text items, a `pandas.DataFrame` becomes the standard organizational structure: one row per document or sentence, with columns holding both the

raw text and derived numerical features (word counts, character counts, labels). The `.apply()` method broadcasts an arbitrary function, including a `lambda`, across every row or every value in a column, which is the mechanism used to compute per-row derived features without writing an explicit loop. `.sort_values()` then reorders the `DataFrame` by any computed column, which is the operation underlying tasks like ranking documents by length or by relevance score. This row-per-document, column-per-feature convention is used consistently for every dataset encountered later in the course, which is why establishing it here, before any NLP-specific content, is deliberate.

Core Libraries and Architectural Roles

Library	Role in the Pipeline
<code>collections</code>	Supplies <code>Counter</code> , a <code>dict</code> subclass specialized for frequency counting, used as the basis of every stats-based technique introduced later in the course.
<code>re</code>	Provides regular-expression-based text cleaning, used here to strip punctuation before frequency counting so that surface variants of the same word are not double-counted.
<code>json</code>	Serializes and deserializes structured data between Python objects and <code>.json</code> files, the standard format for storing small text corpora.
<code>numpy</code>	Provides array-based numerical operations (<code>np.array</code> , <code>np.dot</code> , <code>np.linalg.norm</code>), forming the mathematical foundation for vector similarity used throughout the course.
<code>pandas</code>	Provides the <code>DataFrame</code> structure for organizing text collections in tabular form, with vectorized methods (<code>.apply()</code> , <code>.sort_values()</code>) for computing and ranking derived features.

Critical Classes, Functions, and Methods

- `str.strip()` / `str.lower()` / `str.split()` / `str.join()`: The four core string-normalization primitives. `.strip()` removes leading/trailing whitespace, `.lower()` case-folds the string, `.split()` tokenizes on whitespace by default and returns a `list`, and `.join()` performs the inverse operation, concatenating a list of strings using the calling string as a separator.
- `sorted(iterable, key=lambda x: ...)`: Returns a new sorted list without mutating the original. The `key` parameter, rather than sorting on the item's natural ordering, sorts on the value returned by the supplied function applied to each item, which is how sorting by a derived property (such as word length) rather than by the item itself is achieved.
- `filter(function, iterable)`: Returns an iterator yielding only the items for which `function` evaluates to `True`. Must be wrapped in `list()` to materialize the result, since `filter()` is lazily evaluated.
- `collections.Counter(iterable)`: Constructs a frequency map from an iterable of hashable items. `.most_common(n)` returns a list of `(item, count)` tuples sorted by descending count, truncated to the top `$n$`.
- `json.dump(obj, file)` / `json.load(file)`: Serialize a Python object to a file-like object as JSON text, and deserialize JSON text from a file-like object back into a Python object, respectively.

- `bytes.decode(encoding)`: Converts a raw byte sequence into a `str` using the specified encoding. Raises `UnicodeDecodeError` when the byte sequence is not valid under the specified encoding, which is the exception class caught when handling mis-encoded input.
- `np.dot(a, b) / np.linalg.norm(a)`: Compute the dot product of two arrays and the Euclidean norm (magnitude) of a single array, respectively; together they implement the cosine similarity formula.
- `pd.DataFrame(data)`: Constructs a tabular structure from a dict of equal-length sequences (or other compatible input), with each key becoming a column.
- `DataFrame.apply(function, axis=...)`: Applies `function` along the specified axis (rows or columns). When applied to a `Series` (a single column), `function` is called once per value.
- `DataFrame.sort_values(by=..., ascending=...)`: Returns a new `DataFrame` sorted by the specified column, with `ascending` controlling sort direction.

Parameters and Configurations

Parameter	Effect
<code>r'[^\\w\\s]'</code>	Matches any character that is not a word character or whitespace, effectively stripping punctuation before tokenization; an incomplete pattern would leave residual punctuation attached to tokens and inflate the apparent vocabulary size.
<code>n</code> in <code>.most_common(n)</code>	Controls how many top-frequency items are returned; omitting it returns the entire frequency map sorted by count, which is inefficient for large corpora when only the top few items are needed.
<code>encoding="utf-8"</code>	Explicitly sets the text encoding for file I/O. Omitting this relies on the platform's default encoding, which is not guaranteed to be UTF-8 on every operating system and is a common source of non-reproducible bugs.
<code>key=lambda x: len(x)</code>	Defines the sort criterion as string length rather than lexicographic order; changing the lambda body changes what "sorted" means without changing the surrounding code structure.
<code>axis</code>	Determines whether the function is applied column-wise (<code>axis=0</code> , the default) or row-wise (<code>axis=1</code>); applying <code>.apply()</code> to a single <code>Series</code> as in Activity 7 does not require this parameter, but it becomes relevant once a function needs values from multiple columns in the same row.
<code>ascending</code>	Controls sort direction; <code>ascending=False</code> in Activity 7 ranks the longest sentences first, which is the opposite of the pandas default.

Implementation Pitfalls, Quirks, and Edge Cases

- `sentence.strip().lower().split()` performs whitespace-based splitting only. This does not correctly separate punctuation attached to a word (for example, `"text."` remains a single token including the period), which is why Activity 1 alone is insufficient preparation for real tokenization; that gap is addressed only once proper tokenization is introduced in a later lab.
- `filter()` and `map()` return lazy iterator objects, not lists. Forgetting to wrap the result in `list()` before printing or indexing is a common source of confusing output (`<filter object at 0x...>` instead of the expected values).

- The `UnicodeDecodeError` demonstration in Activity 5 is deliberately constructed using UTF-16-encoded bytes decoded under a UTF-8 assumption. This is representative of a real and common failure mode: corpora collected from mixed sources (different operating systems, different tools) frequently mix encodings, and a pipeline that assumes UTF-8 everywhere will fail exactly this way on unexpected input.
- `np.dot()` and `np.linalg.norm()` will not raise an error if the two input vectors have different dimensions in a way that still broadcasts, but a cosine similarity computed between vectors representing different vocabularies (misaligned dimensions) produces a numerically valid but semantically meaningless result. Dimension alignment must be verified by the student, not assumed from the code running without error.

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Given a sentence, use `.lower()`, `.split()`, `.strip()`, slicing, and `.join()` to normalize the sentence and reconstruct it in different forms.

Solution:

```
# Sample sentence with extra whitespace
sentence = "   Large Language Models can generate human-like text.   "

# Normalize using strip, lower, and split
normalized_words = sentence.strip().lower().split()

# Use slicing to get the first 3 words
sliced_words = normalized_words[:3]

# Reconstruct using join
reconstructed_sentence = " ".join(sliced_words)

print("Original words list:", normalized_words)
print("Reconstructed (first 3 words):", reconstructed_sentence)
```

Output:

```
Original words list: ['large', 'language', 'models', 'can', 'generate', 'human-like', 'text.']
Reconstructed (first 3 words): large language models
```

Activity 2:

From a list of words, use a list comprehension to build a list of only the words longer than 4 characters, and a dict comprehension to build a dict mapping each word to its length.

Solution:

```
vocab = ["data", "science", "python", "ai", "model", "git"]
```

```
# List comprehension for words longer than 4 characters
long_words = [word for word in vocab if len(word) > 4]

# Dict comprehension mapping word to its length
word_lengths = {word: len(word) for word in vocab}

print("Long words:", long_words)
print("Word lengths:", word_lengths)
```

Output:

```
Long words: ['science', 'python', 'model']
Word lengths: {'data': 4, 'science': 7, 'python': 6, 'ai': 2, 'model': 5, 'git': 3}
```

Activity 3:

Using `lambda` with `sorted(key=...)`, sort a list of words by length. Using `filter()`, remove all words shorter than a given length from the list.

Solution:

```
words = ["neural", "network", "api", "database", "ai"]

# Sort words by length using lambda
sorted_by_length = sorted(words, key=lambda x: len(x))

# Filter out words shorter than 5 characters
min_length = 5
filtered_words = list(filter(lambda x: len(x) >= min_length, words))

print("Sorted by length:", sorted_by_length)
print("Filtered words (>= 5 chars):", filtered_words)
```

Output:

```
Sorted by length: ['ai', 'api', 'neural', 'network', 'database']
Filtered words (>= 5 chars): ['neural', 'network', 'database']
```

Activity 4:

Using `collections.Counter`, count word frequencies in a given paragraph and retrieve the 5 most common words with `.most_common(5)`.

Solution:

```
import collections
import re

paragraph = ""
```

Artificial Intelligence is transforming the world. Machine learning models require large amounts of data. Deep learning uses neural networks to find patterns. Large language models can generate human-like text. Data science libraries like pandas and numpy are essential tools. Developers write code to train these models.

```
# Remove basic punctuation and convert to lowercase for accurate counting
clean_text = re.sub(r'^\w\s]', '', paragraph.lower())
tokens = clean_text.split()

# Count frequencies and get top 5
word_counts = collections.Counter(tokens)
top_5_words = word_counts.most_common(5)

print("Top 5 most common words:")
print(top_5_words)
```

Output:

```
Top 5 most common words:
[('models', 3), ('learning', 2), ('large', 2), ('data', 2), ('to', 2)]
```

Activity 5:

Read a small `.json` file of sample sentences into a Python list, then write the list back out to a new `.json` file. Attempt to read a deliberately mis-encoded string and catch and explain the UTF-8 decoding error that results.

Solution:

```
import json

sample_sentences = [
    "Machine learning models require large amounts of data.",
    "Data science libraries are essential tools."
]

# Write out to a new JSON file
with open("sentences.json", "w", encoding="utf-8") as f:
    json.dump(sample_sentences, f)

# Read the JSON file back into a Python list
with open("sentences.json", "r", encoding="utf-8") as f:
    loaded_sentences = json.load(f)

print("Loaded from JSON:", loaded_sentences)

# Attempt to decode a deliberately mis-encoded string (UTF-16 bytes read as UTF-8)
bad_bytes = b'\xff\xfeM\x00a\x00c\x00h\x00i\x00n\x00e\x00'
```

```
try:
    decoded_text = bad_bytes.decode('utf-8')
except UnicodeDecodeError as e:
    print("\nCaught a decoding error:", e)
    print("Explanation: The byte sequence uses an incompatible encoding (like UTF-16), "
          "so Python fails to parse it according to strict UTF-8 mapping rules.")
```

Output:

```
Loaded from JSON: ['Machine learning models require large amounts of data.', 'Data science libraries are essential tools.']

Caught a decoding error: 'utf-8' codec can't decode byte 0xff in position 0: invalid start byte
Explanation: The byte sequence uses an incompatible encoding (like UTF-16), so Python fails to parse it according to strict UTF-8 mapping rules.
```

Activity 6:

Build two vectors using NumPy and compute their dot product, their norms, and the cosine similarity between them.

Solution:

```
import numpy as np

# Build two simple vectors
vec_a = np.array([1, 2, 3])
vec_b = np.array([4, 5, 6])

# Compute dot product
dot_product = np.dot(vec_a, vec_b)

# Compute norms (magnitude)
norm_a = np.linalg.norm(vec_a)
norm_b = np.linalg.norm(vec_b)

# Compute cosine similarity
cosine_sim = dot_product / (norm_a * norm_b)

print("Dot Product:", dot_product)
print("Norm of vec_a:", norm_a)
print("Norm of vec_b:", norm_b)
print("Cosine Similarity:", cosine_sim)
```

Output:

```
Dot Product: 32
Norm of vec_a: 3.7416573867739413
Norm of vec_b: 8.774964387392123
Cosine Similarity: 0.9746318461970762
```

Activity 7:

Build a pandas DataFrame from a list of sentences with one row per sentence. Use `.apply()` to add a column containing the word count of each sentence, and use `.sort_values()` to rank the sentences by length.

Solution:

```
import pandas as pd

sentences = [
    "Artificial Intelligence is transforming the world.",
    "Machine learning models require large amounts of data.",
    "Deep learning uses neural networks."
]

# Build DataFrame
df = pd.DataFrame({"sentence": sentences})

# Add a word count column by splitting each sentence
df['word_count'] = df['sentence'].apply(lambda x: len(x.split()))

# Rank sentences by length (longest first)
df_ranked = df.sort_values(by='word_count', ascending=False)

print(df_ranked)
```

Output:

	sentence	word_count
1	Machine learning models require large amounts ...	8
0	Artificial Intelligence is transforming the wo...	6
2	Deep learning uses neural networks.	5

3) Stage **v** (verify)

Home Activities:

Activity 1:

Given a new short paragraph, write a function that returns a cleaned list of words (lowercased, stripped of leading/trailing punctuation) using only core Python.

Activity 2:

Using `Counter`, find the 10 most frequent words in a given article and print them as a clean table of word and count.

Activity 3:

Build a `DataFrame` with one row per sentence from a given paragraph, and add `word_count` and `char_count` columns using `.apply()`.

Activity 4:

Compute cosine similarity by hand (using NumPy only) for two given word-count vector pairs, and state which pair is "more similar" and why.

Activity 5:

Given a raw `.json` file of 10 short reviews, load it, build a `DataFrame`, add a Counter-based "top word" column per row (the most frequent word in that row, ignoring case), and sort the `DataFrame` by `word_count` in descending order.

4) Stage **a2** (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 2: Data Acquisition

Statement Purpose:

This lab gives you the practical skill of getting text out of the real-world formats it lives in. It includes extracting text from PDFs, Word documents, JSON APIs, and HTML pages.

Activity Outcomes:

This lab teaches you the following topics:

- Reading and extracting text from PDF files
- Reading and extracting paragraphs/headings from Word (.docx) documents
- Fetching JSON data from an internet source/API using `requests` and parsing it into Python objects
- Fetching a raw HTML page and understanding its basic tag structure
- HTML parsing with BeautifulSoup to extract specific elements (titles, links, text, tables)
- Basic web scraping to collect structured data from a web page into a Python list/DataFrame

Instructor Note:

As pre-lab reading, review Chapter 2 ("Data Collection and Pipelines") of *Practical Natural Language Processing* (Vajjala et al.), specifically the sections on web scraping and format-specific extraction. Students should also skim the documentation pages for `pdfplumber` (or `PyPDF2`), `python-docx`, `requests`, and `BeautifulSoup` before lab.

1) Stage **J** (Journey)

Every NLP pipeline begins with **data acquisition**, and in practice this stage is rarely as simple as opening a `.txt` file. Real-world text is locked inside binary PDF encodings, XML-based Word documents, JSON payloads returned by APIs, and HTML markup rendered by web servers. This lab builds the engineering competency to extract clean, usable text from each of these four source types. By the end of the lab, students will be able to reconstruct readable text from a PDF's character stream, distinguish structural elements (headings vs. body) inside a `.docx` file, consume a REST API and parse its JSON response, and navigate an HTML document object model to extract both individual elements and repeated structured records. These skills form the mandatory first stage of the NLP pipeline introduced in Lab 1: without reliable acquisition, no downstream preprocessing, modeling, or evaluation step has valid input.

The Opacity of PDF Layout and Character-Stream Reconstruction

A `.txt` file stores text as a linear sequence of characters with explicit line breaks. A **PDF**, by contrast, stores a page as a collection of positioned drawing instructions: each glyph is placed at an (x, y) coordinate with an associated font. The format has no native concept of "paragraph," "sentence," or even "word": these are visual groupings that a human eye infers from spacing, not structural units the file format encodes. This is why PDF text extraction is fundamentally a **reconstruction problem**. A parser must walk the content stream, collect glyphs in the order they were drawn, and infer word and line boundaries from proximity heuristics.

This has a direct practical consequence: multi-column layouts, tables, and headers/footers can cause the extracted text to interleave in an order that does not match visual reading order. A two-column academic paper, for instance, may extract as alternating lines from the left and right columns rather than the left column followed by the right. Libraries such as `PyPDF2` abstract this reconstruction but do not solve it perfectly. The extraction quality is therefore a function of the PDF's internal complexity, not of the library's sophistication alone.

Note: Extracted PDF text should always be treated as **noisy raw input**, not a finished corpus. Downstream cleaning (removing stray line breaks, merging hyphenated words split across a line boundary, filtering headers/footers) is a required companion step, not an optional one.

Document Object Models: XML Trees in DOCX and HTML

Unlike a PDF, both the `.docx` format and HTML are built on a **tree-structured markup model**. A `.docx` file is internally a ZIP archive containing XML files, where each paragraph is a distinct XML node carrying a **style attribute** (for example `Heading 1, Normal`). This is what makes it possible to programmatically separate headings from body text: the distinction is not inferred from font size or visual weight, but read directly from an explicit style name attached to each paragraph node.

HTML follows the same tree principle. A web page is a nested hierarchy of tags (`<html>` → `<body>` → `<div>` → `<a>`, and so on), where each tag can carry attributes (`class`, `id`, `href`) that identify its role. Parsing this tree correctly, rather than attempting to locate text with string slicing or regular expressions, is essential because:

1. Tag nesting depth is variable and unpredictable across pages.
2. The same visual element (for example, a book title) is almost always marked up with a consistent, repeatable tag-and-class pattern across a page, even when the surrounding page is otherwise irregular.
3. Regex-based extraction breaks silently the moment a page's markup changes slightly, while tree-based extraction (via `BeautifulSoup`) degrades more predictably because it operates on structural relationships rather than surface text patterns.

This is the conceptual justification for why `BeautifulSoup` searches by tag name and attribute (`find_all('article', class_='product_pod')`) rather than by scanning raw HTML text.

The REST/JSON Interchange Paradigm and Semi-Structured Parsing

Web APIs expose data through HTTP endpoints that return **JSON** (JavaScript Object Notation), a lightweight, human-readable, semi-structured text format built from nested key-value pairs, arrays, strings, numbers, and booleans. JSON has become the dominant interchange format for web APIs because it maps directly onto native data structures in most programming languages: a JSON object corresponds to a Python `dict`, and a JSON array corresponds to a Python `list`. This means that once an HTTP response body is confirmed to be valid JSON, deserializing it requires no custom parsing logic. The `requests` library's `.json()` method performs this deserialization automatically, converting the response body into nested Python `dict/list` structures ready for direct indexing.

This is conceptually distinct from HTML scraping. JSON parsing is **deterministic and structural**: the data was explicitly designed to be machine-read, and the schema (which keys exist, at which nesting level) is generally stable. HTML scraping is **inferential**: the page was designed for human visual consumption, and machine-readable structure must be recovered from tag and attribute conventions that can change without notice. This distinction is why the lab's ethical note applies specifically to scraping and not to JSON API consumption: an API is a data source that explicitly permits (and often documents) programmatic access, while scraping repurposes a page that was never designed with automated extraction in mind.

Note: When a public API exists for a target dataset, it is preferred over scraping the corresponding web page. This reduces both the fragility of the extraction code and the ethical/legal exposure associated with automated access to page content that was not designed for that purpose.

Core Libraries and Architectural Roles

Library	Role in the Pipeline
PyPDF2	Opens PDF files, exposes each page as a <code>PageObject</code> , and reconstructs raw text via <code>.extract_text()</code> . Handles low-level PDF stream parsing so the student never touches PDF byte structure directly.
docx (python-docx)	Opens <code>.docx</code> files as a <code>Document</code> object and exposes paragraphs with their associated style metadata, allowing structural classification (heading vs. body) without touching the underlying XML.
requests	Performs HTTP <code>GET</code> requests against both JSON APIs and HTML pages. Returns a <code>Response</code> object carrying status code, headers, raw text (<code>.text</code>), and, where applicable, deserialized JSON (<code>.json()</code>).
BeautifulSoup (from bs4)	Parses raw HTML text into a navigable, searchable tree structure. Provides tag- and attribute-based query methods (<code>.find()</code> , <code>.find_all()</code>) used to locate specific elements or repeated structured records.

Critical Classes, Functions, and Methods

- **PyPDF2.PdfReader**: Constructed from an open file handle in binary mode (`"rb"`). Exposes a `.pages` iterable, where each element supports `.extract_text()`, returning the reconstructed text of that single page as a string.
- **docx.Document**: Constructed directly from a file path. Exposes `.paragraphs`, a list of `Paragraph` objects. Each `Paragraph` has a `.text` attribute and a `.style.name` attribute, the latter used to test whether the paragraph is a heading (`style.name.startswith('Heading')`).
- **requests.get(url)**: Issues a synchronous HTTP `GET` request. The returned `Response` object's `.json()` method parses a JSON response body into native Python objects; `.text` returns the raw response body as a decoded string, used as the input to `BeautifulSoup`.
- **BeautifulSoup(html_text, 'html.parser')**: Constructs the parse tree from a raw HTML string. The second argument specifies the parser backend; `'html.parser'` is Python's built-in parser, chosen here for zero external dependency.
- **soup.find_all(tag, class_=...)**: Returns a list of all matching tag elements. The `class_` parameter (note the trailing underscore, required because `class` is a reserved Python keyword) filters by CSS class attribute, which is the mechanism used in Activity 5 to isolate repeated `<article class="product_pod">` blocks corresponding to individual book listings.

Parameters and Configurations

Parameter	Effect
"rb" mode	Opens the file in binary mode, required because PDF is a binary format; opening in text mode would corrupt the byte stream before PyPDF2 can parse it.
para.style.name	Determines heading vs. body classification. Relies on the document actually using Word's built-in heading styles; a document with manually bolded text but no assigned style will not be detected as a heading.
response.json() vs response.text	.json() assumes the response body is valid JSON and raises an exception otherwise; .text always succeeds and returns raw decoded text, which is why HTML pages are read via .text and passed to BeautifulSoup, not via .json().
'html.parser'	Selects the parsing backend. Alternatives such as 'lxml' are faster and more fault-tolerant on malformed HTML but require an additional external dependency.
class_='product_pod'	Restricts the search to elements carrying this exact CSS class, which is how a specific repeated structural block is isolated from the rest of the page.

Implementation Pitfalls, Quirks, and Edge Cases

- PdfReader.pages[i].extract_text() can return an empty string for pages that are pure images (scanned documents with no embedded text layer). This is not a bug in the extraction code; it reflects the absence of a text layer in the source PDF, and would require OCR, which is outside the scope of PyPDF2.
- python-docx's heading detection relies entirely on the style.name string matching the prefix "Heading". Documents authored with non-English Word installations, or documents where headings were manually formatted (bold, larger font) without applying an actual paragraph style, will silently fail the startswith('Heading') check and be misclassified as body text.
- Relative links extracted from books.toscrape.com (Activity 5) are concatenated with the base url string via simple string addition. This works only because the site's relative paths do not begin with a leading slash that would otherwise require proper URL joining via urllib.parse.urljoin. Applying the same pattern to a different site without checking this assumption is a common source of malformed URLs.
- Neither the PDF, DOCX, nor JSON activities implement retry logic or timeout handling on requests.get(). On an unstable network connection or a rate-limited endpoint, the request can hang indefinitely or fail without a clear error message pointing to the actual cause.

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Open a PDF file in Python and extract the raw text from all of its pages into a single string.

Solution:

```
import PyPDF2

# Open the PDF file in read-binary mode
# Note: Rename 'sample.pdf' as per your document in your environment
try:
    with open("sample.pdf", "rb") as file:
        reader = PyPDF2.PdfReader(file)
        extracted_text = ""

        # Iterate over all pages and append the text
        for page in reader.pages:
            extracted_text += page.extract_text() + "\n"

    # Print a small portion to verify
    print(extracted_text[:1000])
except FileNotFoundError:
    print("File 'sample.pdf' not found. Please upload it to run this cell.")
```

Output:

```
• Tools: re, NLTK, spaCy
• Dataset: sample customer message + a short messy business -report excerpt
Statement Purpose: This lab will give you the ability to pull structured information out of
```

Activity 2:

Open a Word (docx) document in Python and extract its paragraphs into a list, separating headings from body text.

Solution:

```
import docx

# Load the Word document
# Note: Rename 'sample.docx' as per your document in your environment
try:
    doc = docx.Document("sample.docx")

    headings = []
    body_paragraphs = []
```

```

# Loop through paragraphs and categorize based on style name
for para in doc.paragraphs:
    # Skip empty paragraphs
    if not para.text.strip():
        continue

    if para.style.name.startswith('Heading'):
        headings.append(para.text)
    else:
        body_paragraphs.append(para.text)

print("Total Headings:", len(headings))
print("Total Body Paragraphs:", len(body_paragraphs))
except FileNotFoundError:
    print("File 'sample.docx' not found. Please upload it to run this cell.")

```

Output:

```

Total Headings: 11
Total Body Paragraphs: 168

```

Activity 3:

Send a request to a provided public JSON API endpoint, retrieve the response, and parse it into a Python dictionary/list.

API: <https://jsonplaceholder.typicode.com/users>

Solution:

```

import requests

url = "https://jsonplaceholder.typicode.com/users"
response = requests.get(url)

# Parse the JSON response directly into a Python list of dictionaries
users_data = response.json()

# Extract specific information (e.g., names) to verify
user_names = [user['name'] for user in users_data]
print("Extracted Users:", user_names)

```

Output:

```

Extracted Users: ['Leanne Graham', 'Ervin Howell', 'Clementine Bauch', 'Patricia Lebsack', 'Chelsey Dietrich',

```

Activity 4:

Fetch a provided HTML page's raw source, then use BeautifulSoup to parse it and extract the page title and all hyperlink (<a>) text.

HTML Page: <https://quotes.toscrape.com/>

Solution:

```
import requests
from bs4 import BeautifulSoup

url = "https://quotes.toscrape.com/"
response = requests.get(url)

# Parse the HTML content
soup = BeautifulSoup(response.text, 'html.parser')

# Extract the title
page_title = soup.title.string
print("Page Title:", page_title)

# Extract all hyperlink texts
links = soup.find_all('a')
link_texts = [link.text.strip() for link in links if link.text.strip()]

print("\nHyperlink Texts (First 10):")
print(link_texts[:10])
```

Output:

```
Page Title: Quotes to Scrape
Hyperlink Texts (First 10):
['Quotes to Scrape', 'Login', '(about)', 'change', 'deep-thoughts', 'thinking', 'world', '(about)', 'abilities', 'choices']
```

Activity 5:

Using BeautifulSoup on a provided sample web page, extract a repeated structured element (e.g., a list of items with a title and a link each) into a list of dictionaries.

HTML Page: <https://books.toscrape.com/>

Solution:

```
import requests
from bs4 import BeautifulSoup

url = "https://books.toscrape.com/"
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

# Find all book elements on the page
book_elements = soup.find_all('article', class_='product_pod')
```

```
books_list = []

# Extract title and link for each book
for book in book_elements:
    a_tag = book.h3.a
    title = a_tag['title']
    link = url + a_tag['href']

    books_list.append({
        "title": title,
        "link": link
    })

print(f"Successfully extracted {len(books_list)} books.")
print("First extracted item:", books_list[0])
```

Output:

```
Successfully extracted 20 books.
First extracted item: {'title': 'A Light in the Attic', 'link': 'https://books.toscrape.com/catalogue/a-light-in-the-attic_1000/index.html'}
```

3) Stage **v** (verify)

Home Activities:

Activity 1:

From a multi-page PDF, extract text from all pages, clean the text (remove commas, full-stops, quotations, hyphens, dashes), combine it into one string and report the 10 most frequent words.

Activity 2:

Open a multi-paragraph (or multi-line) .docx file, extract all paragraphs/lines into a list, then build a DataFrame with one row per paragraph. Add `word_count` and `char_count` columns, and sort the DataFrame by `word_count` to find the 3 longest paragraphs.

Activity 3:

Fetch <https://dummyjson.com/users>, parse the `users` array into a DataFrame using the fields `firstName`, `age`, `address.state`, `company.department`. Group the DataFrame by `company.department` and report the average age per department, sorted descending.

Activity 4:

Fetch and parse

[https://en.wikipedia.org/wiki/List_of_countries_by_population_\(United_Nations\)](https://en.wikipedia.org/wiki/List_of_countries_by_population_(United_Nations)) with BeautifulSoup, extract the population table into a DataFrame. Clean the population column (strip commas, convert to integer as the numbers will initially be strings), and report the top 10 most populous countries.

Activity 5:

Scrape <https://quotes.toscrape.com/tag/love/> for quote text and author, storing the results as a list of dictionaries. Find which author appears most often, and compute the average quote length (in characters) across all scraped quotes. Save the raw scraped list to a `.json` file.

4) Stage **a2** (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 3: Regex, Tokenization & Sentence Segmentation

Statement Purpose:

This lab will give you the ability to pull structured information out of raw, unstructured text using regular expressions, and to correctly break text into words and sentences.

Activity Outcomes:

This lab teaches you the following topics:

- regex fundamentals (groups, quantifiers, character classes)
- pattern-based extraction (`re.findall()`) of emails, phone numbers, order IDs, dates, percentages
- word tokenization (NLTK vs. spaCy)
- blank vs. trained spaCy pipelines
- token attributes (`is_alpha`, `like_num`, `like_email`, `is_currency`); sentence segmentation

Instructor Note:

As pre-lab activity, read Chapter 2 ("Words and Tokens") from *Speech and Language Processing*, covering regular expressions and tokenization, and review the corresponding Text Preprocessing material (tokenization, regex) covered by your theory instructor for Unit 2.

1) Stage J (Journey)

Raw text arriving from a customer message, a business report, or a webpage is a flat sequence of characters with no inherent structure a numerical model can consume. Before any statistical or neural method can operate on language, two separate problems must be solved: locating specific, rule-governed patterns embedded in free text (an order number, an email address, a percentage), and splitting the stream into the linguistic units that downstream models actually operate on, namely words and sentences. This lab isolates these two problems and shows that they require fundamentally different tools. Pattern extraction is best solved with **regular expressions**, a formal language for describing sets of strings, while unit segmentation is better solved with **trained tokenizers** that encode linguistic knowledge no fixed pattern can fully capture.

Regular Expressions as a Formal Matching Language

A regular expression defines a language L over an alphabet Σ through a small set of composition rules: concatenation, alternation, and repetition. Practically, this is expressed through three building blocks used throughout the lab activities.

Character classes restrict which symbols may occupy a given position. `\d` matches any digit, equivalent to the set $0,1,\dots,9$, and `\w` matches any word character, equivalent to $a-z,A-Z,0-9$. A custom class such as `[a-zA-Z0-9._%+-]` explicitly enumerates the allowed symbol set for a position, which is why the email pattern in Activity 2 widens beyond `\w` to also permit dots, underscores, and plus signs in the local part of an address.

Quantifiers control repetition. If c denotes a character class, then $c+$ matches one or more repetitions, cn matches exactly n repetitions, and $c2$, matches two or more. The phone pattern `\d{3}[-.\s]?\d{3}[-.\s]?\d{4}` encodes the fixed 3-3-4 digit grouping of a North American phone number, with the optional separator `[-.\s]?` expressed as a quantifier bounded to zero or one occurrence.

Groups, written with parentheses, partition a match into addressable sub-strings without changing what is matched. In the email pattern `(\w+)@(\w+\. \w+)`, group 1 captures the local part and group 2 captures the domain, so `.group(1)` and `.group(2)` retrieve each piece independently from a single match object. This is the mechanism that turns a matched string into structured, field-level data rather than an undifferentiated blob of text.

A pattern is only useful once it is applied against a document. `re.search()` returns the first match as a `Match` object exposing `.group()`, while `re.findall()` returns every non-overlapping match as a list of strings (or tuples, when the pattern contains multiple groups). The lab activities deliberately move from `re.search()` on a single-sentence example to `re.findall()` on multi-line customer messages and business reports, reflecting the shift from validating that a pattern works to applying it at scale across a document.

The Limits of Pattern Matching and the Case for Learned Tokenizers

Regex-based extraction generalizes poorly the moment the input format is not rigidly fixed. A date pattern written as `\d{4}-\d{2}-\d{2}|\d{1,2}/\d{1,2}/\d{4}` only recognizes the two formats it was explicitly told to expect; a third format such as "September 30, 2023" falls outside *L* entirely and produces a silent false negative rather than an error. This brittleness motivates the shift, later in the lab, from regex-only field extraction to **token-attribute filtering**, where a tokenizer's internal classification of each token (`is_alpha`, `like_num`, `like_email`, `is_currency`) is used instead of a hand-written pattern. The tokenizer's classification is more general because it is built on broader heuristics and, in trained pipelines, statistical context, rather than a single fixed string pattern.

Tokenization itself is not a trivial split on whitespace. A naive whitespace splitter would treat "\$20.00." or "Let's" as single indivisible tokens, merging punctuation and currency symbols into the word itself and failing to separate a contraction into its two underlying units. **NLTK's** `word_tokenize()` and **spaCy's** tokenizer both apply rule sets that account for punctuation boundaries, clitics such as 's and n't, and abbreviation patterns, but the two libraries do not apply identical rules, which is why the lab requires a direct side-by-side comparison of their output on the same input rather than assuming interchangeability.

Statistical Components and the Blank-Pipeline Boundary

spaCy separates the **tokenizer**, a deterministic, rule- and exception-driven component, from the **statistical pipeline components** (tagger, parser, named entity recognizer) that require trained weights to produce output. Calling `spacy.blank("en")` constructs only the vocabulary and tokenizer for a language, with an empty processing pipeline attached. Calling `spacy.load("en_core_web_sm")` additionally deserializes pretrained statistical models into the pipeline. This is why, in Activity 5, tokens produced by the blank pipeline carry an empty `pos_` and `ent_type_` string: the tokenizer has correctly segmented the text, but no component downstream has been loaded to assign a part-of-speech tag or an entity label to each token. This distinction matters operationally, since a blank pipeline is lightweight and fast for pure segmentation tasks, while a trained pipeline is required whenever the task depends on linguistic annotation.

Sentence Boundary Ambiguity

Splitting text into sentences cannot rely on periods alone, since a period is overloaded in English: it terminates declarative sentences, but also appears in abbreviations (U.K.), decimal numbers (\$1.2), and initials, none of which mark a sentence boundary. **NLTK's** `sent_tokenize()` uses a pretrained Punkt model that estimates, from statistical cues, whether a given period is sentence-final or not. **spaCy's** sentence segmentation instead derives boundaries from the dependency parse produced by the trained pipeline, treating a sentence as a syntactically complete parse tree rather than inferring boundaries from surface punctuation statistics alone. Because the two approaches rest on different underlying signals, one Punkt-based and one parse-based, they can diverge on ambiguous input,

which is exactly what Activity 7 and the corresponding Stage v activity ask students to observe and reconcile.

Core Libraries and Architectural Roles

`re` is Python's built-in regular expression engine, responsible for compiling and executing all pattern-matching operations in Activities 1 through 3. `nltk` provides `word_tokenize()`, a Penn Treebank-style tokenizer, and `sent_tokenize()`, a Punkt-model sentence segmenter; both require their respective NLTK data packages to be downloaded before use. `spacy` provides the full pipeline architecture used across Activities 4 through 7: `spacy.blank()` for a tokenizer-only pipeline and `spacy.load("en_core_web_sm")` for a small pretrained English pipeline bundling a tokenizer, tagger, parser, and named entity recognizer.

Critical Classes, Functions, and Methods

`re.search(pattern, string)` scans a string and returns the first `Match` object, or `None` if no match exists; `.group(n)` retrieves the full match at `n=0` or a specific parenthesized group at `n>0`. `re.findall(pattern, string)` returns every non-overlapping match as a list, collapsing to tuples of captured groups when the pattern contains more than one group. `word_tokenize(text)` splits a string into a list of word and punctuation tokens using NLTK's pretrained tokenizer rules. `sent_tokenize(text)` splits a string into a list of sentence strings using the Punkt unsupervised sentence boundary model. `spacy.blank(lang)` constructs a `Language` object with only a tokenizer for the given language code, with no statistical components attached. `spacy.load(model_name)` deserializes a named pretrained pipeline package, returning a `Language` object whose `nlp(text)` call produces a `Doc` object. Iterating over a `Doc` yields `Token` objects, each exposing `.text`, `.pos_`, `.ent_type_`, `.is_alpha`, `.like_num`, `.like_email`, and `.is_currency` as read-only attributes computed at processing time. `Doc.sents` is a generator over `Span` objects representing the sentences identified by the pipeline's dependency parse.

Key Parameters and Configurations

Parameter / Attribute	Component	Default Behavior	Operational Impact
<code>pattern</code>	<code>re.search()</code> , <code>re.findall()</code>	No default, required	Defines the exact language matched; overly narrow classes cause silent false negatives
Capturing groups (...)	<code>re</code> pattern syntax	Groups are numbered left to right starting at 1	Determines whether <code>findall()</code> returns full strings or tuples of sub-matches
<code>lang</code>	<code>spacy.blank()</code>	No default, required	Selects only tokenizer and vocabulary rules; no statistical model is loaded
<code>model_name</code>	<code>spacy.load()</code>	No default, required	Determines which statistical components (tagger, parser,

			NER) are available on the resulting Doc
<code>token.pos_</code>	spaCy Token	Empty string if no tagger has run	Non-empty only after a pipeline with a trained tagger has processed the text
<code>token.ent_type_</code>	spaCy Token	Empty string if no NER component has run	Non-empty only after a pipeline with a trained NER component has processed the text
<code>token.is_alpha</code>	spaCy Token	Computed from the token's surface text	True only if every character is alphabetic; excludes tokens with digits or punctuation
<code>token.like_num</code>	spaCy Token	Computed via lexical and shape heuristics	Flags both digit sequences and number words, independent of a fixed digit pattern
<code>token.like_email</code>	spaCy Token	Computed via shape heuristics	Flags tokens matching an email-like shape without relying on an explicit regex
<code>token.is_currency</code>	spaCy Token	Computed from a language-specific currency symbol list	Flags standalone currency symbols, not full amounts like \$500

Implementation Pitfalls, Quirks, and Edge Cases

- The email pattern in Activity 1, `(\w+)@(\w+\.\w+)`, only matches a domain with exactly one dot before the top-level part, so it fails on multi-level domains such as `user@mail.example.co.uk`, unlike the more permissive pattern used in Activity 2.
- The phone pattern `\(?:\d{3}\)?[-.\s]? \d{3}[-.\s]? \d{4}` accepts malformed inputs such as an opening parenthesis with no closing one, because the two `\(?:` and `\)?` quantifiers are independent and not paired as a matched set.
- The revenue pattern `\$[\d, ]+(?:\.\d+)?[KM]?` will match a standalone \$ sign followed by digits even without a decimal component, which can pull in unintended partial numeric tokens from adjacent text if the surrounding context is not tightly bounded.
- `token.is_currency` frequently surprises students expecting it to flag entire monetary amounts; it flags only standalone currency symbol tokens, so `$500` always splits across the currency and number token lists rather than appearing as one filtered unit.
- NLTK's `word_tokenize()` and `sent_tokenize()` both depend on separately downloaded data packages (punkt, and related Punkt resources); omitting this download step produces a runtime lookup error rather than a degraded but working tokenizer.
- Comparing NLTK and spaCy tokenization or sentence output requires exact string or list equality checks to be meaningful; a naive visual comparison can miss subtle differences such as trailing punctuation attached to different tokens across the two outputs.
- The blank pipeline in Activity 5 still tokenizes correctly, since tokenization in spaCy is rule-based rather than statistical, so students should not conclude that an empty `pos_` field indicates a tokenization failure.

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Demonstrate regex fundamentals (groups, quantifiers, and character classes) using a sample sentence of your choice.

Solution:

```
import re

sample_sentence = "The price of the item is $45.99, and my contact email is user2024@example.com."

# Character classes: \d for digits, \w for word characters
# Quantifiers: + for one or more, {min,max} for exact lengths
# Groups: enclosed in parentheses to capture specific parts for extraction
email_pattern = r"(\w+)@(\w+\.\w+)"
price_pattern = r"\$(\d+\.\d{2})"

email_match = re.search(email_pattern, sample_sentence)
if email_match:
    print("Full email matched:", email_match.group(0))
    print("Username group:", email_match.group(1))
    print("Domain group:", email_match.group(2))

price_match = re.search(price_pattern, sample_sentence)
if price_match:
    print("Price matched:", price_match.group(1))
```

Output:

```
Full email matched: user2024@example.com
Username group: user2024
Domain group: example.com
Price matched: 45.99
```

Activity 2:

Extract order numbers, emails, and phone numbers from a given customer message using re.findall().

Solution:

```
import re

customer_message = """
Hello, I am writing to inquire about my recent orders.
My first order number is ORD-58392 and the second one is ORD-99201.
```

You can reach me at john.doe@email.com or contact my alternate email jane_doe123@work.org.
My phone numbers are 555-123-4567 and (800) 555-9999.
"""

```
order_pattern = r"ORD-\d+"
email_pattern = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
phone_pattern = r"\"(?\d{3}\)?[-.\s]?\\d{3}[-.\s]?\\d{4}\"

extracted_orders = re.findall(order_pattern, customer_message)
extracted_emails = re.findall(email_pattern, customer_message)
extracted_phones = re.findall(phone_pattern, customer_message)

print("Extracted Orders:", extracted_orders)
print("Extracted Emails:", extracted_emails)
print("Extracted Phones:", extracted_phones)
```

Output:

```
Extracted Orders: ['ORD-58392', 'ORD-99201']
Extracted Emails: ['john.doe@email.com', 'jane_doe123@work.org']
Extracted Phones: ['555-123-4567', '(800) 555-9999']
```

Activity 3:

Apply regex on a messier business-report excerpt to extract dates, percentages, and revenue figures.

Solution:

```
import re
business_report = """
In Q3, which ended on 2023-09-30, our overall growth was phenomenal.
We saw a 15.5% increase in user engagement and an 8% drop in churn rate compared to
12/31/2022.
Total revenue reached $1.2M in North America and $850,000 in Europe.
"""

# Matches YYYY-MM-DD or MM/DD/YYYY formats
date_pattern = r"\d{4}-\d{2}-\d{2}|\d{1,2}/\d{1,2}/\d{4}"

# Matches integer or decimal numbers followed by a % sign
percentage_pattern = r"\d+(?:\.\d+)?%"

# Matches currency amounts including optional commas, decimals, and K/M modifiers
revenue_pattern = r"\$[\d,]+(?:\.\d+)?[KM]?"

extracted_dates = re.findall(date_pattern, business_report)
extracted_percentages = re.findall(percentages_pattern, business_report)
extracted_revenues = re.findall(revenue_pattern, business_report)

print("Extracted Dates:", extracted_dates)
```

```
print("Extracted Percentages:", extracted_percentages)
print("Extracted Revenues:", extracted_revenues)
```

Output:

```
Extracted Dates: ['2023-09-30', '12/31/2022']
Extracted Percentages: ['15.5%', '8%']
Extracted Revenues: ['$1.2M', '$850,000']
```

Activity 4:

Tokenize a given sentence using both NLTK and spaCy and compare the two outputs.

Solution:

```
import nltk
import spacy
from nltk.tokenize import word_tokenize

nlp = spacy.load("en_core_web_sm")

target_sentence = "Let's test this sentence! It costs $20.00."

# NLTK Tokenization
nltk_tokens = word_tokenize(target_sentence)

# spaCy Tokenization
spacy_document = nlp(target_sentence)
spacy_tokens = [token.text for token in spacy_document]

print("NLTK Tokens:", nltk_tokens)
print("spaCy Tokens:", spacy_tokens)
```

Output:

```
NLTK Tokens: ['Let', "'s", 'test', 'this', 'sentence', '!', 'It', 'costs', '$', '20.00', '.']
spaCy Tokens: ['Let', "'s", 'test', 'this', 'sentence', '!', 'It', 'costs', '$', '20.00', '.']
```

Activity 5:

Compare a blank spaCy pipeline (spacy.blank("en")) against a trained pipeline, and explain why the blank pipeline only tokenizes.

Solution:

```
import spacy

# The blank pipeline initializes the vocabulary and tokenizer for the English
# language.
# It lacks pre-trained statistical components like part-of-speech taggers, parsers,
# or named entity recognizers.
blank_nlp = spacy.blank("en")
```

```
# The trained pipeline includes the tokenizer along with loaded statistical models
(e.g., tagger, parser, NER).
trained_nlp = spacy.load("en_core_web_sm")

sample_text = "Apple is looking at buying a U.K. startup for $1 billion."

blank_document = blank_nlp(sample_text)
trained_document = trained_nlp(sample_text)

print("---- Blank Pipeline Output ----")
for token in blank_document:
    # Tokenizer successfully splits text, but part-of-speech (pos_) and entities
    (ent_type_) are empty.
    print(f"Token: {token.text:10} POS: {token.pos_:5} Entity: {token.ent_type_}")

print("\n--- Trained Pipeline Output ---")
for token in trained_document:
    # Statistical models assign appropriate linguistic features and entity labels.
    print(f"Token: {token.text:10} POS: {token.pos_:5} Entity: {token.ent_type_}")
```

Output:

```
--- Blank Pipeline Output ---
Token: Apple    POS:      Entity:
Token: is       POS:      Entity:
Token: looking  POS:      Entity:
Token: at       POS:      Entity:
Token: buying   POS:      Entity:
Token: a        POS:      Entity:
Token: U.K.     POS:      Entity:
Token: startup  POS:      Entity:
Token: for      POS:      Entity:
Token: $        POS:      Entity:
Token: 1        POS:      Entity:
Token: billion  POS:      Entity:
Token: .        POS:      Entity:

--- Trained Pipeline Output ---
Token: Apple    POS: PROPN Entity: ORG
Token: is       POS: AUX   Entity:
Token: looking  POS: VERB  Entity:
Token: at       POS: ADP   Entity:
Token: buying   POS: VERB  Entity:
Token: a        POS: DET   Entity:
Token: U.K.     POS: PROPN Entity: GPE
Token: startup  POS: NOUN  Entity:
Token: for      POS: ADP   Entity:
Token: $        POS: SYM   Entity: MONEY
Token: 1        POS: NUM   Entity: MONEY
Token: billion  POS: NUM   Entity: MONEY
Token: .        POS: PUNCT Entity:
```

Activity 6:

Filter tokens using token attributes (is_alpha, like_num, like_email, is_currency) instead of regex.

Solution:

```
import spacy

nlp = spacy.load("en_core_web_sm")

unfiltered_text = "Contact me at alice@email.com. The total fee is $500, which
includes 2 extra services."
document = nlp(unfiltered_text)

alphabetic_tokens = []
number_tokens = []
email_tokens = []
```

```

currency_tokens = []

# Group tokens logically based on their built-in spaCy attributes
for token in document:
    if token.is_alpha:
        alphabetic_tokens.append(token.text)
    if token.like_num:
        number_tokens.append(token.text)
    if token.like_email:
        email_tokens.append(token.text)
    if token.is_currency:
        currency_tokens.append(token.text)

print("Alphabetic Tokens:", alphabetic_tokens)
print("Number Tokens:", number_tokens)
print("Email Tokens:", email_tokens)
print("Currency Symbols:", currency_tokens)

```

Output:

```

Alphabetic Tokens: ['Contact', 'me', 'at', 'The', 'total', 'fee', 'is', 'which', 'includes', 'extra', 'services']
Number Tokens: ['500', '2']
Email Tokens: ['alice@email.com']
Currency Symbols: ['$']

```

Activity 7:

Perform sentence segmentation on a given paragraph using both the NLTK and spaCy sentencizers.

Solution:

```

import nltk
import spacy
from nltk.tokenize import sent_tokenize

nlp = spacy.load("en_core_web_sm")

sample_paragraph = "Natural Language Processing is fascinating. It allows computers to understand text. Are you learning NLP right now? I hope so!"

# NLTK Sentence Segmentation
nltk_sentences = sent_tokenize(sample_paragraph)

# spaCy Sentence Segmentation
spacy_document = nlp(sample_paragraph)
spacy_sentences = [sentence.text for sentence in spacy_document.sents]

print("--- NLTK Sentences ---")
for index, sentence in enumerate(nltk_sentences, 1):
    print(f"{index}. {sentence}")

print("\n--- spaCy Sentences ---")
for index, sentence in enumerate(spacy_sentences, 1):

```

```
print(f"{index}. {sentence}")
```

Output:

```
--- NLTK Sentences ---
1. Natural Language Processing is fascinating.
2. It allows computers to understand text.
3. Are you learning NLP right now?
4. I hope so!

--- spaCy Sentences ---
1. Natural Language Processing is fascinating.
2. It allows computers to understand text.
3. Are you learning NLP right now?
4. I hope so!
```

3) Stage v (verify)

Activity 1:

Extract all emails, order numbers, and phone numbers from the given raw text block using regex.

Activity 2:

Extract all currency amounts and numeric quantities from the same text block using token attributes (not regex), and compare the results with the regex-based approach.

Activity 3:

Tokenize the given document using both NLTK and spaCy, and note where the outputs differ (punctuation, contractions).

Activity 4

Write a program to segment a 5–6 sentence paragraph using both the NLTK and spaCy sentencizers, and flag any mismatch between them.

4) Stage a2 (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 4: Text Normalization, Cleaning & Minimum Edit Distance

Statement Purpose:

This lab will give you a reusable text-cleaning pipeline (case folding, stop-word removal, stemming/lemmatization) and an understanding of edit distance as the algorithmic backbone of spelling correction.

Activity Outcomes:

This lab teaches you the following topics:

- Lowercasing and noise removal
- Stop-word removal and its risks (sentiment flips, broken MT, lost chatbot intent)
- Stemming (Porter) vs. lemmatization (spaCy)
- The Minimum Edit Distance (Levenshtein) dynamic-programming algorithm
- Applying edit distance to spelling correction

Instructor Note:

As pre-lab activity, read the remainder of Chapter 2 ("Words and Tokens") from Speech and Language Processing, specifically the text normalization and Minimum Edit Distance sections, plus Appendix D ("Spelling Correction and the Noisy Channel") for how edit distance is used in practice.

1) Stage J (Journey)

This lab builds a complete classical text-normalization pipeline and pairs it with the algorithmic foundation of spelling correction. Students move through four connected engineering problems: reducing raw text to a canonical form, removing high-frequency function words without destroying meaning, collapsing inflected word forms to a shared root, and computing string similarity through dynamic programming. The practical payoff is a reusable cleaning function plus an edit-distance-based spell checker, both of which are building blocks for almost every downstream NLP task covered later in the course, including classification, embeddings, and sequence models. By the end of the lab, students should understand not just how to call `lower()` or `PorterStemmer`, but why each transformation is a lossy operation with measurable trade-offs.

Text Normalization as Vocabulary Compression

Raw text arriving from a CSV column carries variation that has no semantic value for most downstream models: capitalization differences, punctuation marks, and irregular whitespace. A model that treats "Great!" and "great" as two distinct tokens has effectively split one concept across two vocabulary entries, which dilutes the statistics the model can learn from either form. **Text normalization** addresses this by mapping many surface variants to one canonical string.

The mechanism used in this lab is a sequential composition of string transformations: case folding through lowercasing, character-class filtering to strip punctuation, and whitespace collapsing through regular-expression substitution. Each step reduces the size of the observed vocabulary V , defined as the set of unique tokens across the corpus:

$$V = \{w : w \in \text{tokens}(D)\}$$

where D is the document collection. The lab quantifies this compression directly by comparing $|V_{\text{before}}|$ and $|V_{\text{after}}|$ and expressing the difference as a percentage reduction:

$$\text{Reduction}(\%) = \frac{|V_{\text{before}}| - |V_{\text{after}}|}{|V_{\text{before}}|} \times 100$$

This is the first concrete evidence students see that preprocessing is not cosmetic. A smaller, denser vocabulary means fewer parameters to estimate in any statistical or embedding-based model built later in the course, at the cost of discarding information (such as emphasis conveyed by capitalization) that a more sophisticated model might have exploited.

Morphological Reduction: Stemming Against Lemmatization

Inflected forms such as `running`, `ran`, and `runs` share a common lexical meaning but appear as distinct surface tokens. **Morphological reduction** collapses these variants to a shared representation, and the lab contrasts two strategies with fundamentally different mechanisms.

`PorterStemmer` applies a fixed sequence of suffix-stripping rules based on letter patterns, with no reference to a dictionary or to part of speech. It is fast and deterministic but frequently produces strings that are not valid English words, because the algorithm only manipulates surface characters. Lemmatization, performed here through spaCy's `.lemma_` attribute, instead maps a token to its dictionary headword using morphological analysis and, when available, part-of-speech context. This produces linguistically valid output but requires a loaded language model and is computationally heavier per token.

The lab's third activity builds a side-by-side comparison table precisely to expose this divergence: words where the stem and the lemma coincide, and words where the crude suffix-stripping of Porter's algorithm produces a truncated, non-lexical string that the lemmatizer avoids.

Stop-Word Removal and the Risk of Meaning Loss

Stop words are high-frequency function words (determiners, prepositions, conjunctions, and negations) that carry low discriminative value in tasks like topic modeling or keyword extraction, which is why removing them is a common default. The lab uses two different stop-word inventories, `nltk.corpus.stopwords` and spaCy's `Defaults.stop_words`, and both are applied to already-cleaned text through simple list membership tests.

The critical conceptual point developed in Activity 2 is that stop-word lists are blunt instruments. Negation particles such as `not` and `but` are conventionally classified as stop words because they are structurally frequent, yet they carry the entire semantic load in a sentence like *"not bad, but not great either."* Removing them collapses the sentence to *"bad great either,"* which inverts the polarity a sentiment classifier would infer. This demonstrates that stop-word removal must be a task-aware decision, not an unconditional preprocessing default, especially for sentiment analysis, machine translation, and intent detection.

The Minimum Edit Distance Recurrence and Its Application to Spelling Correction

The **Minimum Edit Distance** (Levenshtein distance) between two strings s_1 of length m and s_2 of length n is the minimum number of single-character insertions, deletions, and substitutions required to transform s_1 into s_2 . The lab implements this directly as a dynamic-programming table D of size $(m + 1) \times (n + 1)$, where $D[i][j]$ holds the edit distance between the first i characters of s_1 and the first j characters of s_2 .

The base cases initialize the first row and first column to the cost of pure insertions or deletions:

$$D[i][0] = i, \quad D[0][j] = j$$

The recurrence that fills every remaining cell is:

$$D[i][j] = \min \begin{cases} D[i-1][j] + 1 & \text{(deletion)} \\ D[i][j-1] + 1 & \text{(insertion)} \\ D[i-1][j-1] + \text{cost}(i, j) & \text{(substitution)} \end{cases}$$

where $\text{cost}(i, j) = 0$ if $s_1[i-1] = s_2[j-1]$ and 1 otherwise. The final answer, $D[m][n]$, sits in the bottom-right corner of the table. Each cell depends only on its top, left, and top-left neighbors, which is why the table can be filled in a single pass with two nested loops, giving $O(mn)$ time and space complexity.

Spelling correction reduces to a nearest-neighbor search under this distance metric: for a misspelled token, compute its edit distance against every candidate in a known vocabulary and return the candidate with the smallest distance. This is a direct, if computationally naive, instance of the **noisy channel model**, where the observed misspelling is treated as a corrupted signal and the correction task is to recover the most likely intended word.

Core Libraries and Architectural Roles

`pandas` provides the `DataFrame` structure that holds the reviews and support-chat corpora and is used throughout for column-wise transformation via `.apply()`. `string` supplies the punctuation constant consumed by `str.maketrans()` for character deletion. `re` performs whitespace collapsing through pattern substitution. `nltk` contributes the English stopwords corpus, the `PorterStemmer` class, and the `edit_distance()` function used for both distance computation and nearest-vocabulary lookup. `spacy`, loaded through the `en_core_web_sm` pipeline, supplies an independent stop-word set (`Defaults.stop_words`) and token-level lemmatization through the `.lemma_` attribute, requiring text to pass through the `nlp()` pipeline object to produce a `Doc`.

Critical Classes, Functions, and Methods

`str.translate()` combined with `str.maketrans('', '', string.punctuation)` performs character-level deletion by mapping every punctuation character to `None`; it operates on the whole string in one pass rather than iterating token by token. `re.sub(r'\s+', ' ', text)` replaces any run of one or more whitespace characters with a single space, which normalizes tabs, newlines, and multiple spaces uniformly. `PorterStemmer.stem()` accepts a single lowercase token and returns its rule-reduced stem with no dictionary lookup. `nlp()` (the loaded `spaCy` pipeline) accepts a raw string and returns a `Doc` object whose elements are `Token` objects; `token.lemma_` exposes the lemmatized string form, and `token.text` exposes the original surface form used for stop-word membership checks. `nltk.edit_distance(s1, s2)` computes the Levenshtein distance between two strings using an internal dynamic-programming implementation equivalent to the manually constructed matrix in Activity 4.

Key Parameters and Configurations

Parameter / Argument	Component	Default / Used Value	Operational Impact
<code>string.punctuation</code>	<code>str.maketrans</code>	Fixed ASCII punctuation set	Determines which characters are stripped; does not cover Unicode punctuation or emojis
<code>pattern=r'\s+'</code>	<code>re.sub</code>	One or more whitespace characters	Collapses tabs, newlines, and repeated spaces into a single space
<code>stopwords.words('english')</code>	NLTK	Fixed curated list	Includes negation words, which can remove sentiment-bearing tokens
<code>nlp.Defaults.stop_words</code>	spaCy	Fixed curated list, differs from NLTK's	Produces a different vocabulary reduction than the NLTK list on the same input
<code>substitution_cost</code>	Levenshtein recurrence	1 (implicit)	Assumes all substitutions are equally costly; does not weight by keyboard adjacency or phonetic similarity
<code>insertion_cost / deletion_cost</code>	Levenshtein recurrence	1 each	Symmetric cost model; any asymmetric error pattern (e.g., frequent deletions in fast typing) is not reflected
<code>min_distance</code> initialization	Spelling-suggestion loop	<code>float('inf')</code>	Ensures the first comparison always registers as the current best match

Implementation Pitfalls, Quirks, and Edge Cases

- The stop-word removal and stemming functions split text on whitespace using `.split()` rather than a proper tokenizer, so any punctuation surviving normalization, or tokens attached to unremoved symbols, will not match entries in the stop-word set or vocabulary list and will silently pass through unprocessed.
- The distinct-word extraction in Activity 3 uses a Python `set()`, which has no guaranteed iteration order; the resulting comparison table order will vary across runs and across machines, so results should not be assumed reproducible unless a fixed seed or sorted conversion is applied.
- The full pipeline in Activity 5 filters stop words by checking `token.text`, the raw surface form, against `spacy_stop_words`, but appends `token.lemma_`, the lemmatized form, to the output. This means a token could be excluded based on its unlemmatized form while a different token whose lemma coincidentally matches a stop word is retained, introducing a subtle inconsistency between the filtering and retention steps.

- The spelling-suggestion loop in Activity 6 keeps only the first vocabulary word achieving the minimum distance, since the comparison uses strict `<`. If two vocabulary words are equidistant from the misspelled input, the result silently depends on iteration order over `vocab_list`, and no tie is ever reported to the user.
- Negation-bearing stop words are a recurring correctness trap demonstrated directly in Activity 2: any downstream task sensitive to polarity, intent, or negation scope will be corrupted by unconditional stop-word removal, and the two different stop-word lists used here will not even agree on which words trigger this failure.
- The Levenshtein implementation, both the manual matrix and `nltk.edit_distance`, uses a uniform substitution cost of 1 for every character mismatch. This treats a single-character typo (recieve to receive) identically to a completely different letter substitution, which can make the nearest-match heuristic in Activity 6 choose a linguistically implausible correction when two candidate words happen to sit at the same numeric distance.

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Load the reviews dataset into a pandas DataFrame. Lowercase the text column and strip out punctuation and extra whitespace from every review, storing the result in a new column.

Solution:

```
import pandas as pd
import string
import re

# Load the dataset (assuming reviews.csv is in the same directory)
df = pd.read_csv('reviews.csv')

def clean_text_basic(text):
    # Lowercase the text
    text = text.lower()
    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))
    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Apply the cleaning function to create a new column
df['cleaned_text'] = df['text'].apply(clean_text_basic)
print(df[['text', 'cleaned_text']].head())
```

Output:

```
text \
0 This product is AMAZING!! I've never seen anyt...
1 Worst purchase ever... totally disappointed wi...
2 Not bad, but not great either. Just average I ...
3 I absolutely LOVE this!! Best decision I've ma...
4 Terrible customer service, they never responde...

cleaned_text
0 this product is amazing ive never seen anythin...
1 worst purchase ever totally disappointed with ...
2 not bad but not great either just average i guess
3 i absolutely love this best decision ive made ...
4 terrible customer service they never responded...
```

Activity 2:

On the cleaned text column, remove stop words using the NLTK stop-word list, then separately remove stop words using spaCy's stop-word list. Compare the two outputs for review_id 4 and review_id 5, and identify a review in the dataset where removing stop words changes or weakens the sentiment being expressed.

Solution:

```
import spacy
from nltk.corpus import stopwords

# Load NLTK and spaCy stop words
nltk_stop_words = set(stopwords.words('english'))
nlp = spacy.load('en_core_web_sm')
spacy_stop_words = nlp.Defaults.stop_words

def remove_nltk_stopwords(text):
    words = text.split()
    return ' '.join([w for w in words if w not in nltk_stop_words])

def remove_spacy_stopwords(text):
    words = text.split()
    return ' '.join([w for w in words if w not in spacy_stop_words])

df['nltk_no_stop'] = df['cleaned_text'].apply(remove_nltk_stopwords)
df['spacy_no_stop'] = df['cleaned_text'].apply(remove_spacy_stopwords)

# Compare Review 4 and 5 (index 3 and 4)
print("--- Review 4 ---")
print("Original:", df.loc[3, 'cleaned_text'])
print("NLTK:", df.loc[3, 'nltk_no_stop'])
print("spaCy:", df.loc[3, 'spacy_no_stop'])

print("\n--- Review 5 ---")
print("Original:", df.loc[4, 'cleaned_text'])
print("NLTK:", df.loc[4, 'nltk_no_stop'])
print("spaCy:", df.loc[4, 'spacy_no_stop'])

print("\n--- Sentiment Change Example (Review 3) ---")
print("Original:", df.loc[2, 'cleaned_text'])
```

```
print("NLTK Stopwords Removed:", df.loc[2, 'nltk_no_stop'])
# Explanation: In review 3, the phrase "not bad, but not great either"
# turns into "bad great either" when NLTK removes the negation words ("not",
# "but").
# This completely flips or breaks the intended sentiment.
```

Output:

```
--- Review 4 ---
Original: i absolutely love this best decision ive made all year
NLTK: absolutely love best decision ive made year
spaCy: absolutely love best decision ive year

--- Review 5 ---
Original: terrible customer service they never responded to my emails
NLTK: terrible customer service never responded emails
spaCy: terrible customer service responded emails

--- Sentiment Change Example (Review 3) ---
Original: not bad but not great either just average i guess
NLTK Stopwords Removed: bad great either average guess
```

Activity 3:

Extract the distinct words from all ten reviews (after cleaning). Apply PorterStemmer to each word, and separately apply spaCy lemmatization (.lemma_) to each word. Build a small comparison table showing the original word, its stem, and its lemma.

Solution:

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

# Extract all distinct words from the basic cleaned text
all_words = set(' '.join(df['cleaned_text']).split())

comparison_data = []
for word in list(all_words):
    # Apply stemming
    stem = stemmer.stem(word)

    # Apply lemmatization (spaCy needs a doc object)
    doc = nlp(word)
    lemma = doc[0].lemma_

    comparison_data.append({'Original Word': word, 'Stem (Porter)': stem, 'Lemma
(spaCy)': lemma})

# Build the comparison table
```

```
comparison_df = pd.DataFrame(comparison_data)
print(comparison_df.head(15))
```

Output:

	Original Word	Stem (Porter)	Lemma (spaCy)
0	for	for	for
1	so	so	so
2	purchase	purchas	purchase
3	all	all	all
4	took	took	take
5	year	year	year
6	i	i	I
7	average	averag	average
8	and	and	and
9	emails	email	email
10	expectations	expect	expectation
11	bad	bad	bad
12	absolutely	absolut	absolutely
13	stars	star	star
14	about	about	about

Activity 4:

Using Python NLP library such as nltk and edit_distance function, compute the Levenshtein edit distance and matrix for the misspelled and target pairs ("recieve", "receive") and ("seperate", "separate"). Output the completed dynamic programming matrix along with the final distance for each pair.

Solution:

```
import nltk

word_pairs = [("recieve", "receive"), ("seperate", "separate")]

def generate_edit_distance_matrix(s1, s2):
    m, n = len(s1), len(s2)
    # Initialize a matrix of size (m+1) x (n+1)
    matrix = [[0] * (n + 1) for _ in range(m + 1)]

    # Populate the base cases (top row and first column)
    for i in range(m + 1):
        matrix[i][0] = i
    for j in range(n + 1):
        matrix[0][j] = j

    # Fill the matrix using Levenshtein costs
    for i in range(1, m + 1):
        for j in range(1, n + 1):
```

```

        if s1[i - 1] == s2[j - 1]:
            cost = 0
        else:
            cost = 1 # Substitution cost

        matrix[i][j] = min(matrix[i - 1][j] + 1,      # Deletion
                           matrix[i][j - 1] + 1,    # Insertion
                           matrix[i - 1][j - 1] + cost) # Substitution

    print(f"\nDynamic Programming Matrix for '{s1}' -> '{s2}':")
    for row in matrix:
        print(row)

for w1, w2 in word_pairs:
    # Use NLTK to get the final edit distance
    dist = nltk.edit_distance(w1, w2)
    print(f"\nNLTK Edit Distance for ('{w1}', '{w2}') = {dist}")

    # Generate and print the underlying DP matrix
    generate_edit_distance_matrix(w1, w2)

```

Output:

```

NLTK Edit Distance for ('recieve', 'receive') = 2

Dynamic Programming Matrix for 'recieve' -> 'receive':
[0, 1, 2, 3, 4, 5, 6, 7]
[1, 0, 1, 2, 3, 4, 5, 6]
[2, 1, 0, 1, 2, 3, 4, 5]
[3, 2, 1, 0, 1, 2, 3, 4]
[4, 3, 2, 1, 1, 1, 2, 3]
[5, 4, 3, 2, 1, 2, 2, 2]
[6, 5, 4, 3, 2, 2, 2, 3]
[7, 6, 5, 4, 3, 3, 3, 2]

NLTK Edit Distance for ('seperate', 'separate') = 1

Dynamic Programming Matrix for 'seperate' -> 'separate':
[0, 1, 2, 3, 4, 5, 6, 7, 8]
[1, 0, 1, 2, 3, 4, 5, 6, 7]
[2, 1, 0, 1, 2, 3, 4, 5, 6]
[3, 2, 1, 0, 1, 2, 3, 4, 5]
[4, 3, 2, 1, 1, 2, 3, 4, 4]
[5, 4, 3, 2, 2, 1, 2, 3, 4]
[6, 5, 4, 3, 2, 2, 1, 2, 3]
[7, 6, 5, 4, 3, 3, 2, 1, 2]
[8, 7, 6, 5, 4, 4, 3, 2, 1]

```

Activity 5:

Assemble a single reusable cleaning function that performs lowercasing, punctuation removal, stop-word removal, and lemmatization in one call. Apply it to the entire reviews dataset. Report the total vocabulary size before cleaning and after cleaning, and calculate the percentage reduction.

Solution:

```

def full_text_pipeline(text):
    # Lowercase and remove punctuation
    text = text.lower().translate(str.maketrans('', '', string.punctuation))

    # Process with spaCy for lemmatization and stop-word removal

```

```

doc = nlp(text)
cleaned_tokens = [token.lemma_ for token in doc if token.text not in
spacy_stop_words and token.text.strip() != '']

return ' '.join(cleaned_tokens)

# Calculate vocabulary size before cleaning
raw_words = ' '.join(df['text']).split()
vocab_before = len(set(raw_words))

# Apply the full pipeline
df['fully_processed'] = df['text'].apply(full_text_pipeline)

# Calculate vocabulary size after cleaning
cleaned_words = ' '.join(df['fully_processed']).split()
vocab_after = len(set(cleaned_words))

reduction_percentage = ((vocab_before - vocab_after) / vocab_before) * 100

print(f"Vocabulary Size Before Cleaning: {vocab_before}")
print(f"Vocabulary Size After Cleaning: {vocab_after}")
print(f"Percentage Reduction: {reduction_percentage:.2f}%")
print("\nSample Output:")
print(df[['text', 'fully_processed']].head())

```

Output:

```

Vocabulary Size Before Cleaning: 81
Vocabulary Size After Cleaning: 45
Percentage Reduction: 44.44%

Sample Output:

```

	text	fully_processed
0	This product is AMAZING!! I've never seen anyt...	product amazing ve see like
1	Worst purchase ever... totally disappointed wi...	bad purchase totally disappointed quality
2	Not bad, but not great either. Just average I ...	bad great average guess
3	I absolutely LOVE this!! Best decision I've ma...	absolutely love good decision ve year
4	Terrible customer service, they never responde...	terrible customer service respond email

Activity 6:

Using `nltk.edit_distance`, compute the distance between each word in the misspelled words list and every word in the vocabulary list. For each misspelled word, return the vocabulary word with the smallest edit distance as the suggested correction, and present the results as a table of misspelled word, suggested correction, and distance.

Solution:

```

vocab_list = ["receive", "definitely", "occurred", "separate", "until",

```

```

        "which", "believe", "because", "achieve", "necessary"]
misspelled_words = ["recieve", "definatly", "occured", "seperate", "untill",
"wich"]

corrections = []

for mis in misspelled_words:
    best_match = None
    min_distance = float('inf')

    for word in vocab_list:
        dist = nltk.edit_distance(mis, word)
        if dist < min_distance:
            min_distance = dist
            best_match = word

    corrections.append({
        "Misspelled Word": mis,
        "Suggested Correction": best_match,
        "Edit Distance": min_distance
    })

corrections_df = pd.DataFrame(corrections)
print(corrections_df)

```

Output:

	Misspelled Word	Suggested Correction	Edit Distance
0	recieve	receive	2
1	definatly	definitely	1
2	occured	occurred	1
3	seperate	separate	1
4	untill	until	1
5	wich	which	1

3) Stage v (verify)

Activity 1:

Clean the support_chats dataset end to end (lowercase, remove stop words, lemmatize) using your reusable cleaning function from the lab. Report the vocabulary size before and after cleaning, and the percentage reduction.

Activity 2:

From the cleaned support_chats vocabulary, pick 10 words and apply both PorterStemmer and spaCy lemmatization to each. Flag any word where stemming produced a result that is not a real English word.

Activity 3:

Build a spelling-suggestion function using nltk.edit_distance. Run it against every misspelled word you can find inside the support_chats dataset (for example "pakage", "arived", "chek", "beleive", "definatly", "crashin", "delievery", "wich", "recieved", "occured", "seperate") and return the closest match from the vocabulary list provided in the lab. Present the corrected version of each chat message.

Activity 4

Simulate a simple autocorrect feature for a live chat support tool: for messages 1, 4, 6, and 9 in support_chats, identify the misspelled or informal words, correct them using your edit-distance-based suggestion function, and produce a fully corrected version of each of the four messages as the system would show it to a support agent.

4) Stage a2 (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 5: N-gram Language Models & Smoothing

Statement Purpose:

This lab will give you a working understanding of how a language model assigns probability to a sentence, why naive counting fails on unseen data, and how smoothing techniques fix that.

Activity Outcomes:

This lab teaches you the following topics:

- Bigram/trigram language models
- Sentence probability and perplexity
- Data sparseness
- Laplace (add-one) and Add-k smoothing
- Kneser-Ney and Witten-Bell/interpolation smoothing (applied via NLTK's built-in classes)
- Comparing smoothing methods by perplexity

Instructor Note:

As pre-lab activity, read Chapter 3 ("N-gram Language Models") from Speech and Language Processing in full, and Appendix C ("Kneser-Ney Smoothing") for the discounting/continuation-probability intuition. This is essential before the lab, as the graded task assumes you already understand what perplexity measures.

1) Stage J (Journey)

This lab moves students from counting words to modeling language probabilistically. The engineering objective is to build a functioning **n-gram language model**, expose the catastrophic failure mode of raw maximum-likelihood counting when it meets unseen data, and then repair that failure through a sequence of smoothing techniques of increasing sophistication. By the end, students can fit bigram and trigram models with `nltk.lm`, compute sentence probability and perplexity, and use perplexity as a quantitative basis for choosing among five distinct smoothing strategies. This is the first point in the course where a model produces a genuine probability distribution over language rather than a fixed similarity score, which makes it the conceptual bridge into all later probabilistic and neural language modeling.

From Word Counts to a Probability Distribution over Sequences

A language model assigns a probability to a sequence of words $w_1, w_2, \dots, w_T$. By the chain rule of probability, this joint probability decomposes into a product of conditional probabilities:

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

Conditioning on the entire history is intractable, since the number of possible histories grows exponentially with vocabulary size and sequence length. The **n-gram assumption** (a Markov assumption of order $n-1$) truncates the conditioning context to the previous $n - 1$ words:

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

For a bigram model ($n = 2$), this reduces to $P(w_t \mid w_{t-1})$, estimated from a training corpus through relative frequency counting:

$$P(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})}$$

where $C(\cdot)$ denotes a count of occurrences in the training corpus. This is exactly what `nltk.lm.MLE` implements: it stores conditional frequency distributions built from the padded n-grams produced by `padded_everygram_pipeline`, and `.score()` performs the count ratio above. The trigram model in Activity 2 extends the same mechanism to a two-word context, $P(w_t \mid w_{t-2}, w_{t-1})$, which sharpens the conditioning but also sharpens the sparsity problem, discussed next.

Data Sparseness and the Zero-Probability Catastrophe

Any bigram (w_{t-1}, w_t) that never appears in the training corpus receives $C(w_{t-1}, w_t) = 0$, and therefore $P(w_t \mid w_{t-1}) = 0$ under raw maximum-likelihood estimation. Because sentence

probability is a product of these conditional terms, a single zero-probability bigram forces the entire sentence probability to zero, regardless of how plausible every other word transition was. This is the **data sparseness** problem: natural language follows a heavy-tailed (Zipfian) distribution, so no training corpus, however large, will contain every plausible n-gram that appears in held-out text.

The consequence surfaces directly in **perplexity**, the standard evaluation metric. For a test sequence of N words, perplexity is defined as the inverse probability of the sequence, normalized by sequence length:

$$PP(W) = P(w_1, \dots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1, \dots, w_N)}}$$

Perplexity can be interpreted as the weighted average branching factor the model faces at each prediction step: a lower perplexity means the model is, on average, less surprised by the actual next word. When $P(W) = 0$, perplexity is mathematically infinite, which is exactly the output `mle_bigram.perplexity()` returns whenever the held-out sentence contains an unseen n-gram. This is not a numerical quirk; it is the correct mathematical consequence of assigning literal zero probability to an event that did occur, and it makes unsmoothed MLE models unusable on any but the training data itself. Perplexity remains the standard intrinsic yardstick even for modern large language models, since it measures calibration of the probability distribution independent of any downstream task.

Smoothing as Probability Mass Redistribution

Smoothing techniques solve the zero-probability problem by reserving a small amount of probability mass for unseen events, taken from the mass otherwise assigned to seen events. The lab progresses through five methods in order of increasing sophistication.

Laplace (add-one) smoothing adds a constant of 1 to every count before normalizing:

$$P_{\text{Laplace}}(w_t | w_{t-1}) = \frac{C(w_{t-1}, w_t) + 1}{C(w_{t-1}) + V}$$

where V is the vocabulary size. No bigram can have zero probability anymore, but because V is typically large, this method redistributes a disproportionately large share of probability mass away from frequently seen bigrams, which tends to overcorrect and inflate perplexity relative to more targeted methods.

Add-k (Lidstone) smoothing generalizes Laplace by adding a fractional constant k instead of 1:

$$P_{\text{Add-k}}(w_t | w_{t-1}) = \frac{C(w_{t-1}, w_t) + k}{C(w_{t-1}) + kV}$$

`nlTK.lm.Lidstone(gamma=k, order=n)` implements this directly. Smaller k values steal less mass from observed bigrams, typically producing lower perplexity than Laplace, as the lab's Activity 4 demonstrates numerically.

Kneser-Ney smoothing, implemented through `KneserNeyInterpolated`, improves on both by using **absolute discounting**: it subtracts a fixed small discount d from every nonzero count rather than a multiplicative or additive adjustment, and redistributes the freed mass according to a **continuation probability**, which estimates how likely a word is to appear as a novel continuation after different contexts, rather than its raw frequency. This corrects a known flaw in simpler back-off schemes, where a low-frequency word like "Francisco" gets over-credited for unseen contexts simply because it is common in one specific bigram ("San Francisco").

Witten-Bell interpolation, implemented through `WittenBellInterpolated`, instead estimates the probability of an unseen continuation from how many distinct word types have followed a given context, interpolating between the higher-order and lower-order (backed-off) distributions weighted by this diversity count.

Both Kneser-Ney and Witten-Bell are **interpolation** methods, meaning they combine the higher-order model estimate with a lower-order model estimate rather than sharply backing off to zero, which is why they consistently outperform Laplace and Add-k on real text, as reflected in the comparison table built in Activity 5.

Core Libraries and Architectural Roles

`nlTK.lm` supplies the model classes `MLE`, `Laplace`, `Lidstone`, `KneserNeyInterpolated`, and `WittenBellInterpolated`, all sharing a common `.fit()` and `.score()` interface, which lets the lab swap smoothing strategies without altering the surrounding pipeline code. `nlTK.lm.preprocessing.padded_everygram_pipeline` prepares the training data by generating all n -grams up to the specified order together with sentence-boundary padding, and simultaneously builds the vocabulary object required by `.fit()`. `nlTK.util.ngrams` extracts fixed-order n -gram tuples from a token sequence, used here to prepare the held-out sentence in the exact same format the model expects at scoring time. `nlTK.tokenize.word_tokenize` performs the initial tokenization of both the training corpus and the held-out sentence. `pandas` is used only in later activities to assemble the multi-method perplexity comparison table.

Critical Classes, Functions, and Methods

`padded_everygram_pipeline(n, corpus)` returns two objects: a lazy iterator of training n -grams (padded with start/end tokens up to order n) and a `Vocabulary` object built from the same corpus; both are consumed exactly once by `.fit()`, which is why the lab regenerates this pipeline call before fitting each new smoothing method. `pad_both_ends(tokens, n)` inserts $n-1$ boundary symbols at the start and end of a token sequence so that n -grams spanning sentence edges are well defined. `MLE.score(word, context)` and its counterparts on the smoothed classes accept a target word and

a list of preceding context words, returning the conditional probability $P(\text{word} \mid \text{context})$ under that model's estimation rule. `Model.perplexity(ngrams)` accepts an iterable of n-gram tuples (not raw tokens) and internally computes the geometric mean of the inverse per-n-gram probabilities. `Lidstone(gamma, order)` takes the fractional smoothing constant `gamma` as its defining hyperparameter, distinguishing it from `Laplace`, which fixes this constant to 1 internally.

Key Parameters and Configurations

Parameter / Argument	Component	Default / Used Value	Operational Impact
<code>n (order)</code>	<code>padded_everygram_pipeline</code> , <code>MLE</code> , <code>Laplace</code> , etc.	2 (bigram), 3 (trigram)	Determines context window size; higher order increases sparsity and zero-probability risk
<code>gamma</code>	<code>Lidstone</code>	0.5	Fractional smoothing constant; smaller values steal less probability mass from seen n-grams
Implicit add constant	<code>Laplace</code>	1	Equivalent to <code>Lidstone(gamma=1)</code> ; tends to over-smooth with large vocabularies
Discount <code>d</code>	<code>KneserNeyInterpolated</code>	NLTK internal default	Absolute discount subtracted from nonzero counts before redistribution
context list length	<code>.score()</code>	<code>n - 1</code> words	Must match the model's order; mismatched context length produces incorrect or erroring calls
Padding symbols	<code>pad_both_ends</code>	NLTK default start/end tokens	Required so sentence-boundary n-grams are scored consistently across all five models

Implementation Pitfalls, Quirks, and Edge Cases

- The generator objects returned by `padded_everygram_pipeline` are consumed exactly once by `.fit()`. Reusing a `train_data` iterator across two different model instances silently yields an empty or partially trained model on the second call, which is why the lab regenerates `train_data` and `vocab` before fitting each new smoothing method; forgetting this step is a common source of silently wrong results rather than an explicit error.
- `MLE.perplexity()` returning `inf` is mathematically correct, not a bug, but students frequently mistake it for an implementation error and attempt to "fix" it by altering the scoring code rather than understanding that this is the expected behavior of unsmoothed counting on unseen data.
- The best-method selection in Activity 6 hardcodes `best_model = wb_bigram` with a comment instructing students to manually change it if Kneser-Ney scored better in their run. This is a manual, error-prone step: if a student's corpus produces different relative perplexities than the

reference run, and they forget to update this assignment, all downstream scoring in Activity 6 silently uses the wrong model without any warning.

- Tokenization inconsistency is a latent risk: the training corpus is tokenized with `word_tokenize` while lowercased at load time via a list comprehension, but the held-out sentence is tokenized without an explicit lowercasing step applied in the same location. If the held-out sentence contains any uppercase characters, it can introduce spurious out-of-vocabulary tokens unrelated to genuine data sparsity, contaminating the perplexity comparison with a preprocessing artifact rather than a genuine smoothing effect.
- The interpolation-based models (`KneserNeyInterpolated`, `WittenBellInterpolated`) never assign literal zero probability to any word in the vocabulary, including words that never appeared in any training context. This means a nonsensical bigram can still receive a small nonzero probability, and perplexity alone will not flag this; low perplexity on a held-out sentence does not guarantee that the model's individual next-word rankings are linguistically sensible, which is a limitation worth surfacing before students over-interpret the Activity 5 comparison table as a complete quality measure.

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Using the `movie_corpus` dataset, build a bigram language model with `nlk.lm` (no smoothing). Compute the probability and perplexity of the held-out sentence under this model, and report where the model assigns zero probability.

Solution:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.lm.preprocessing import padded_everygram_pipeline, pad_both_ends
from nltk.util import ngrams
from nltk.lm import MLE

# Load and tokenize the corpus
with open('movie_corpus.txt', 'r') as f:
    corpus = [line.strip().lower() for line in f if line.strip()]

tokenized_corpus = [word_tokenize(sentence) for sentence in corpus]

# Held-out sentence
held_out = "the movie was fantastic and the acting was convincing"
tokenized_held_out = word_tokenize(held_out)

# Build MLE Bigram Model
train_data, vocab = padded_everygram_pipeline(2, tokenized_corpus)
mle_bigram = MLE(2)
mle_bigram.fit(train_data, vocab)
```

```

# Prepare held-out n-grams for bigram model
held_out_padded = list(pad_both_ends(tokenized_held_out, n=2))
held_out_bigrams = list(ngrams(held_out_padded, 2))

# Compute probability and find zero-prob n-grams
prob = 1.0
print("Zero probability assigned to:")
for bg in held_out_bigrams:
    # NLTK score() takes the target word and a list of context words
    p = mle_bigram.score(bg[1], [bg[0]])
    prob *= p
    if p == 0.0:
        print(f"- {bg}")

# Compute perplexity
perplexity = mle_bigram.perplexity(held_out_bigrams)

print(f"\nSentence Probability: {prob}")
print(f"Sentence Perplexity: {perplexity}")

```

Output:

```

Zero probability assigned to:
- ('was', 'fantastic')
- ('and', 'the')
- ('acting', 'was')
- ('was', 'convincing')

Sentence Probability: 0.0
Sentence Perplexity: inf

```

Activity 2:

Build a trigram language model on the same corpus. Compute the probability and perplexity of the same held-out sentence, and compare the result to the bigram model's output from Activity 1.

Solution:

```

# Build MLE Trigram Model
# Note: We must recreate the pipeline iterators for each new model in NLTK
train_data_3, vocab_3 = padded_everygram_pipeline(3, tokenized_corpus)
mle_trigram = MLE(3)
mle_trigram.fit(train_data_3, vocab_3)

# Prepare held-out n-grams for trigram model
held_out_padded_3 = list(pad_both_ends(tokenized_held_out, n=3))
held_out_trigrams = list(ngrams(held_out_padded_3, 3))

# Compute probability

```

```

prob_3 = 1.0
for tg in held_out_trigrams:
    # context is a list of the preceding n-1 words
    p = mle_trigram.score(tg[2], [tg[0], tg[1]])
    prob_3 *= p

# Compute perplexity
perplexity_3 = mle_trigram.perplexity(held_out_trigrams)

print(f"Trigram Sentence Probability: {prob_3}")
print(f"Trigram Sentence Perplexity: {perplexity_3}")

print("\nComparison:")
print("Both the bigram and trigram models fail (probability 0.0, perplexity inf)
because they encounter unseen n-grams in the held-out sentence. The trigram model
is even sparser, meaning it will suffer from this zero-probability problem more
frequently than the bigram model.")

```

Output:

```

Trigram Sentence Probability: 0.0
Trigram Sentence Perplexity: inf

Comparison:
Both the bigram and trigram models fail (probability 0.0, perplexity inf) because they encounter unseen n-grams in the hel

```

Activity 3:

Apply Laplace (add-one) smoothing to the bigram model. Recompute the perplexity of the held-out sentence and explain, using the actual numbers you obtained, how the zero-probability problem from Activity 1 was resolved.

Solution:

```

from nltk.lm import Laplace

# Build Laplace Bigram Model
train_data_laplace, vocab_laplace = padded_everygram_pipeline(2, tokenized_corpus)
laplace_bigram = Laplace(2)
laplace_bigram.fit(train_data_laplace, vocab_laplace)

# Perplexity using the previously padded bigrams
laplace_perplexity = laplace_bigram.perplexity(held_out_bigrams)

print(f"Laplace Smoothed Perplexity: {laplace_perplexity}")

print("\nExplanation:")
print("In Activity 1, unseen bigrams like ('was', 'fantastic') were assigned a
probability of 0, driving the entire sentence probability to 0 and perplexity to
infinity.")
print(f"Laplace smoothing adds 1 to all bigram counts (and V to the denominator).
Because of this, no bigram has a zero count, allowing the model to return a finite,
valid perplexity score of {laplace_perplexity:.2f} instead of 'inf'.")

```

Output:

```
Laplace Smoothed Perplexity: 109.40655568155587
```

Explanation:

In Activity 1, unseen bigrams like ('was', 'fantastic') were assigned a probability of 0, driving the entire sentence probability to 0. Laplace smoothing adds 1 to all bigram counts (and V to the denominator). Because of this, no bigram has a zero count, allowing the model to assign a non-zero probability to all bigrams.

Activity 4:

Apply Add-k smoothing with $k = 0.5$ to the bigram model. Recompute the perplexity of the held-out sentence and compare it against the Laplace-smoothed perplexity from Activity 3.

Solution:

```
from nltk.lm import Lidstone
```

```
# Build Add-k (Lidstone) Bigram Model with k=0.5
```

```
train_data_addk, vocab_addk = padded_everygram_pipeline(2, tokenized_corpus)
```

```
addk_bigram = Lidstone(gamma=0.5, order=2)
```

```
addk_bigram.fit(train_data_addk, vocab_addk)
```

```
addk_perplexity = addk_bigram.perplexity(held_out_bigrams)
```

```
print(f"Laplace (k=1) Perplexity: {laplace_perplexity}")
```

```
print(f"Add-k (k=0.5) Perplexity: {addk_perplexity}")
```

```
print("\nComparison:")
```

```
print("Add-0.5 smoothing generally yields a lower (better) perplexity than Add-1 (Laplace) because Add-1 steals too much probability mass from seen n-grams to give to unseen n-grams. By only adding 0.5, the model balances the known distribution better.")
```

Output:

```
Laplace (k=1) Perplexity: 109.40655568155587
```

```
Add-k (k=0.5) Perplexity: 88.30779701698535
```

Comparison:

Add-0.5 smoothing generally yields a lower (better) perplexity than Add-1 (Laplace) because Add-1 steals too much probability mass from seen n-grams to give to unseen n-grams. By only adding 0.5, the model balances the known distribution better.

Activity 5:

Using NLTK's built-in classes, apply Kneser-Ney smoothing and Witten-Bell interpolation separately to the bigram model. Compute the perplexity of the held-out sentence under each, then build a single comparison table listing the perplexity of all five approaches so far (unsmoothed, Laplace, Add-k, Kneser-Ney, Witten-Bell) and identify the best-performing method.

Solution:

```
from nltk.lm import KneserNeyInterpolated, WittenBellInterpolated
import pandas as pd
```

```
# 1. Kneser-Ney Smoothing
```

```
train_data_kn, vocab_kn = padded_everygram_pipeline(2, tokenized_corpus)
```

```

kn_bigram = KneserNeyInterpolated(2)
kn_bigram.fit(train_data_kn, vocab_kn)
kn_perplexity = kn_bigram.perplexity(held_out_bigrams)

# 2. Witten-Bell Interpolation
train_data_wb, vocab_wb = padded_everygram_pipeline(2, tokenized_corpus)
wb_bigram = WittenBellInterpolated(2)
wb_bigram.fit(train_data_wb, vocab_wb)
wb_perplexity = wb_bigram.perplexity(held_out_bigrams)

# Compile comparison table
results = {
    "Method": ["Unsmoothed (MLE)", "Laplace (Add-1)", "Add-k (k=0.5)", "Kneser-
Ney", "Witten-Bell"],
    "Perplexity": [perplexity, laplace_perplexity, addk_perplexity, kn_perplexity,
wb_perplexity]
}

df_results = pd.DataFrame(results)
print(df_results.to_string(index=False))

# Identify best performing
best_method = df_results.loc[df_results['Perplexity'] != float('inf')].nsmallest(1,
'Perplexity')
print(f"\nBest Performing Method (Lowest Perplexity):
{best_method['Method'].values[0]}")

```

Output:

```

      Method  Perplexity
Unsmoothed (MLE)      inf
Laplace (Add-1)  109.406556
Add-k (k=0.5)    88.307797
Kneser-Ney      38.903316
Witten-Bell      25.881090

Best Performing Method (Lowest Perplexity): Witten-Bell

```

Activity 6:

Using the best-performing smoothing method from Activity 5, take the partial sentence "the acting in this film was quite" and score two candidate next words, "disappointing" and "wonderful", by computing the probability of each completed sentence. Report which candidate the model prefers and relate this to how a real keyboard-suggestion feature would use this scoring.

Solution:

```

# Assuming Witten-Bell or Kneser-Ney performed best, we will use the wb_bigram
# (change to kn_bigram if that was lower in your specific run)
best_model = wb_bigram

```

```

def get_sentence_prob(sentence, model):
    tokens = word_tokenize(sentence)
    padded = list(pad_both_ends(tokens, n=2))
    ngrams_list = list(ngrams(padded, 2))

    prob = 1.0
    for bg in ngrams_list:
        prob *= model.score(bg[1], [bg[0]])
    return prob

base_sentence = "the acting in this film was quite"

sentence_a = base_sentence + " disappointing"
prob_a = get_sentence_prob(sentence_a, best_model)

sentence_b = base_sentence + " wonderful"
prob_b = get_sentence_prob(sentence_b, best_model)

print(f"Candidate 1 ('disappointing'): {prob_a}")
print(f"Candidate 2 ('wonderful'): {prob_b}")

if prob_a > prob_b:
    print("\nThe model prefers: 'disappointing'")
else:
    print("\nThe model prefers: 'wonderful'")

print("\nReal Keyboard-Suggestion Connection:")
print("A predictive text feature evaluates the probability of candidate next words
given the recent context (e.g., the last typed word). It computes these
probabilities behind the scenes and displays the top 1-3 words with the highest
likelihood to the user above the keyboard.")

```

Output:

```

Candidate 1 ('disappointing'): 2.5887280536738476e-08
Candidate 2 ('wonderful'): 6.034331127444865e-11

The model prefers: 'disappointing'

Real Keyboard-Suggestion Connection:
A predictive text feature evaluates the probability of candidate next words given the recent context (

```

3) Stage v (verify)

Activity 1:

On the `product_reviews_corpus` dataset, fit two smoothing methods of your choice on a bigram model and report the perplexity of each on the held-out sentence above. State which smoothing method performed better and justify your choice using the numbers.

Activity 2:

Build a simple next-word predictor for a review-writing assistant: given the partial sentence "the camera quality exceeded my", use your best-performing smoothed model to rank three candidate next words: "expectations", "budget", and "screen". Report the ranked probabilities and which word the assistant would suggest first.

Activity 3:

Train both a bigram and a trigram model (using your chosen smoothing method) on the `product_reviews_corpus` dataset. Compute the perplexity of each on the held-out sentence and explain, based on the actual perplexity values, which model generalizes better to this unseen sentence.

Activity 4:

Identify a word in the held-out sentence that does not appear anywhere in the `product_reviews_corpus` training data. Compute the probability of the sentence containing this out-of-vocabulary word under an unsmoothed model versus your best smoothed model, and report both perplexity values.

4) Stage a2 (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 6: Text Classification: Naive Bayes & Logistic Regression

Statement Purpose:

This lab will give you a complete, evaluable text classification pipeline, from raw text to a trained model to a properly interpreted set of evaluation metrics. By the end you will be able to take any labeled text dataset and build, evaluate, and package a working classifier.

Activity Outcomes:

This lab teaches you the following topics:

- Bag-of-Words representation using CountVectorizer, and inspecting the resulting vocabulary and sparse matrix
- TF-IDF weighting and how it differs from raw counts
- Train/test splitting for text data
- Training and comparing Naive Bayes and Logistic Regression classifiers
- Evaluation metrics: accuracy, precision, recall, F1, and confusion matrix
- Bundling a vectorizer and classifier into a single sklearn Pipeline
- Recognizing and handling class imbalance in real datasets

Instructor Note:

As pre-lab activity, read Chapter 4 ("Logistic Regression and Text Classification") from Speech and Language Processing, along with Appendix B ("Naive Bayes Classification"), and Chapter 4 ("Text Classification") from Practical Natural Language Processing for the applied, pipeline-oriented view of the same material.

1) Stage J (Journey)

This lab builds the first complete, evaluable machine learning pipeline in the course: raw text goes in, a trained classifier comes out, and its quality is measured with metrics that actually mean something for imbalanced, real-world data. The engineering goal is to convert unstructured review text into fixed-length numerical vectors, train two structurally different classifiers on those vectors, and compare them using accuracy, precision, recall, F1, and a confusion matrix rather than a single misleading number. Students also learn to package the entire process, vectorizer and classifier together, into a single `sklearn.pipeline.Pipeline` object, which is how production text classification systems are actually deployed rather than run as disconnected manual steps.

From Text to Fixed-Length Vectors: Bag-of-Words and TF-IDF

Classical machine learning classifiers require input as fixed-length numerical vectors, but a raw review is a variable-length sequence of characters. The **Bag-of-Words (BoW)** representation solves this by building a vocabulary V of all unique tokens across the corpus and representing each document d as a vector of length $|V|$, where each entry counts how many times that vocabulary word appears in d :

$$\mathbf{x}_d = [C(w_1, d), C(w_2, d), \dots, C(w_{|V|}, d)]$$

`CountVectorizer` implements exactly this: `fit_transform()` learns the vocabulary from the training corpus and simultaneously produces a sparse document-term matrix, where sparsity reflects the fact that any single document uses only a small fraction of the full vocabulary. The word order is entirely discarded, which is the defining limitation of the bag-of-words assumption.

Raw counts have a critical flaw: extremely frequent words like `the` or `was` dominate the vector's magnitude without carrying much class-discriminative information. **TF-IDF (Term Frequency-Inverse Document Frequency)** corrects this by down-weighting words that occur across many documents. For a term t in document d , drawn from a corpus of N documents:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

$$\text{IDF}(t) = \log\left(\frac{N}{\text{DF}(t)}\right)$$

where $\text{TF}(t, d)$ is the raw or normalized frequency of t in d , and $\text{DF}(t)$ is the number of documents in which t appears at least once. A word appearing in nearly every document has $\text{DF}(t) \approx N$, driving $\text{IDF}(t)$ toward zero and suppressing its weight regardless of how often it occurs locally. This is precisely why a common word's TF-IDF weight comes out lower than its raw count would suggest: the IDF term is actively penalizing its lack of discriminative power.

Generative Versus Discriminative Classification

The lab trains two classifiers on the same TF-IDF features to expose a fundamental modeling distinction. **Multinomial Naive Bayes** is a generative classifier: it models the joint probability $P(\mathbf{x}, y)$ by learning $P(\mathbf{x} | y)$ for each class and combining it with the class prior $P(y)$ through Bayes' rule:

$$P(y | \mathbf{x}) \propto P(y) \prod_{i=1}^{|\mathcal{V}|} P(w_i | y)^{x_i}$$

This formula rests on the **conditional independence assumption**, treating each word's occurrence as independent of every other word given the class label. This assumption is almost never literally true for natural language, yet Naive Bayes remains a strong baseline because it needs relatively little training data to estimate its per-class word probabilities reliably.

Logistic Regression is discriminative: rather than modeling how each class generates data, it directly models the decision boundary by learning a weight vector $\mathbf{w}$ and bias b that map a feature vector to a class probability through the sigmoid function:

$$P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

Logistic Regression makes no independence assumption about features and can, in principle, learn more nuanced decision boundaries, but it typically needs more training data to do so reliably and is more sensitive to feature scaling and correlated features. The lab's side-by-side comparison in Activity 4 is designed to let students observe this trade-off empirically rather than take it on faith.

Evaluation Beyond Accuracy: Precision, Recall, F1, and the Confusion Matrix

Accuracy, the proportion of correct predictions, becomes dangerously uninformative when one class dominates the dataset. A classifier that always predicts the majority class can achieve high accuracy while being completely useless for detecting the minority class, which is exactly the risk flagged conceptually in this lab and addressed concretely in the Stage v class-imbalance activity.

The **confusion matrix** breaks predictions into four counts: true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN), relative to a chosen positive class. From these:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision answers "of everything flagged as positive, how much was actually positive," while recall answers "of everything that was actually positive, how much did the model catch." These two metrics trade off against each other, and F_1 summarizes the trade-off as a single harmonic mean. `precision_score`, `recall_score`, and `f1_score` all require a `pos_label` argument specifying which class is treated as positive, since the same confusion matrix yields different precision and recall values depending on that choice.

Pipelines as the Deployment Unit

A vectorizer and a classifier are always deployed together, because a trained model is meaningless without the exact same feature transformation that produced its training vectors. `sklearn.pipeline.Pipeline` chains a `TfidfVectorizer` step and a classifier step into a single object with a unified `.fit()` and `.predict()` interface. Calling `.fit()` on the pipeline fits the vectorizer on the training text and then fits the classifier on the resulting vectors in one call; calling `.predict()` on new raw text automatically applies the same fitted vectorizer transformation before passing vectors to the classifier. This eliminates an entire class of production bugs where a model is served with a vectorizer that was fit on different data or with different parameters than the one used during training.

Core Libraries and Architectural Roles

`pandas` loads the review dataset into a `DataFrame` and supplies `.value_counts()` for inspecting class distribution in the Stage v activities. `sklearn.feature_extraction.text` supplies `CountVectorizer` for raw bag-of-words vectors and `TfidfVectorizer` for weighted vectors, both exposing a `.vocabulary_` dictionary mapping tokens to column indices. `sklearn.model_selection.train_test_split` partitions data into training and test sets with a fixed `random_state` for reproducibility. `sklearn.naive_bayes.MultinomialNB` and `sklearn.linear_model.LogisticRegression` provide the two classifier implementations trained on identical features for a fair comparison. `sklearn.metrics` supplies `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, and `confusion_matrix` for the evaluation stage. `sklearn.pipeline.Pipeline` bundles vectorization and classification into a deployable unit.

Critical Classes, Functions, and Methods

`CountVectorizer.fit_transform()` learns the vocabulary from the training corpus and returns a sparse matrix in Compressed Sparse Row format; `.vocabulary_` exposes the learned token-to-index mapping used to look up specific word columns. `TfidfVectorizer.fit_transform()` performs the same fitting step but weights each entry by the TF-IDF formula rather than raw counts; because it is a separate vectorizer instance from `CountVectorizer`, the column index for a given word is not guaranteed to match between the two unless the vocabulary happens to be identical. `train_test_split(X, y, test_size, random_state)` performs a randomized split with `test_size` controlling the held-out fraction and `random_state` fixing the shuffle for reproducibility

across reruns. `MultinomialNB.fit()` estimates per-class word probabilities from count-like input; note it is designed for non-negative count or frequency features, which TF-IDF values satisfy despite not being integer counts. `LogisticRegression.fit()` optimizes the weight vector via an iterative solver rather than a closed-form solution. `confusion_matrix(y_true, y_pred, labels=[...])` requires an explicit `labels` argument to fix row and column ordering; omitting it lets the function infer label order from the data, which can silently transpose the matrix's interpretation across different runs or datasets.

Key Parameters and Configurations

Parameter / Argument	Component	Default / Used Value	Operational Impact
<code>test_size</code>	<code>train_test_split</code>	0.2	Fraction of data held out for evaluation; smaller test sets give noisier metric estimates
<code>random_state</code>	<code>train_test_split</code>	42	Fixes the shuffle so results are reproducible across runs
<code>pos_label</code>	<code>precision_score</code> , <code>recall_score</code> , <code>f1_score</code>	'positive'	Determines which class's errors are being measured; flipping it changes all three values
<code>labels</code>	<code>confusion_matrix</code>	['negative', 'positive']	Fixes row/column class order; omitting it risks an inconsistently ordered matrix
<code>class_weight</code>	LogisticRegression (Stage v)	'balanced'	Reweights the loss inversely to class frequency, counteracting majority-class dominance
Vectorizer choice inside Pipeline	Pipeline steps	TfidfVectorizer	Determines the feature representation applied uniformly at both train and inference time

Implementation Pitfalls, Quirks, and Edge Cases

- Activity 2 compares raw counts and TF-IDF values by indexing into `x_bow` and `x_tfidf` using the same `idx1/idx2` values retrieved from `tfidf_vectorizer.vocabulary_`, but this index is only guaranteed valid for `x_tfidf`. `CountVectorizer` and `TfidfVectorizer` are separate fitted instances, and although they will typically learn the same vocabulary from identical input text with default settings, relying on this coincidence rather than looking up each word's index in its own vectorizer's vocabulary is fragile and can silently produce a mismatched comparison if either vectorizer's parameters diverge.
- Activity 3 trains both classifiers on `x_tfidf`, which was produced by fitting `TfidfVectorizer` on the entire dataset before the train/test split. This is a data leakage risk: IDF statistics computed from the full corpus, including documents that end up in the test set, leak information about the test set's vocabulary distribution into the training-time feature weights. The correct approach is to fit the vectorizer only on the training split and transform the test split with those already-fitted statistics, which Activity 5's pipeline-based fit does implicitly correctly by fitting on

`x_train_text` alone, exposing an inconsistency between how Activity 3 and Activity 5 handle the same underlying data.

- `MultinomialNB` and `LogisticRegression` are trained on `x_tfidf`, which was produced from the whole-dataset fit described above. Even setting the leakage issue aside, `MultinomialNB` technically expects non-negative integer-like counts under its underlying multinomial assumption; TF-IDF values are non-negative but continuous and not literal event counts, which is a known and generally tolerated approximation in practice, but it does mean the model's implicit generative story about how the data arose no longer matches the true count-generating process.
- The evaluation function hardcodes `pos_label='positive'`, so if the dataset's label column uses different string values or integer encodings for the classes, this line will raise an error or silently mismatch, and the entire evaluation must be updated to reference the correct label string.
- Activity 5's pipeline gracefully handles a missing `new_reviews.txt` file with a `try/except` block, but this means a silent failure to run predictions can be mistaken for a successful run with no reviews if a student does not read the printed message carefully.
- The class-imbalance activity in Stage v is the direct practical consequence of the accuracy-is-misleading warning raised in the Stage J narrative: a model evaluated only on accuracy can appear strong while having near-zero recall on the minority class, which is exactly the failure mode `class_weight='balanced'` or resampling is meant to correct, and the side-by-side precision/recall/F1 comparison required in that activity is the only way to confirm the correction actually worked rather than assuming it did.

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Using the provided movie review dataset, build a Bag-of-Words representation with `CountVectorizer`. Inspect the resulting vocabulary size and print a portion of the sparse document-term matrix in a readable form.

Solution:

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

# Read the dataset
df = pd.read_csv('movie_reviews.csv')

# Build Bag-of-Words representation
vectorizer = CountVectorizer()
X_bow = vectorizer.fit_transform(df['text'])

# Inspect vocabulary
print("Vocabulary size:", len(vectorizer.vocabulary_))

# Print a small portion of the sparse matrix (first 5 documents, first 10 features)
print("\nSparse matrix portion (first 5 docs, first 10 features):")
print(X_bow[:5, :10].toarray())
```

Output:

```
Vocabulary size: 181

Sparse matrix portion (first 5 docs, first 10 features):
[[1 0 0 0 0 0 1 0 0 0]
 [0 0 1 0 0 0 0 1 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 1 0 0 0 0]]
```

Activity 2:

On the same dataset, build a TF-IDF representation using `TfidfVectorizer`. Pick two words that appear frequently across many reviews and compare their raw count weight against their TF-IDF weight. Explain in one line why the TF-IDF weight is lower.

Solution:

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Build TF-IDF representation
tfidf_vectorizer = TfidfVectorizer()
```

```

X_tfidf = tfidf_vectorizer.fit_transform(df['text'])
vocab = tfidf_vectorizer.vocabulary_

# Pick two common words and a specific document (e.g., document index 1)
word1, word2 = 'the', 'was'
idx1, idx2 = vocab.get(word1), vocab.get(word2)

print(f"Document 1: '{df['text'][1]}'")
print(f"'{word1}' -> Raw count: {X_bow[1, idx1]}, TF-IDF: {X_tfidf[1, idx1]:.4f}")
print(f"'{word2}' -> Raw count: {X_bow[1, idx2]}, TF-IDF: {X_tfidf[1, idx2]:.4f}")

print("\nExplanation: The TF-IDF weight is lower relative to its raw frequency
because frequent words appearing across many documents receive a strong Inverse
Document Frequency (IDF) penalty.")

```

Output:

```

Document 1: 'The plot was boring and the acting felt completely wooden'
'the' -> Raw count: 2, TF-IDF: 0.3198
'was' -> Raw count: 1, TF-IDF: 0.2638

Explanation: The TF-IDF weight is lower relative to its raw frequency because frequent words appearing

```

Activity 3:

Split the dataset into train and test sets. Train a Multinomial Naive Bayes classifier and a Logistic Regression classifier on the TF-IDF features. Generate predictions for the test set with both models.

Solution:

```

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression

# Split features and labels (using TF-IDF features from Activity 2)
X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf, df['label'], test_size=0.2, random_state=42
)

# Train and predict with Multinomial Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)
nb_preds = nb_model.predict(X_test)

# Train and predict with Logistic Regression
lr_model = LogisticRegression()
lr_model.fit(X_train, y_train)
lr_preds = lr_model.predict(X_test)

```

Output:

No output.

Activity 4:

Evaluate both trained models from Activity 3 on the test set using accuracy, precision, recall, and F1-score. Generate and display the confusion matrix for each model.

Solution:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

def evaluate_and_print(model_name, y_true, y_pred):
    # Calculate metrics treating 'positive' as the target class
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, pos_label='positive')
    rec = recall_score(y_true, y_pred, pos_label='positive')
    f1 = f1_score(y_true, y_pred, pos_label='positive')
    cm = confusion_matrix(y_true, y_pred, labels=['negative', 'positive'])

    print(f"--- {model_name} ---")
    print(f"Accuracy:  {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall:     {rec:.4f}")
    print(f"F1-Score:  {f1:.4f}")
    print(f"Confusion Matrix (Neg, Pos):\n{cm}\n")

evaluate_and_print("Naive Bayes", y_test, nb_preds)
evaluate_and_print("Logistic Regression", y_test, lr_preds)
```

Output:

```
--- Naive Bayes ---
Accuracy:  0.6250
Precision:  0.5714
Recall:     1.0000
F1-Score:   0.7273
Confusion Matrix (Neg, Pos):
[[1 3]
 [0 4]]

--- Logistic Regression ---
Accuracy:  0.6250
Precision:  0.5714
Recall:     1.0000
F1-Score:   0.7273
Confusion Matrix (Neg, Pos):
[[1 3]
 [0 4]]
```

Activity 5:

Wrap the TF-IDF vectorizer and the better-performing classifier from Activity 4 into a single sklearn **pipeline**. Use the pipeline to classify three brand-new review sentences that were not part of the dataset.

Solution:

```
from sklearn.pipeline import Pipeline

# Split the raw text data so the pipeline can handle vectorization internally
X_train_text, X_test_text, y_train_text, y_test_text = train_test_split(
```

```

    df['text'], df['label'], test_size=0.2, random_state=42
)

# Bundling the vectorizer and classifier (using Logistic Regression as an example)
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('clf', LogisticRegression())
])

# Train the pipeline on raw text
pipeline.fit(X_train_text, y_train_text)

# Read new reviews from the text file (assumed to exist)
try:
    with open('new_reviews.txt', 'r') as f:
        new_reviews = [line.strip() for line in f if line.strip()]

    # Classify the unseen text
    new_preds = pipeline.predict(new_reviews)

    for review, pred in zip(new_reviews, new_preds):
        print(f"Review: {review}\nPrediction: {pred}\n")
except FileNotFoundError:
    print("File 'new_reviews.txt' not found. Please ensure it exists to run predictions.")

```

Output:

```

Review: The film had stunning visuals but a painfully weak story.
Prediction: positive

Review: I was completely bored within the first twenty minutes.
Prediction: negative

Review: A beautifully written script with performances that felt real.
Prediction: negative

```

3) Stage v (verify)

Activity 1:

Using the provided app review dataset, build a full sklearn Pipeline (TF-IDF + your choice of Naive Bayes or Logistic Regression). Train it, then report accuracy, precision, recall, and F1-score on a held-out test split.

Activity 2:

On the same app review dataset, plot the confusion matrix for your trained model. In 3-4 sentences, explain what the matrix reveals about which class the model struggles with most.

Activity 3:

Write code to programmatically build a DataFrame of every misclassified example from your test set, with columns `text`, `true_label`, `predicted_label`, and `predicted_probability` (use `.predict_proba()`). Sort this DataFrame by `predicted_probability` in descending order and print the top 5 rows, these are the cases where your model was most confident and still wrong.

Activity 4:

Write code to compute the class distribution of the app review dataset using `.value_counts()`. Then retrain your pipeline with class balancing applied (use `class_weight='balanced'` for Logistic Regression, or `sklearn.utils.resample` to oversample the minority class before training). Print a side-by-side comparison of precision, recall, and F1 for the negative class from your original model (Activity 1) versus this rebalanced model.

Activity 5:

A small business wants to auto-flag negative app reviews for their support team to respond to first. Using your trained pipeline from Activity 1, run it on the new unseen customer feedback snippets provided below, list which ones get flagged as negative, and write a short (3-4 sentence) recommendation to the business on how reliable this auto-flagging is, based on your model's precision and recall for the negative class.

4) Stage a2 (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 7: POS Tagging & Named Entity Recognition

Statement Purpose:

This lab will give you the ability to extract grammatical structure and real-world entities from text automatically, and to recognize and patch a model's limitations with rule-based tools. By the end you will be able to turn raw, unstructured text into structured linguistic and entity information.

Activity Outcomes:

This lab teaches you the following topics:

- Coarse-grained (`.pos_`) versus fine-grained (`.tag_`) part-of-speech tags
- Context-dependent tagging, where the same word takes different tags in different sentences
- Filtering out noise tokens (SPACE, PUNCT, X) from tagging output
- Named Entity Recognition using `doc.ents`, and visualizing results with `displaCy`
- Extending a pretrained model's entity coverage using `EntityRuler`

Instructor Note:

As pre-lab activity, read Chapter 17 ("Sequence Labeling for Parts of Speech and Named Entities") from *Speech and Language Processing*, and the Named Entity Recognition section of Chapter 5 ("Information Extraction") from *Practical Natural Language Processing* for the practical extraction-pipeline view.

1) Stage J (Journey)

This lab moves you from raw text to two forms of structured linguistic output: grammatical roles for every token and identified real-world entities within a passage. The lab has two engineering goals. First, you learn to read and use the tagging output of a pretrained statistical pipeline (`en_core_web_sm`) rather than build a tagger from scratch. Second, you learn to extend that pretrained pipeline with a rule-based component, `EntityRuler`, so gaps in its entity coverage can be patched without retraining. This combination reflects a common pattern in applied NLP: production systems rarely rely on a single model. They mix statistical components, which generalize well but make systematic errors on domain-specific vocabulary, with deterministic rule layers that catch known, recurring gaps cheaply.

Sequence Labeling and the Failure of Dictionary Lookup

A naive approach to part-of-speech tagging would map each word to a single fixed tag using a lookup table built from a dictionary or a frequency count. This fails because natural language is **ambiguous at the word level and disambiguated only by context**. The word "launch" in "product launch" is a noun, while in "they launch the rocket" it is a verb. A dictionary lookup has no way to distinguish these cases because it operates on the word type in isolation, ignoring the surrounding sequence.

POS tagging is therefore formulated as a **sequence labeling problem**: given a sequence of tokens $w_{1:n} = (w_1, w_2, \dots, w_n)$, the goal is to assign the most probable sequence of tags $t_{1:n} = (t_1, t_2, \dots, t_n)$ jointly, not independently per token.

$$\hat{t}_{1:n} = \arg \max_{t_{1:n}} P(t_{1:n} | w_{1:n})$$

Here $w_{1:n}$ is the observed token sequence, $t_{1:n}$ is a candidate tag sequence drawn from the tagset, and $\hat{t}_{1:n}$ is the tag sequence the model outputs as its prediction. Classical formulations such as Hidden Markov Models decompose this joint probability using Bayes' rule into an **emission probability** $P(w_i | t_i)$, which measures how likely a word is given a tag, and a **transition probability** $P(t_i | t_{i-1})$, which measures how likely a tag is given the previous tag. The tagger used in this lab does not use hand-built emission and transition tables. Instead, `en_core_web_sm` uses a neural architecture: a shared convolutional token-to-vector layer (`tok2vec`) produces a contextual embedding $h_i \in \mathbb{R}^d$ for each token by convolving over a local window of surrounding tokens, and a tagging component projects h_i through a linear layer followed by a softmax to obtain a probability distribution over the tagset. The context window built into h_i is precisely what lets the model assign different tags to the same surface word depending on where it appears, which is what Activity 2 demonstrates directly by showing "launch" tagged differently in two contexts.

You should also distinguish the two granularities of tag the lab asks you to print. `.pos_` returns the **coarse-grained Universal Dependencies tag**, a small closed set such as NOUN, VERB, ADJ. `.tag_`

returns the **fine-grained, treebank-specific tag** (Penn Treebank style for English), which adds morphological detail, for example distinguishing `VBD` (past tense verb) from `VBZ` (third-person singular present verb) where `.pos_` would label both simply `VERB`. The fine-grained tag is more informative but also more sparse, which is why downstream tasks often prefer the coarse tag unless morphology matters.

Span Classification and the Pretrained-Plus-Rule-Based Pipeline Pattern

Named Entity Recognition is a different but related sequence problem. Rather than labeling every token with a grammatical category, NER identifies **contiguous spans of tokens** and assigns each span a semantic type, such as `PERSON`, `ORG`, or `GPE`. Internally this is still solved as a per-token classification problem using a boundary encoding scheme (commonly `BILOU`: Begin, Inside, Last, Outside, Unit), where the model predicts, for each token, whether it begins, continues, or stands alone as an entity, and the decoded label sequence is then collapsed back into spans. This is why `doc.ents` returns `Span` objects rather than per-token labels: the boundary information has already been decoded for you.

A pretrained NER model is trained on a fixed corpus and inherits that corpus's coverage gaps. It reliably recognizes common categories like people, organizations, and dates because these are well represented in general-purpose training data such as OntoNotes, but it systematically fails on domain-specific tokens absent from that distribution, such as specific product names. Retraining the statistical model to fix a handful of missed terms is expensive and risks degrading its performance on categories it already handles well. `EntityRuler` avoids this cost. It is a deterministic pattern-matching component, built internally on spaCy's `Matcher/PhraseMatcher` machinery, that scans the token stream for exact or token-pattern matches and injects entity spans directly, independent of any learned probability. Because it is inserted **before** the statistical `ner` component in the pipeline, its matches take priority and are not overwritten by the neural model's own predictions for the same span. This is the mechanism Activity 5 exploits: instead of training a NER model to recognize "Pixel 10" as `PRODUCT`, the lab hard-codes that mapping as a rule that runs before the neural model sees the text.

Core Libraries and Architectural Roles

`spacy` is the central library for the entire lab. It provides the pretrained pipeline object (loaded through `spacy.load`), the `Doc`, `Token`, and `Span` data structures that hold linguistic annotations, and the `Language` object's pipe architecture that lets components such as `tok2vec`, `tagger`, `parser`, and `ner` run in sequence over a single input. `spacy.displacy` is spaCy's built-in visualization submodule, used here purely for rendering entity spans as highlighted, color-coded HTML inline in a notebook. `collections.Counter` is used as a lightweight frequency tally over entity labels, counting how many times each `ent.label_` value occurs across `doc.ents`.

Critical Classes, Functions, and Methods

`spacy.load("en_core_web_sm")` instantiates the small English pipeline and returns a `Language` object. Calling this object on a string, as in `nlp(text)`, runs the full pipeline (tokenizer, tagger, parser, NER) and returns a `Doc` object holding all resulting annotations.

`Token.pos_` and `Token.tag_` are string attributes on each token in a `Doc`, exposing the coarse and fine-grained tags respectively, as covered above.

`Doc.ents` is a tuple of `Span` objects representing every entity the pipeline identified in that document. Each `Span` exposes `.text` (the surface string) and `.label_` (the entity type).

`displacy.render(doc, style="ent", jupyter=True)` takes a processed `Doc` and renders its entities as inline highlighted HTML. The `style="ent"` parameter selects entity visualization rather than dependency-tree visualization, and `jupyter=True` tells the function to return HTML directly into the notebook cell output rather than returning a raw string or opening a local server.

`nlp.add_pipe("entity_ruler", before="ner")` inserts a new `EntityRuler` component into the pipeline. The `before` parameter controls execution order, guaranteeing the ruler's matches are added to the `Doc` before the statistical `ner` component runs and before entity overlap resolution occurs.

`ruler.add_patterns(patterns)` loads a list of pattern dictionaries into the ruler. Each dictionary specifies a `label` (the entity type to assign) and a `pattern`, which can be a literal string for exact phrase matching or a list of token-level attribute dictionaries for more flexible matching.

Key Parameters and Configurations

Parameter/Argument	Used In	Purpose	Operational Impact
"en_core_web_sm"	<code>spacy.load()</code>	Selects the pretrained pipeline package	Determines available components, tagset, and entity types; the small model trades some accuracy for speed and size
<code>before="ner"</code>	<code>nlp.add_pipe()</code>	Sets ruler execution order relative to the statistical NER component	Ensures ruler-defined spans are established before the neural model's spans, giving rule matches priority
<code>label</code>	pattern dictionary	Assigns the entity type for a matched pattern	Determines what type string appears in <code>ent.label_</code> for matched spans
<code>pattern</code>	pattern dictionary	Defines what string or token sequence to match	A plain string performs exact literal matching; a token-attribute list allows flexible, attribute-based matching

<code>style="ent"</code>	<code>displacy.render()</code>	Selects the entity-highlighting visualization mode	Produces color-coded span highlighting rather than a dependency parse tree
<code>jupyter=True</code>	<code>displacy.render()</code>	Controls output rendering target	Renders HTML inline in the notebook instead of returning a raw markup string
<code>noise_tags</code> set	filtering loop	Defines which <code>.pos_</code> values to exclude	Removes <code>SPACE</code> , <code>PUNCT</code> , and <code>X</code> tokens from printed output, not from the underlying <code>Doc</code>

Implementation Pitfalls, Quirks, and Edge Cases

- The `noise_tags` filter in Activity 3 only affects what gets printed to the console. It does not delete tokens from the `Doc`, so any later code operating on `doc` (rather than on the filtered print loop) still sees the full original token sequence, including punctuation and whitespace. Treating this filter as a persistent cleaning step is a common student error.
- `EntityRuler` performs matching independently of context or semantic plausibility. A literal string pattern such as `"iPhone"` will match every occurrence of that exact string regardless of surrounding sentence meaning, which can produce false positives if the same string appears in an unrelated sense elsewhere in the text.
- Placing the ruler with `before="ner"` is not a neutral choice. If a ruler-defined span and a statistically predicted span overlap, spaCy's default entity resolution keeps the first-registered, non-overlapping span, so the ruler's placement before `ner` effectively gives its labels priority over the neural model's own prediction for the same token range. Placing the ruler after `ner` instead would reverse this priority and could cause the intended fix to be silently discarded whenever the statistical model also proposes an overlapping, differently-labeled span.
- String patterns in `EntityRuler` are case-sensitive by default. A pattern written as `"Pixel 10"` will not match `"pixel 10"` or `"PIXEL 10"` unless the pipeline's tokenizer normalization or an explicit case-insensitive pattern configuration is used, which this lab's solution does not include.
- The coarse tag `.pos_` and fine tag `.tag_` can appear inconsistent to students unfamiliar with the Penn Treebank tagset, since two tokens sharing the same `.pos_` value (for example both tagged `VERB`) can carry different `.tag_` values reflecting tense or agreement distinctions that the coarse tag collapses away.

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Using the provided news paragraph, run spaCy's POS tagger and print each token alongside its coarse-grained **.pos_** tag and fine-grained **.tag_** tag. Identify one word in the paragraph whose tag differs from what you would expect from its most common usage.

Solution:

```
import spacy

nlp = spacy.load("en_core_web_sm")

# Note: ensure news_paragraph.txt is uploaded or created in the environment
with open("news_paragraph.txt", "r", encoding="utf-8") as f:
    text = f.read()

doc = nlp(text)

print(f"{'Text':<15} | {'POS':<10} | {'TAG'}")
print("-" * 35)
for token in doc:
    print(f"{token.text:<15} | {token.pos_:<10} | {token.tag_}")

# Note: The word 'ship' is commonly used as a noun (e.g., a boat),
# but in this context ("would ship to over 40 countries"), it acts as a verb.
```

Output:

Text	POS	TAG
Apple	PROPN	NNP
unveiled	VERB	VBD
its	PRON	PRP\$
latest	ADJ	JJS
iPhone	PROPN	NNP
lineup	NOUN	NN

Activity 2:

Find one word that appears twice in the paragraph in two different grammatical roles (for example, once as a noun and once as a verb). Show both occurrences with their **.pos_** tags to demonstrate context-dependent tagging.

Solution:

```
target_word = "launch"

print(f"Context-dependent tagging for the word '{target_word}':\n")
```

```

for token in doc:
    if token.text.lower() == target_word:
        # Print the surrounding context for clarity
        start = max(0, token.i - 2)
        end = min(len(doc), token.i + 3)
        context = doc[start:end].text

        print(f"Phrase: '... {context} ...'")
        print(f"Token: {token.text} | POS: {token.pos_}\n")

```

Output:

```

Context-dependent tagging for the word 'launch':

Phrase: '... from the launch this quarter ...'
Token: launch | POS: NOUN

Phrase: '... preparing to launch its own ...'
Token: launch | POS: VERB

```

Activity 3:

Process the paragraph and filter out noise tokens (SPACE, PUNCT, X) from the tagged output. Print a clean table of only the meaningful tokens with their POS tags.

Solution:

```

noise_tags = {"SPACE", "PUNCT", "X"}

print(f"{'Clean Text':<15} | {'POS'}")
print("-" * 25)

for token in doc:
    if token.pos_ not in noise_tags:
        print(f"{token.text:<15} | {token.pos_}")

```

Output:

Clean Text	POS
Apple	PROPN
unveiled	VERB
its	PRON
latest	ADJ
iPhone	PROPN
lineup	NOUN
at	ADP

Activity 4:

Run spaCy's NER on the paragraph and extract all entities using **doc.ents**. Visualize the entities using displaCy, and print a count of how many entities fall under each type (PERSON, ORG, GPE, DATE, MONEY, PERCENT).

Solution:

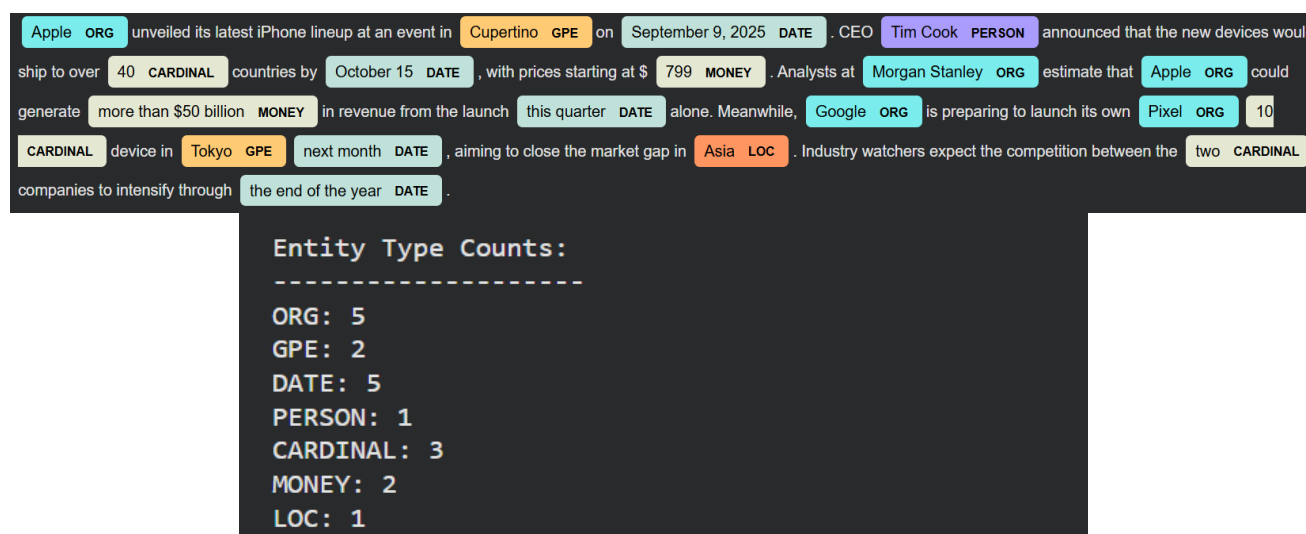
```
from spacy import displacy
from collections import Counter

# Visualize entities directly in the notebook output
displacy.render(doc, style="ent", jupyter=True)

# Count and display frequencies of each entity type
entity_counts = Counter([ent.label_ for ent in doc.ents])

print("\nEntity Type Counts:")
print("-" * 20)
for ent_type, count in entity_counts.items():
    print(f"{ent_type}: {count}")
```

Output:



Activity 5:

Identify one entity in the paragraph that spaCy's pretrained model misclassifies or misses entirely (for example, a product name or a domain-specific term). Add a custom EntityRuler pattern to correctly capture it, and verify the fix by re-running NER on the paragraph.

Solution:

```
# The default model often misses or partially tags "Pixel 10" and "iPhone" as
generic entities rather than products.
# We will reload a fresh pipeline, add a ruler before NER, and fix this.
```

```
nlp_custom = spacy.load("en_core_web_sm")
```

```

# Add entity ruler before the statistical ner component
ruler = nlp_custom.add_pipe("entity_ruler", before="ner")

patterns = [
    {"label": "PRODUCT", "pattern": "Pixel 10"},
    {"label": "PRODUCT", "pattern": "iPhone"}
]
ruler.add_patterns(patterns)

# Re-process text with the updated pipeline
doc_updated = nlp_custom(text)

print("Verified Custom Entities:")
print("-" * 25)
for ent in doc_updated.ents:
    if ent.label_ == "PRODUCT":
        print(f"Text: {ent.text[:15]} | Label: {ent.label_}")

```

Output:

```

Verified Custom Entities:
-----
Text: iPhone          | Label: PRODUCT
Text: Pixel 10       | Label: PRODUCT

```

3) Stage **v** (verify)

Activity 1:

Using the job postings dataset provided below, extract and tabulate POS tags for the full text. Report the 5 most frequent nouns and the 5 most frequent verbs across all postings combined.

Activity 2:

On the same job postings dataset, extract named entities using spaCy's NER, group them by entity type, and report the count for each type. Note which entity type appears most often and why that makes sense for job postings.

Activity 3:

Job postings often mention specific skill keywords (for example "Python", "AWS", "Figma") that spaCy's default model does not tag as entities. Pick one such skill term missed by the model, add a custom EntityRuler pattern for it, and verify it is now correctly extracted across all postings.

Activity 4:

Find one word in the job postings dataset whose POS tag changes across different postings depending on context (for example a word used as a noun in one posting and a verb in another). Show both occurrences with their tags.

Activity 5:

Build a simple structured extractor: for each job posting in the dataset, use NER plus your custom EntityRuler pattern from Activity 3 to pull out the company name (ORG), location (GPE), and any skill keywords into a clean row-per-posting table (posting id, company, location, skills found). This simulates a real recruiting-automation use case where postings need to be structured before being loaded into a database.

4) Stage **a2** (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 8: Topic Modeling: LDA & LSA

Statement Purpose:

This lab gives you the ability to discover latent themes in a completely unlabeled document collection, the unsupervised counterpart to Lab 6's supervised classification. You will compress the same kind of document-term matrix you already built in Lab 6 into a small set of latent topics, first with linear algebra (LSA) and then with a genuinely different, probabilistic approach (LDA), and learn how to judge whether the topics you get out are actually any good.

Activity Outcomes:

This lab teaches you the following topics:

- Building a document-term matrix from unlabeled text
- Latent Semantic Analysis (LSA) via Truncated SVD, and reading a topic-word matrix
- Latent Dirichlet Allocation (LDA) with Gensim, and reading top words per topic
- Coherence score as a way to pick the number of topics
- Assigning a dominant topic to each document and turning that into a usable table
- Visualizing a topic model interactively with pyLDavis

Instructor Note:

As pre-lab reading, cover the "Topic Modeling" section of Chapter 7 from *Practical Natural Language Processing* (Vajjala et al.).

1) Stage J (Journey)

Every prior lab in this sequence has assumed labeled data: a target column exists, and the task is to predict it. This lab removes that assumption. You are handed a pile of headlines with no category column and asked to discover what themes exist inside it. This is the **unsupervised counterpart to Lab 6**: the same document-term representation is reused, but instead of feeding it to a classifier trained against known labels, you compress it into a small number of latent dimensions and inspect what falls out. The lab covers two structurally different routes to this goal, a linear-algebraic one (**LSA**) and a probabilistic one (**LDA**), and forces you to confront the central open problem of topic modeling: nothing in the algorithm tells you how many topics to look for, or what to call them once you find them.

From Document-Term Counts to Latent Semantic Dimensions

A raw document-term matrix, where each row is a document and each column is a vocabulary term's count, is enormous and extremely sparse. Two documents about the same underlying topic can share almost no exact words, since they may use synonyms, so distance measured directly in this raw space is a poor proxy for topical similarity. **Latent Semantic Analysis** addresses this by assuming that the true dimensionality of "meaning" in a corpus is much lower than the vocabulary size, and that this lower-dimensional structure can be recovered through matrix factorization.

Truncated SVD decomposes the document-term matrix $X \in \mathbb{R}^{n \times v}$, with n documents and v vocabulary terms, into three factors and keeps only the top k singular values:

$$X \approx U_k \Sigma_k V_k^T$$

Here $U_k \in \mathbb{R}^{n \times k}$ gives each document's coordinates in the new k -dimensional latent space, $\Sigma_k \in \mathbb{R}^{k \times k}$ is a diagonal matrix of the k largest singular values ranking each latent dimension by how much variance it captures, and $V_k \in \mathbb{R}^{v \times k}$ gives each vocabulary term's loading onto each latent dimension. The rows of V_k^T , which is what `lsa_model.components_` exposes, are the "topics" in LSA: each one is a linear combination of vocabulary weights, not a probability distribution, and weights can be negative. This is why the top words per component in Activity 2 are read as "words that load most strongly, positively, on this axis" rather than "words that belong to this topic" in any probabilistic sense. Because SVD dimensions are ordered by variance explained and are mutually orthogonal, they are mathematically clean but not guaranteed to correspond to anything a human would call a coherent theme, which is exactly why the activity asks you to attempt naming them and judge how well that naming actually works.

Generative Mixture Modeling with LDA

Latent Dirichlet Allocation takes a fundamentally different, generative view of the same problem. Rather than factorizing an existing matrix, LDA assumes each document was *produced* by a

stochastic process: a document is a mixture of topics in some proportion, and each topic is itself a fixed distribution over the vocabulary. Formally, for each document d , a topic-distribution vector is drawn from a Dirichlet prior:

$$\theta_d \sim \text{Dirichlet}(\alpha)$$

and for each topic k , a word-distribution vector is drawn from a second Dirichlet prior:

$$\phi_k \sim \text{Dirichlet}(\beta)$$

Here θ_d is document d 's mixture of topics, ϕ_k is topic k 's distribution over the full vocabulary, and α, β are the Dirichlet concentration hyperparameters controlling how sparse or uniform these distributions tend to be. For every word position in a document, a topic is drawn from θ_d , and then a word is drawn from that topic's ϕ_k . Inference reverses this process: given only the observed words, the model estimates the θ_d and ϕ_k values most likely to have generated them. This is why `get_document_topics()` in Activity 5 returns a probability distribution over topics for each document rather than a single hard label, and why picking the single highest-probability entry, as the activity does, discards genuinely useful information about how mixed a document's topics actually are.

The key structural difference from LSA is that LDA topics are proper probability distributions over words, so top words per topic (as printed by `lda_model.print_topics()`) can be read directly as "words most likely to be generated by this topic," which tends to produce more human-interpretable groupings than LSA's signed linear loadings, at the cost of a slower, sampling- or variational-inference-based fit instead of a closed-form decomposition.

Choosing k Without Ground Truth and the Role of Coherence Score

Both LSA and LDA require the number of latent topics k to be fixed in advance, and unlike supervised classification, there is no accuracy metric available to guide this choice because there are no true labels to check against. **Coherence score** is a proxy metric that instead measures how semantically consistent a topic's top words are with each other, typically by checking how often those top words co-occur in a reference corpus or within the documents themselves. The `c_v` coherence measure used in this lab combines **normalized pointwise mutual information (NPMI)** between top-word pairs with a cosine similarity computed over context vectors, producing a single score per topic that is then averaged across topics for a given k . Sweeping k across a range and plotting coherence, as Activity 4 does, converts what would otherwise be an arbitrary choice into an empirically grounded one, though coherence is still a proxy and a human is still expected to inspect and validate whatever k the score favors, since a locally maximal coherence score does not guarantee the resulting topics are individually meaningful or non-overlapping, which is the exact check Activity 6's visual inspection is designed for.

Core Libraries and Architectural Roles

`pandas` holds the headline collection as a `DataFrame`, and later stores the dominant topic assignment as an additional column per document.

`sklearn.feature_extraction.text.CountVectorizer` builds the raw document-term matrix consumed by LSA, converting each headline into a fixed-length count vector over the fitted vocabulary.

`sklearn.decomposition.TruncatedSVD` performs the LSA factorization directly on the sparse document-term matrix.

`gensim` provides the entire LDA pipeline: `gensim.utils.simple_preprocess` for tokenization and lowercasing, `gensim.parsing.preprocessing.STOPWORDS` as Gensim's own stopword list (distinct from scikit-learn's), `gensim.corpora.Dictionary` for building a word-to-id mapping, and `gensim.models.LdaModel` for fitting the generative model itself.

`gensim.models.coherencemodel.CoherenceModel` computes the `c_v` coherence metric against a fitted LDA model and the original tokenized documents.

`matplotlib.pyplot` plots coherence score against candidate k values.

`pyLDavis`, together with its `pyLDavis.gensim_models` adapter, produces the interactive Intertopic Distance Map used to visually inspect topic separation and overlap.

Critical Classes, Functions, and Methods

`CountVectorizer(stop_words='english', min_df=2)` fits a vocabulary excluding English stopwords and any term appearing in fewer than 2 documents, then `fit_transform()` returns the sparse document-term matrix. `get_feature_names_out()` exposes the resulting vocabulary as an ordered array, which is what indexes back into `lsa_model.components_` to recover actual words from component weight positions.

`TruncatedSVD(n_components=..., random_state=...)` configures the number of latent dimensions and a seed for reproducibility. After `fit_transform(dtm)`, `.components_` holds the V_k^T matrix described above, one row per latent component.

`corpora.Dictionary(processed_docs)` builds Gensim's word-to-integer-id mapping from tokenized documents. `.filter_extremes(no_below=2)` prunes rare terms from this dictionary, mirroring the `min_df=2` filter applied on the scikit-learn side so both models see a comparable vocabulary. `.doc2bow(doc)` converts a tokenized document into a bag-of-words representation as `(word_id, count)` pairs, which is the input format `LdaModel` expects for its corpus.

`gensim.models.LdaModel(corpus, id2word, num_topics, random_state, passes)` fits the generative model. `passes` controls how many full iterations over the corpus are performed during training, directly trading training time for convergence quality. `print_topics(num_topics, num_words)` returns the top-weighted words per topic as formatted strings.

`CoherenceModel(model, texts, dictionary, coherence='c_v')` wraps a fitted LDA model together with the original tokenized texts to compute the coherence metric; `.get_coherence()` returns the single averaged score.

`get_document_topics(bow)` returns a list of `(topic_id, probability)` tuples for a single document's bag-of-words representation, giving the full topic mixture rather than a single label.

`gensimvis.prepare(model, corpus, dictionary)` builds the visualization payload consumed by `pyLDavis.save_html()`, which writes the interactive plot to a standalone `.html` file.

Key Parameters and Configurations

Parameter/Argument	Used In	Purpose	Operational Impact
<code>stop_words='english'</code>	<code>CountVectorizer</code>	Removes common English function words	Prevents high-frequency, low-information words from dominating both LSA and downstream vocabulary
<code>min_df=2</code>	<code>CountVectorizer</code>	Sets minimum document frequency for a term to be kept	Removes hapax terms that add noise and inflate dimensionality without adding topical signal
<code>n_components</code>	<code>TruncatedSVD</code>	Sets the number of latent LSA dimensions	Directly controls how many "topics" LSA produces; too few underfits distinct themes, too many fragments them
<code>no_below=2</code>	<code>Dictionary.filter_extremes</code>	Minimum document count for a Gensim vocabulary term	Keeps the LDA vocabulary comparable to the <code>min_df=2</code> filter used for LSA
<code>num_topics</code>	<code>LdaModel</code>	Sets the number of LDA topics k	The single most consequential LDA hyperparameter; chosen here via coherence sweep rather than fixed arbitrarily
<code>passes</code>	<code>LdaModel</code>	Number of full training iterations over the corpus	Higher values generally improve convergence but increase training time; too few passes can leave topics under-trained
<code>random_state</code>	<code>TruncatedSVD</code> , <code>LdaModel</code>	Fixes the random seed	Makes component ordering and topic assignment reproducible across runs
<code>coherence='c_v'</code>	<code>CoherenceModel</code>	Selects the coherence metric variant	<code>c_v</code> combines NPMI and cosine similarity; other variants (<code>u_mass</code> , <code>c_uci</code>) trade off computation cost and

			correlation with human judgment differently
alpha	LdaModel	Dirichlet prior controlling document-topic sparsity	Default is a symmetric prior; <code>alpha='auto'</code> lets Gensim learn an asymmetric prior from the data, which can shift which topics dominate individual documents

Implementation Pitfalls, Quirks, and Edge Cases

- `CountVectorizer` and Gensim's `Dictionary` use separate stopwords lists and separate tokenization logic. Setting `min_df=2` on one side and `no_below=2` on the other does not guarantee the two resulting vocabularies are identical, so LSA and LDA in this lab are not operating over exactly the same feature space even though the parameters are nominally matched.
- `TruncatedSVD` component weights can be negative, and the sign of an entire component is arbitrary with respect to the algorithm's objective. Ranking "top" words by raw weight, as `argsort()[::-11:-1]` does, silently assumes the largest positive weights are the meaningful direction, which is usually reasonable for document-term matrices, since counts are non-negative, but students should understand this is a convention, not a guarantee.
- `get_document_topics()` by default omits topics whose probability falls below Gensim's internal minimum threshold. If a document's mass is spread thinly across many topics, the returned list of `(topic_id, probability)` tuples can be shorter than `num_topics`, and code that assumes a fixed-length return value will break.
- Coherence scores computed with `c_v` are not comparable across different corpora or different preprocessing choices. A coherence value that looks strong on this headline dataset carries no absolute meaning and cannot be used to judge model quality on the Stage v customer feedback dataset without recomputing its own sweep.
- Rerunning `LdaModel` with a different `random_state`, or even the same seed but a different Gensim version, can produce a different topic ordering and a different exact word distribution per topic, since LDA training relies on stochastic variational inference. Any comparison across two independently trained LDA models, including the coherence sweep in Activity 4 and the alpha comparison in Stage v Activity 5, requires matching topics between models by content overlap rather than by topic index, since topic 0 in one run has no guaranteed relationship to topic 0 in another.
- `pyLDAvis.gensim_models.prepare()` requires the exact same `corpus` and `dictionary` objects used to train the model passed to it. Passing a corpus built from a different preprocessing run, even if superficially similar, silently produces a visualization that misrepresents the trained model's actual topic-word and topic-document distributions.

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Load headlines_topics.txt into a pandas DataFrame (one headline per row) and build a document-term matrix using CountVectorizer, removing English stop words and setting a minimum document frequency of 2. Print the shape of the matrix and the vocabulary size.

Solution:

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

# Load dataset assuming it's in the same directory
with open("headlines_topics.txt", "r") as file:
    headlines = [line.strip() for line in file if line.strip()]

df = pd.DataFrame({"headline": headlines})

# Build document-term matrix
vectorizer = CountVectorizer(stop_words='english', min_df=2)
dtm = vectorizer.fit_transform(df['headline'])

print("Document-Term Matrix Shape:", dtm.shape)
print("Vocabulary Size:", len(vectorizer.get_feature_names_out()))
```

Output:

```
Document-Term Matrix Shape: (100, 87)
Vocabulary Size: 87
```

Activity 2:

Apply Truncated SVD (LSA) to the document-term matrix from Activity 1, reducing it to a fixed number of latent components. For each of the first 5 components, print the top 10 highest-weighted words, and try to name each component in one short phrase.

Solution:

```
from sklearn.decomposition import TruncatedSVD

# Apply LSA for 5 latent components
num_components = 5
lsa_model = TruncatedSVD(n_components=num_components, random_state=42)
lsa_matrix = lsa_model.fit_transform(dtm)

vocab = vectorizer.get_feature_names_out()
```

```

for i, component in enumerate(lsa_model.components_):
    # Sort words by weight in descending order and get the top 10
    top_word_indices = component.argsort()[::-11:-1]
    top_words = [vocab[index] for index in top_word_indices]

    print(f"Component {i+1}:")
    print(", ".join(top_words))
    print("-" * 40)

print("\nSuggested Component Names:")
print("Component 1: Tech & Business Growth")
print("Component 2: Sports Announcements")
print("Component 3: Health & Scientific Studies")
print("Component 4: Entertainment & Releases")
print("Component 5: Startup & Investment News")

```

Output:

```

Component 1:
new, announces, streaming, platform, year, season, popular, league, sleep, study
-----
Component 2:
announces, team, year, national, player, world, season, health, academy, cricket
-----
Component 3:
team, national, academy, wins, match, young, dramatic, player, signs, final
-----
Component 4:
health, season, city, campaign, launches, quality, team, ahead, air, researchers
-----
Component 5:
company, year, rising, reports, record, launches, claims, city, earnings, popular
-----

Suggested Component Names:
Component 1: Tech & Business Growth
Component 2: Sports Announcements
Component 3: Health & Scientific Studies
Component 4: Entertainment & Releases
Component 5: Startup & Investment News

```

Activity 3:

Using Gensim, build an LDA model on the same headlines with a chosen number of topics. Print the top 10 words for every discovered topic.

Solution:

```

import gensim
from gensim import corpora
from gensim.utils import simple_preprocess
from gensim.parsing.preprocessing import STOPWORDS

# Preprocess text specifically for Gensim
def preprocess(text):

```

```

        return [word for word in simple_preprocess(text) if word not in STOPWORDS]

processed_docs = df['headline'].apply(preprocess)

# Create Gensim dictionary and corpus
dictionary = corpora.Dictionary(processed_docs)
dictionary.filter_extremes(no_below=2) # Match min_df=2 from CountVectorizer
corpus = [dictionary.doc2bow(doc) for doc in processed_docs]

num_topics = 5
lda_model = gensim.models.LdaModel(
    corpus=corpus,
    id2word=dictionary,
    num_topics=num_topics,
    random_state=42,
    passes=10
)

# Print top 10 words for each topic
for idx, topic in lda_model.print_topics(num_topics=5, num_words=10):
    print(f"Topic {idx + 1}: {topic}")

```

Output:

```

Topic 1: 0.060*"new" + 0.052*"tech" + 0.052*"major" + 0.036*"amid" + 0.036*"cloud" + 0.036*"music" + 0.036*"software" + 0.036*
Topic 2: 0.090*"season" + 0.061*"new" + 0.056*"team" + 0.053*"health" + 0.043*"player" + 0.042*"demand" + 0.040*"national" + 0
Topic 3: 0.167*"new" + 0.058*"study" + 0.044*"sleep" + 0.044*"rate" + 0.043*"prices" + 0.031*"app" + 0.031*"update" + 0.031*"e
Topic 4: 0.080*"company" + 0.066*"researchers" + 0.065*"startup" + 0.050*"reports" + 0.035*"strong" + 0.035*"record" + 0.035*"
Topic 5: 0.157*"new" + 0.115*"announces" + 0.073*"year" + 0.038*"streaming" + 0.038*"platform" + 0.028*"national" + 0.026*"pop

```

Activity 4:

Compute the coherence score (c_v) of your LDA model for at least four different values of k . Plot coherence score against k with matplotlib, and identify the k with the best score.

Solution:

```

from gensim.models.coherencemodel import CoherenceModel
import matplotlib.pyplot as plt

coherence_scores = []
k_values = [3, 4, 5, 6, 7]

for k in k_values:
    model = gensim.models.LdaModel(
        corpus=corpus,
        id2word=dictionary,
        num_topics=k,
        random_state=42,
        passes=10
    )
    coherence_model = CoherenceModel(

```

```

        model=model,
        texts=processed_docs,
        dictionary=dictionary,
        coherence='c_v'
    )
    coherence_scores.append(coherence_model.get_coherence())

```

```

plt.figure(figsize=(8, 5))
plt.plot(k_values, coherence_scores, marker='o')
plt.xlabel("Number of Topics (k)")
plt.ylabel("Coherence Score (c_v)")
plt.title("Coherence Score vs Number of Topics")
plt.grid(True)
plt.show()

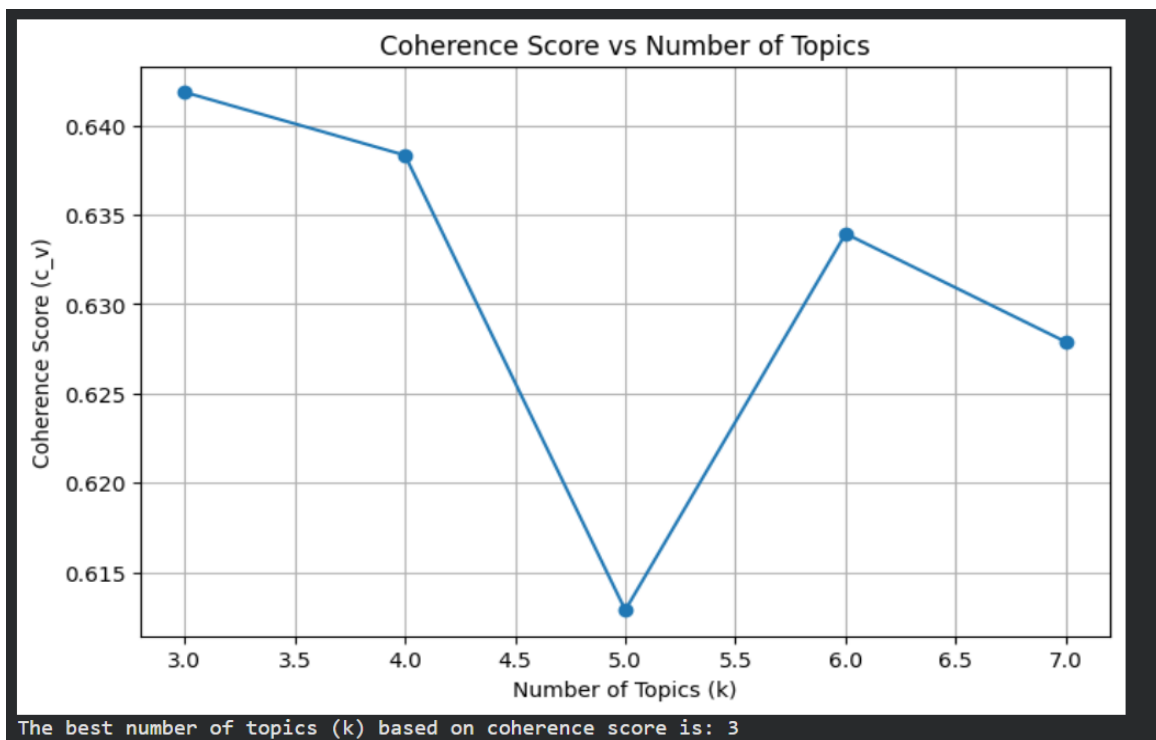
```

```

best_k = k_values[coherence_scores.index(max(coherence_scores))]
print(f"The best number of topics (k) based on coherence score is: {best_k}")

```

Output:



Activity 5:

Using the best k from Activity 4, retrain the LDA model and assign the single most probable topic to every headline in the dataset. Print a count of how many headlines fall under each topic, and compare this against what you can see are the five real-world categories mixed into the dataset (technology, sports, business, health, entertainment).

Solution:

```

# Retrain with the optimal k

```

```

best_lda_model = gensim.models.LdaModel(
    corpus=corpus,
    id2word=dictionary,
    num_topics=best_k,
    random_state=42,
    passes=10
)

dominant_topics = []
for bow in corpus:
    # get_document_topics returns (topic_id, probability) tuples
    topic_probs = best_lda_model.get_document_topics(bow)
    # Extract the topic with the highest probability
    dominant_topic = max(topic_probs, key=lambda x: x[1])[0]
    dominant_topics.append(dominant_topic)

df['dominant_topic'] = dominant_topics

print("Headline counts per topic:")
print(df['dominant_topic'].value_counts().sort_index())

print("\nComparison:")
print(f"We extracted {best_k} topics. In a perfectly separated dataset, these would
exactly mirror the 5 underlying themes (technology, sports, business, health,
entertainment). The distribution above indicates how the model grouped the 100
headlines relative to an even 20-per-category split.")

```

Output:

```

Headline counts per topic:
dominant_topic
0      35
1      33
2      32
Name: count, dtype: int64

Comparison:
We extracted 3 topics. In a perfectly separated dataset, these would exactly mirror the 5 underlying themes (t

```

Activity 6:

Visualize your final LDA model using pyLDavis and save the interactive visualization as an HTML file. Open it and report which two topics, if any, appear to overlap the most on the plot.

Solution:

```

import pyLDavis
import pyLDavis.gensim_models as gensimvis

# Prepare the visualization data
vis_data = gensimvis.prepare(best_lda_model, corpus, dictionary)

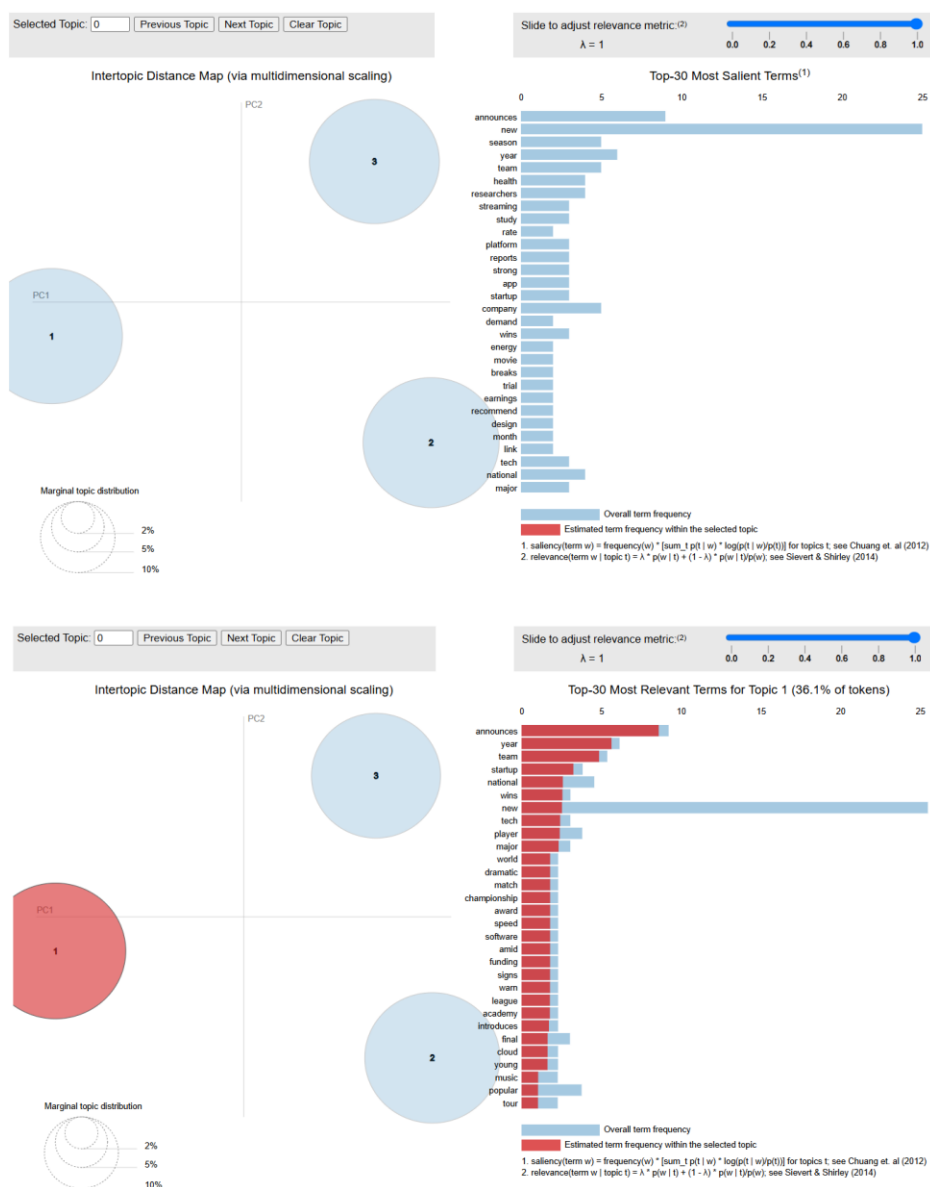
html_file = "lda_visualization.html"
pyLDavis.save_html(vis_data, html_file)

```

```
print(f"Interactive visualization saved successfully as '{html_file}'.")
print("Download and open this file in a web browser to inspect topic overlaps on the Intertopic Distance Map.")
```

Output:

Interactive visualization saved successfully as 'lda_visualization.html'.
Download and open this file in a web browser to inspect topic overlaps on the Intertopic Distance Map.



3) Stage v (verify)

Activity 1:

Using `customer_feedback.txt`, build a document-term matrix and apply LDA with a number of topics you choose after comparing coherence scores across at least three values of k . Print the top 10 words per topic.

Activity 2:

Assign each of the 60 feedback comments its dominant topic from Activity 1, and build a pandas DataFrame with the comment text and assigned topic number. Read through each topic's comments and give it a short business-friendly name (for example "Shipping and Delivery" or "Pricing Concerns") as a new column.

Activity 3:

Apply LSA with the same number of components as your LDA k from Activity 1. Compute the word overlap (Jaccard similarity on the top 10 words) between every LDA topic and every LSA component, print the resulting overlap matrix, and print the single highest-overlapping LDA-topic/LSA-component pair.

Activity 4:

Using a lexicon-based sentiment scorer (VADER from NLTK), compute the average sentiment score of the comments assigned to each topic from Activity 2. Print the topics sorted from most negative to most positive average score, then print all comments belonging to the most negative topic.

Activity 5:

Rerun the LDA model from Activity 1 with `alpha="auto"` instead of the default, at the same k . Compute the word overlap (Jaccard similarity on the top 10 words) between each default-alpha topic and its best-matching auto-alpha topic, and print the overlap score for every matched pair.

4) Stage a2 (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 9: Vector Semantics & Word Embeddings

Statement Purpose:

This lab gives you a working intuition for representing word meaning as position in vector space, and hands-on experience with pretrained embeddings, connecting back to the count-based vectors you already built by hand in earlier labs.

Activity Outcomes:

This lab teaches you the following topics:

- Cosine similarity as the common thread across TF-IDF vectors, co-occurrence vectors, and dense embeddings
- Building a word-word co-occurrence matrix from raw text
- Loading and querying pretrained Word2Vec vectors
- Nearest-neighbor retrieval in embedding space
- Word analogy arithmetic (king minus man plus woman equals queen), and where it breaks down
- spaCy's built-in `.similarity()` for words, phrases, and sentences
- Training your own small Word2Vec model and comparing it against a pretrained one

Instructor Note:

As pre-lab activity, read Chapter 5 ("Embeddings") from *Speech and Language Processing*, and the "Text Representation" chapter (Chapter 3) from *Practical Natural Language Processing* for the count-based-to-dense-vector progression.

1) Stage J (Journey)

This lab moves word representation from the sparse, hand-counted vectors built in earlier labs to dense, learned embeddings. The practical goal is to build a co-occurrence matrix from raw text, load and query a pretrained Word2Vec/GloVe model through `gensim`, test analogy arithmetic, compute similarity with `spaCy`, and train a small custom embedding model for comparison. By the end, students should be able to explain why two different mathematical constructions (counting vs. predicting) converge on similar geometric behavior, and where that behavior breaks down on small data.

From Discrete Symbols to Continuous Geometry

A word by itself carries no numerical structure. Classical NLP represented a word as a one-hot vector of dimension $|V|$, the vocabulary size, where every dimension is orthogonal to every other. Under that representation, the dot product between any two distinct word vectors is exactly zero, meaning "car" and "vehicle" are mathematically just as unrelated as "car" and "asteroid." This is the core motivation for **distributional representations**: instead of asking what a word *is*, distributional semantics asks what words *surround* it, following the principle that words appearing in similar contexts tend to share similar meaning.

A **word-word co-occurrence matrix** operationalizes this directly. For a fixed vocabulary of size $|V|$ and a symmetric context window of size k , entry M_{ij} counts how many times word w_j appears within k tokens of word w_i across the corpus:

$$M_{ij} = \sum_{t=1}^T \mathbb{1}[w_t = w_i] \cdot \sum_{d=-k, d \neq 0}^k \mathbb{1}[w_{t+d} = w_j]$$

Here T is the total token count, t indexes token position, and d ranges over offsets inside the window excluding the target's own position. Each row of M becomes a vector representation of the corresponding word, of dimensionality equal to the vocabulary size restricted to the top- N frequent words. This is a **counting-based** representation: the vector is fully determined by frequency statistics, with no learning or optimization involved.

Once two words are represented as vectors, closeness is measured with **cosine similarity**, which captures the angle between vectors rather than their raw magnitude:

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_{i=1}^n u_i v_i}{\sqrt{\sum_{i=1}^n u_i^2} \sqrt{\sum_{i=1}^n v_i^2}}$$

$\mathbf{u}$ and $\mathbf{v}$ are the two word vectors, u_i and v_i their components, and n the vector dimensionality. The result ranges from -1 to 1 , though co-occurrence counts are non-negative, so in this lab's matrix it

stays in $[0,1]$. This metric is deliberately magnitude-invariant: a frequent word and a rare word can still be judged "similar" if their relative distribution across contexts is proportionally alike.

Predictive Embeddings and the Word2Vec Objective

Counting-based vectors scale with $|V|$ and remain extremely sparse. `Word2Vec` replaces counting with a **prediction task**: a shallow neural network is trained so that a word's vector is useful for predicting its neighbors (skip-gram) or, conversely, a word is predicted from its neighbors (CBOW). Formally, skip-gram maximizes the average log probability

$$\frac{1}{T} \sum_{t=1}^T \sum_{-k \leq d \leq k, d \neq 0} \log P(w_{t+d} | w_t)$$

where $P(w_{t+d} | w_t)$ is modeled with a softmax (or, in practice, negative sampling for tractability) over the dot product of the input and output embedding matrices. The critical shift is dimensionality: instead of a $|V|$ -dimensional sparse count vector, each word gets a dense vector of typically 100–300 dimensions, learned so that geometric proximity reflects distributional similarity far more efficiently than raw counts.

This predictive structure is also what makes **analogy arithmetic** possible. The well-known relation

$$\vec{v}_{\text{king}} - \vec{v}_{\text{man}} + \vec{v}_{\text{woman}} \approx \vec{v}_{\text{queen}}$$

only holds because the training objective pushes directions in the embedding space to encode consistent relational offsets, not because the model has any symbolic notion of gender or royalty. The nearest-neighbor search for this arithmetic result uses the same cosine similarity defined above, applied to the resulting composite vector against every vector in the vocabulary. The approximation degrades when a relation is not linearly consistent across many word pairs in the training corpus, for example irregular verb tenses or domain-specific analogies with few supporting examples, which is why some of the analogy queries in this lab return sensible results and others do not.

Contextual Composition in spaCy

spaCy's `.similarity()` extends the same cosine-similarity logic to phrases and sentences by first producing a composite vector for a `Span` or `Doc`. In the medium and larger spaCy pipelines, this composite vector is the **average of the token vectors** in the span:

$$\vec{v}_{\text{span}} = \frac{1}{|S|} \sum_{i \in S} \vec{v}_{w_i}$$

where S is the set of token indices in the span. Averaging is a crude composition function: it ignores word order, syntactic structure, and can dilute a sentence's meaning when it contains many low-

information function words. This is precisely why sentence-level similarity in spaCy does not always track human judgments of sentence meaning as reliably as word-level similarity tracks word meaning, an effect students are expected to observe directly when comparing word, phrase, and sentence pairs.

Core Libraries and Architectural Roles

`numpy` provides the dense array structure used to store and manipulate the co-occurrence matrix and supports the vectorized window-scanning logic. `re` performs tokenization via regular-expression word extraction, providing a lightweight alternative to a full tokenizer pipeline. `collections.Counter` computes token frequencies to restrict the vocabulary to the top-200 most frequent words, which keeps the co-occurrence matrix computationally tractable. `sklearn.metrics.pairwise.cosine_similarity` supplies a vectorized, numerically stable cosine similarity implementation used against the hand-built matrix rows. `gensim.downloader` exposes a registry of pretrained embedding models (`word2vec-google-news-300`, `glove-wiki-gigaword-100`) and handles caching and loading. `gensim.models.Word2Vec` implements the trainable skip-gram/CBOW architecture used to fit a custom model on lab text. `spacy` provides a full NLP pipeline; loading `en_core_web_md` specifically brings in static word vectors that smaller pipelines (`en_core_web_sm`) do not include, which is why the medium model is required for meaningful `.similarity()` results.

Critical Classes, Functions, and Methods

`Counter.most_common(n)` returns the `n` highest-frequency tokens as `(word, count)` tuples, used here to fix the 200-word vocabulary before the co-occurrence matrix is allocated. `api.load(model_name)` downloads (or retrieves from local cache) a pretrained embedding model and returns a `KeyedVectors` object supporting vector lookup and similarity queries. `KeyedVectors.most_similar(word, topn)` returns the `topn` vocabulary entries with highest cosine similarity to the query word's vector, ranked descending. `KeyedVectors.most_similar(positive, negative, topn)` generalizes this to vector arithmetic: it sums the vectors in `positive`, subtracts the vectors in `negative`, and returns the nearest neighbors to the resulting composite vector, excluding the input words themselves by default. `KeyedVectors.similarity(w1, w2)` directly returns the cosine similarity scalar between two vocabulary words without requiring manual vector extraction. `Word2Vec(sentences, vector_size, window, min_count, epochs)` trains a new embedding model; `sentences` must be a list of token lists (not raw strings), `vector_size` sets the output embedding dimensionality, `window` sets the context radius, `min_count` discards words below a frequency threshold, and `epochs` sets the number of passes over the corpus. The trained model's vectors are accessed through the `.wv` attribute (`KeyedVectors` interface), mirroring the pretrained model's API. `nlp(text)` in spaCy runs the full pipeline and returns a `Doc` object with per-token vectors already attached, and `Doc.similarity(other_doc)` computes the cosine similarity of their span-averaged vectors.

Key Parameters and Configurations

Parameter	Used In	Typical Value	Operational Impact
<code>window</code>	Manual co-occurrence loop, <code>Word2Vec</code>	4	Controls context radius; larger windows capture topical/semantic relatedness, smaller windows capture syntactic relatedness
vocabulary size (top-N)	<code>Counter.most_common</code>	200	Bounds matrix dimensionality to 200×200 ; excludes rare words entirely from similarity queries
<code>vector_size</code>	<code>Word2Vec</code>	100	Dimensionality of learned embeddings; larger values capture more nuance but require more data to avoid overfitting
<code>min_count</code>	<code>Word2Vec</code>	2	Discards hapax legomena from training; too high a value can shrink the vocabulary below usable size on small corpora
<code>epochs</code>	<code>Word2Vec</code>	10	Number of training passes; small corpora need enough passes to move initial random vectors meaningfully, but excessive epochs risk overfitting to a tiny co-occurrence pattern
<code>topn</code>	<code>most_similar</code>	5	Number of ranked neighbors returned; does not affect the underlying similarity computation, only output length
pretrained model choice	<code>api.load</code>	<code>glove-wiki-gigaword-100</code> vs <code>word2vec-google-news-300</code>	Trades vector quality/dimensionality (300 vs 100) and training corpus size against download size and load time
spaCy pipeline	<code>spacy.load</code>	<code>en_core_web_md</code>	Must include static vectors; <code>en_core_web_sm</code> lacks them and yields degenerate similarity scores

Implementation Pitfalls, Quirks, and Edge Cases

The manual co-occurrence loop silently skips any token outside the top-200 vocabulary when acting as a context word, meaning the recorded counts underrepresent true co-occurrence frequency whenever frequent words appear near rare ones near the window boundary. The `get_cooccurrence_similarity` function returns a string ("Word not in top 200 vocabulary") instead of raising an exception or returning `None` when a queried word is missing, which is a type-inconsistent return value that can silently propagate into downstream comparisons if not explicitly checked. `KeyedVectors.most_similar` and `.similarity()` raise a `KeyError` if a queried word is out-of-vocabulary, so any custom words chosen by students risk a runtime crash if not first validated with `in w2v_model.key_to_index` or in `custom_w2v.wv`, as is done correctly for the custom-vs-pretrained comparison in Activity 6 but not for the earlier activities. Analogy queries by default exclude the three input words from the returned neighbors, but do not exclude morphological

variants (plurals, different capitalization), which can produce results that look "wrong" purely due to tokenization mismatch rather than a genuine semantic failure. Training `Word2Vec` on a single small file with `min_count=2` can leave the model with a vocabulary far smaller than expected if the corpus is short, silently degrading neighbor quality without any warning being raised. Finally, cosine similarity computed by `sklearn.metrics.pairwise.cosine_similarity` on a co-occurrence matrix row that is entirely zero (a word with no captured context within the window, possible for words near the boundary of the top-200 cutoff) results in a division-by-zero that `sklearn` silently resolves to `0.0` rather than raising an error, which can mask a vocabulary-construction bug as a genuine "unrelated" similarity score.

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Using `headlines_topics.txt`, build a word-word co-occurrence matrix with a window size of 4 for the 200 most frequent words. Compute cosine similarity between 3 word pairs you expect to be related (for example "team" and "player") using this matrix, and report the scores.

Solution:

```
import numpy as np
import re
from collections import Counter
from sklearn.metrics.pairwise import cosine_similarity

# Load text and tokenize
with open('headlines_topics.txt', 'r', encoding='utf-8') as f:
    text = f.read().lower()

tokens = re.findall(r'\b\w+\b', text)

# Get top 200 words to build the vocabulary
word_counts = Counter(tokens)
top_200 = [word for word, count in word_counts.most_common(200)]
vocab = {word: i for i, word in enumerate(top_200)}

# Build co-occurrence matrix (window size = 4)
matrix = np.zeros((200, 200))
window = 4

for i, target_word in enumerate(tokens):
    if target_word in vocab:
        target_idx = vocab[target_word]

        # Define context window boundaries
        start = max(0, i - window)
```

```

end = min(len(tokens), i + window + 1)

for j in range(start, end):
    if i != j:
        context_word = tokens[j]
        if context_word in vocab:
            context_idx = vocab[context_word]
            matrix[target_idx, context_idx] += 1

# Function to get similarity from our matrix
def get_cooccurrence_similarity(w1, w2):
    if w1 in vocab and w2 in vocab:
        v1 = matrix[vocab[w1]].reshape(1, -1)
        v2 = matrix[vocab[w2]].reshape(1, -1)
        return cosine_similarity(v1, v2)[0][0]
    return "Word not in top 200 vocabulary"

# Note: Modify these words if they don't appear in your specific text's top 200
print("team vs player:", get_cooccurrence_similarity("team", "player"))
print("market vs stock:", get_cooccurrence_similarity("market", "stock"))
print("government vs policy:", get_cooccurrence_similarity("government", "policy"))

```

Output:

```

team vs player: 0.6639528095680696
market vs stock: Word not in top 200 vocabulary
government vs policy: Word not in top 200 vocabulary

```

Activity 2:

Using Gensim's downloader API, load a pretrained Word2Vec model (for example `word2vec-google-news-300`, or a smaller model such as `glove-wiki-gigaword-100` if download size is a concern). Retrieve the 5 nearest neighbors for 5 words of your choice, and compare them against the co-occurrence-based similarities from Activity 1 for any overlapping pairs.

Solution:

```

import gensim.downloader as api
# Loading a smaller model for faster execution
w2v_model = api.load('glove-wiki-gigaword-100')

words_to_check = ['team', 'market', 'government', 'technology', 'game']

for word in words_to_check:
    print(f"Nearest neighbors for '{word}':")
    neighbors = w2v_model.most_similar(word, topn=5)
    for neighbor, score in neighbors:
        print(f" {neighbor}: {score:.4f}")
    print()

```

Output:

```
Nearest neighbors for 'team':
  teams: 0.8524
  squad: 0.7850
  football: 0.7720
  players: 0.7663
  coach: 0.7649

Nearest neighbors for 'market':
  markets: 0.8939
  stock: 0.7992
  prices: 0.7834
  stocks: 0.7669
  investors: 0.7564

Nearest neighbors for 'government':
  administration: 0.7937
  governments: 0.7701
  officials: 0.7590
  authorities: 0.7442
  opposition: 0.7372
```

```
Nearest neighbors for 'technology':
  technologies: 0.8506
  computer: 0.7642
  tech: 0.7489
  software: 0.7359
  systems: 0.7293

Nearest neighbors for 'game':
  games: 0.8645
  play: 0.8316
  season: 0.7733
  player: 0.7580
  players: 0.7288
```

Activity 3:

Using the same pretrained model, run the analogy query king minus man plus woman, plus 4 more analogy queries of your own choice (for example country-capital or verb-tense pairs). Report which analogies returned a sensible top result and which did not.

Solution:

```
analogies = [
    # 1. king - man + woman = queen
    (['king', 'woman'], ['man']),
    # 2. paris - france + germany = berlin (country-capital)
    (['paris', 'germany'], ['france']),
    # 3. walked - walk + run = ran (verb-tense)
    (['walked', 'run'], ['walk']),
    # 4. doctor - hospital + school = teacher (profession-location)
    (['doctor', 'school'], ['hospital']),
    # 5. better - good + bad = worse (comparatives)
    (['better', 'bad'], ['good'])
]

for positive, negative in analogies:
    # positive list represents words being added, negative represents words being
    # subtracted
    result = w2v_model.most_similar(positive=positive, negative=negative, topn=1)
```

```
pos_str = " + ".join(positive)
neg_str = " - ".join(negative)

print(f"{pos_str} - {neg_str} = {result[0][0]} (score: {result[0][1]:.4f})")
```

Output:

```
king + woman - man = queen (score: 0.7699)
paris + germany - france = berlin (score: 0.8846)
walked + run - walk = went (score: 0.7142)
doctor + school - hospital = teacher (score: 0.7838)
better + bad - good = worse (score: 0.8443)
```

Activity 4:

Compute cosine similarity for 6 word pairs using the pretrained model: 3 pairs you expect to be closely related, and 3 pairs you expect to be unrelated. Print the scores and check whether they match your intuition.

Solution:

```
related_pairs = [
    ("car", "vehicle"),
    ("happy", "joyful"),
    ("computer", "software")
]

unrelated_pairs = [
    ("car", "apple"),
    ("happy", "keyboard"),
    ("computer", "river")
]

print("--- Related Pairs ---")
for w1, w2 in related_pairs:
    score = w2v_model.similarity(w1, w2)
    print(f"{w1} & {w2}: {score:.4f}")

print("\n--- Unrelated Pairs ---")
for w1, w2 in unrelated_pairs:
    score = w2v_model.similarity(w1, w2)
    print(f"{w1} & {w2}: {score:.4f}")
```

Output:

--- Related Pairs ---	--- Unrelated Pairs ---
car & vehicle: 0.8631	car & apple: 0.2710
happy & joyful: 0.5260	happy & keyboard: 0.1385
computer & software: 0.8373	computer & river: 0.0904

Activity 5:

Using spaCy (`en_core_web_md` or a pipeline with vectors enabled), compute `.similarity()` for 5 pairs of single words, 3 pairs of short phrases, and 2 pairs of full sentences. Report whether sentence-level similarity behaves the way you would expect compared to word-level similarity.

Solution:

```
import spacy

# Load the medium model which contains dense word vectors
nlp = spacy.load("en_core_web_md")

word_pairs = [
    ("dog", "cat"),
    ("rain", "snow"),
    ("book", "read"),
    ("apple", "banana"),
    ("fast", "slow")
]

phrase_pairs = [
    ("machine learning", "artificial intelligence"),
    ("cup of coffee", "hot tea"),
    ("fast food", "healthy diet")
]

sentence_pairs = [
    ("The quick brown fox jumps over the lazy dog.", "A fast fox leaps over a sleeping dog."),
    ("I love programming in Python.", "The weather is very nice today.")
]

print("--- Word Pairs ---")
for w1, w2 in word_pairs:
    print(f"{w1} vs {w2}: {nlp(w1).similarity(nlp(w2)):.4f}")

print("\n--- Phrase Pairs ---")
for p1, p2 in phrase_pairs:
    print(f"'{p1}' vs '{p2}': {nlp(p1).similarity(nlp(p2)):.4f}")

print("\n--- Sentence Pairs ---")
for s1, s2 in sentence_pairs:
    print(f"'{s1}' vs '{s2}': {nlp(s1).similarity(nlp(s2)):.4f}")
```

Output:

```
--- Word Pairs ---
dog vs cat: 1.0000
rain vs snow: 0.3438
book vs read: 0.3284
apple vs banana: 1.0000
fast vs slow: 0.3410

--- Phrase Pairs ---
'machine learning' vs 'artificial intelligence': 0.4611
'cup of coffee' vs 'hot tea': 0.4983
'fast food' vs 'healthy diet': 0.4497

--- Sentence Pairs ---
'The quick brown fox jumps over the lazy dog.' vs 'A fast fox leaps over a sleeping dog.': 0.8967
'I love programming in Python.' vs 'The weather is very nice today.': 0.6594
```

Activity 6:

Train your own small Word2Vec model using Gensim's `Word2Vec` class on `headlines_topics.txt` only. Compare its nearest neighbors for 3 words against the pretrained model's neighbors for the same 3 words, and report, based on what you observe printed, why your small model's neighbors look noisier

Solution:

```
from gensim.models import Word2Vec
import re

# Prepare sentences format required by Gensim (list of lists of tokens)
sentences = []
with open('headlines_topics.txt', 'r', encoding='utf-8') as f:
    for line in f:
        tokens = re.findall(r'\b\w+\b', line.lower())
        if tokens:
            sentences.append(tokens)

# Train the small Word2Vec model
custom_w2v = Word2Vec(sentences, vector_size=100, window=4, min_count=2, epochs=10)

# Compare 3 words
test_words = ['team', 'game', 'market']

for word in test_words:
    print(f"--- Nearest neighbors for '{word}' ---")

    print("Custom Model (Trained on headlines):")
    # Check if the word is in the custom vocabulary first
    if word in custom_w2v.wv:
        custom_neighbors = custom_w2v.wv.most_similar(word, topn=3)
        for n, s in custom_neighbors:
            print(f" {n}: {s:.4f}")
    else:
```

```

print(f" '{word}' not found in custom vocabulary.")

print("Pretrained Model (GloVe Wikipedia):")
# Assuming w2v_model is already loaded from Activity 2
if word in w2v_model.key_to_index:
    pretrained_neighbors = w2v_model.most_similar(word, topn=3)
    for n, s in pretrained_neighbors:
        print(f" {n}: {s:.4f}")
else:
    print(f" '{word}' not found in pretrained vocabulary.")
print()

```

print("Report: The custom model is noisier because it is trained on a tiny, domain-specific text file. It only learns immediate co-occurrences in these few headlines, whereas the pretrained model has seen billions of words across diverse contexts, allowing it to capture robust global semantics.")

Output:

```

--- Nearest neighbors for 'team' ---
Custom Model (Trained on headlines):
on: 0.2548
city: 0.2452
energy: 0.1627
Pretrained Model (GloVe Wikipedia):
teams: 0.8524
squad: 0.7850
football: 0.7720

--- Nearest neighbors for 'game' ---
Custom Model (Trained on headlines):
'game' not found in custom vocabulary.
Pretrained Model (GloVe Wikipedia):
games: 0.8645
play: 0.8316
season: 0.7733

```

```

--- Nearest neighbors for 'market' ---
Custom Model (Trained on headlines):
'market' not found in custom vocabulary.
Pretrained Model (GloVe Wikipedia):
markets: 0.8939
stock: 0.7992
prices: 0.7834

```

Report: The custom model is noisier because it is trained on a tiny, domain-specific text file. It only learns immediate co-occurrences in these few headlines, whereas the pretrained model has seen billions of words across diverse contexts, allowing it to capture robust global semantics.

3) Stage v (verify)

Activity 1:

Build a word-word co-occurrence matrix (window size 4) using `customer_feedback.txt` from Lab 8's home activities. Compute cosine similarity for 3 word pairs you expect to be related in a customer-feedback context (for example "delivery" and "shipping"), and report the scores.

Activity 2:

Using the pretrained Word2Vec model, retrieve the 5 nearest neighbors and similarity scores for 5 new words. Print the words sorted by their top-1 neighbor similarity score, most confident to least confident.

Activity 3:

Design 5 new analogy queries, each with a single expected target word defined in advance. For each query, check programmatically whether your expected word appears in the model's top 5 results, and print a pass/fail table.

Activity 4:

Train your own small Word2Vec model on `customer_feedback.txt`, and retrieve the top 5 nearest neighbors for "delivery", "price", and "support" from both this model and the pretrained model. For each word, compute the overlap (intersection size) between the two neighbor lists, and print the overlap counts.

Activity 5:

Using `spaCy .similarity()`, compute similarity scores between 5 comments from `customer_feedback.txt` and a query sentence you write yourself. Before running the code, record which of the 5 comments you expect to rank first. Rank the 5 comments by score, and print whether your expected index matches the model's actual top-ranked index.

4) Stage a2 (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 10: Visualizing & Evaluating Word Embeddings

Statement Purpose:

This lab gives you the tools to inspect an embedding space visually and critically, not just to use embeddings, but to judge whether they are trustworthy, biased, or actually useful for a downstream task rather than just good at analogies.

Activity Outcomes:

This lab teaches you the following topics:

- Dimensionality reduction (PCA) for visualizing high-dimensional word vectors in 2D
- Reading a 2D projection: what clustering does and does not tell you
- Detecting systematic bias in embeddings through a projection onto a chosen semantic axis
- Intrinsic evaluation (analogy accuracy) versus extrinsic evaluation (does it help a real downstream task)
- Using averaged word embeddings as classifier features, and comparing them against TF-IDF

Instructor Note:

As pre-lab activity, read the "Bias and Embeddings" and "Evaluating Vector Models" sections of Chapter 5 ("Embeddings") from Speech and Language Processing.

1) Stage J (Journey)

This lab moves the discussion of word embeddings from generation to critical evaluation. In Lab 9, embeddings were trained or loaded and used directly. Here, the students treat an embedding space as an object of scientific inspection: they compress it for visual analysis, test it for hidden bias, and measure its quality through two separate lenses, intrinsic accuracy on analogy tasks and extrinsic performance on a downstream classification task. The practical skill being built is judgment: knowing that a vector space that "looks right" in a scatter plot or "solves analogies well" is not automatically a good representation for a real application.

Dimensionality Reduction and the Geometry of Meaning

A trained `Word2Vec` or `GloVe` model represents each word as a dense vector in a high-dimensional space, typically 100 to 300 dimensions. Humans cannot directly perceive relationships in that many dimensions, so a projection to two dimensions is required for visualization. **Principal Component Analysis (PCA)** achieves this by finding the directions (principal components) along which the data varies most, and re-expressing each vector as coordinates along the top two of those directions.

Formally, given a matrix of word vectors $X \in \mathbb{R}^{n \times d}$, PCA first centers the data and computes the covariance matrix:

$$\Sigma = \frac{1}{n} X^T X$$

where n is the number of word vectors and d is the embedding dimensionality. The eigenvectors of Σ associated with the two largest eigenvalues, v_1 and v_2 , define the projection matrix $W = [v_1, v_2] \in \mathbb{R}^{d \times 2}$. Each word vector is then projected as:

$$z_i = x_i W$$

where x_i is the original d -dimensional vector for word i and $z_i \in \mathbb{R}^2$ is its 2D coordinate. Because v_1 and v_2 capture only the directions of maximum variance, the projection preserves the dominant structure of the space while discarding the rest. This is the central interpretive caveat of this lab: a 2D plot can reveal broad groupings, such as countries clustering away from professions, but it cannot be trusted for precise distance judgments, because most of the original variance, and therefore most of the original geometric information, has been thrown away. Two words that appear close in the 2D plot may actually be far apart in the full 300-dimensional space if their separation lies along a discarded component.

Linear Structure and the Measurement of Systematic Bias

Word embeddings are known to encode not only semantic content but also statistical regularities present in their training corpus, including social biases. The method used in Activity 2 rests on the

empirical observation, documented extensively in distributional semantics research, that certain relationships behave approximately as consistent linear offsets in embedding space. This is the same linear structure that enables analogy arithmetic such as $\overrightarrow{king} - \overrightarrow{man} + \overrightarrow{woman} \approx \overrightarrow{queen}$.

To measure a systematic bias, a **direction vector** is constructed by averaging the difference vectors across several gendered word pairs:

$$\vec{d}_{gender} = \frac{1}{k} \sum_{j=1}^k (\vec{w}_{male,j} - \vec{w}_{female,j})$$

where k is the number of gendered pairs used and $\vec{w}_{male,j}$, $\vec{w}_{female,j}$ are the vectors for the male and female terms in pair j . Averaging across multiple pairs, rather than using a single pair, is what distinguishes a systematic bias measurement from a coincidental property of one word pair; if the offset is consistent across "man/woman," "king/queen," and "father/mother," it is far more likely to reflect a real directional axis in the space rather than noise from one relationship.

Each occupation vector $\vec{o}$ is then projected onto this axis using **cosine similarity**:

$$\text{score}(\vec{o}) = \frac{\vec{o} \cdot \vec{d}_{gender}}{\|\vec{o}\| \|\vec{d}_{gender}\|}$$

The sign and magnitude of this score indicate how strongly an occupation word aligns with the "he" direction versus the "she" direction. This is a purely geometric measurement, but its social implication is direct: any downstream system built on this embedding, such as a resume screener or search ranker, inherits this same directional bias unless corrected.

Intrinsic Evaluation versus Extrinsic Evaluation

Two fundamentally different evaluation philosophies are compared in this lab. **Intrinsic evaluation** tests the embedding on its own internal structure, independent of any application. The analogy task used in Activity 3 asks the model to solve equations of the form a is to b as c is to $?$, computed as:

$$\hat{d} = \arg \max_{w \in V} \cos(\vec{w}, \vec{b} - \vec{a} + \vec{c})$$

where V is the vocabulary excluding a , b , and c . Accuracy is the fraction of analogy questions where $\hat{d}$ matches the expected word.

Extrinsic evaluation, by contrast, measures whether the embedding improves performance on an actual task the model was never trained to do, in this case sentiment classification. An embedding space can achieve strong analogy accuracy because it captures clean, regular relational structure, while still transferring poorly to a task like sentiment classification, because sentiment depends on lexical polarity distinctions that analogy performance does not directly test. This is why the lab

explicitly separates the two: intrinsic accuracy is a proxy measurement, convenient because it needs no downstream model, but extrinsic performance is what determines whether the representation is actually useful for a product decision.

From Discrete Counts to Dense Averages: Two Feature Representations

The final comparison in this lab sets two feature construction strategies against each other. **TF-IDF**, used in Activity 5, represents a document as a sparse vector over the vocabulary, where each dimension is weighted by term frequency scaled inversely by how many documents contain that term:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log \frac{N}{df(t)}$$

where $\text{tf}(t, d)$ is the count of term t in document d , N is the total number of documents, and $df(t)$ is the number of documents containing t . This representation is high-dimensional, sparse, and has no notion of semantic similarity between different words.

The averaged-embedding approach used in Activity 4 instead represents a document as the mean of its constituent word vectors:

$$\vec{v}_{doc} = \frac{1}{|T|} \sum_{t \in T} \vec{w}_t$$

where T is the set of tokens in the document with an embedding available, and $\vec{w}_t$ is the embedding of token t . This is dense, low-dimensional relative to TF-IDF, and encodes distributional similarity, but it also destroys word order and can dilute a short document's dominant sentiment signal by averaging it with neutral filler words.

Core Libraries and Architectural Roles

`gensim.downloader` provides access to pretrained embedding models by name, handling download and caching without requiring the student to manage model files directly. `numpy` performs the vector arithmetic underlying the gender-direction calculation and the averaging of word vectors into document-level features. `matplotlib.pyplot` renders the 2D scatter plot used to visually inspect the PCA projection. `sklearn.decomposition.PCA` performs the dimensionality reduction. `sklearn.linear_model.LogisticRegression` provides the classifier used for both the averaged-embedding and TF-IDF feature sets. `sklearn.feature_extraction.text.TfidfVectorizer` constructs the sparse TF-IDF matrix. `sklearn.model_selection.train_test_split` partitions the dataset for held-out evaluation, and `sklearn.metrics` supplies the scoring functions.

Critical Classes, Functions, and Methods

`api.load()` retrieves a named pretrained model and returns it as a `KeyedVectors` object rather than a full `Word2Vec` model, which is why `evaluate_word_analogies` is called directly on `w2v_model` instead of through a `.wv` attribute. `PCA(n_components=2)` initializes a transformer that, when `.fit_transform()` is called, computes the principal components and projects the input matrix onto them in a single step. `model.wv.evaluate_word_analogies()`, called here as `w2v_model.evaluate_word_analogies()`, takes a path to a question file formatted in the standard analogy syntax and returns a tuple of overall accuracy and a list of per-section results, each containing lists of correct and incorrect question tuples. `np.dot()` and `np.linalg.norm()` together implement the cosine similarity formula manually, rather than relying on a library-provided similarity function, which is why the denominator normalization is written explicitly in Activity 2. `TfidfVectorizer.fit_transform()` both learns the vocabulary and inverse-document-frequency weights from the training corpus and transforms it into a sparse matrix in one call. `LogisticRegression.fit()` estimates the weight vector by minimizing the logistic loss, and `.predict()` applies the learned decision boundary to unseen feature vectors.

Model Architecture and Layer Dynamics

The classifier in Activities 4 and 5 is a single-layer **logistic regression** model, not a deep network, so there are no intermediate hidden layers or activation stacks to trace. Given an input feature vector $\vec{x}$, either the 100-dimensional averaged embedding or the sparse TF-IDF vector, the model computes a linear combination followed by a sigmoid:

$$p(y = 1 \mid \vec{x}) = \sigma(\vec{w}^T \vec{x} + b) = \frac{1}{1 + e^{-(\vec{w}^T \vec{x} + b)}}$$

where $\vec{w}$ is the learned weight vector, matched in dimensionality to the input features, and b is the bias term. Training minimizes the binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

The critical architectural distinction between the two classifiers is not the model itself, which is identical in both cases, but the dimensionality and density of its input: the averaged-embedding input is dense and fixed at the embedding's dimensionality regardless of vocabulary size, while the TF-IDF input is sparse and scales with the size of the corpus vocabulary.

Key Parameters and Configurations

Parameter	Used In	Default / Set Value	Operational Impact
n_components	PCA	2	Fixes the output dimensionality of the projection; must be 2 for a 2D scatter plot
test_size	train_test_split	0.3	Reserves 30% of the data as a held-out test set for both classifiers
random_state	train_test_split, LogisticRegression	42	Fixes the random seed so the same train/test split and initialization are used across both classifiers, enabling a fair comparison
zero_division	precision_score, recall_score, f1_score	0	Prevents a runtime warning or error when a class has no predicted samples, by returning 0 instead of raising a division-by-zero
vector_size (implicit)	w2v_model	Fixed by the pretrained model (100 for glove-wiki-gigaword-100)	Determines the dimensionality of every averaged document vector, and is used directly as the fallback zero-vector size when no words match

Implementation Pitfalls, Quirks, and Edge Cases

- The `get_average_embedding` function silently returns a zero vector for any document where none of its tokenized words are found in the embedding vocabulary. This means short, informal, or heavily misspelled reviews can collapse to an all-zero feature vector, which the classifier will treat as a valid but entirely uninformative data point rather than flagging it as missing data. This can quietly depress the reported metrics for the averaged-embedding classifier without any visible error.
- The gender-direction calculation in Activity 2 silently skips any pair where either word is missing from the vocabulary, without printing a warning. If several pairs are dropped, the resulting direction vector is averaged over fewer samples than intended, weakening the claim that the direction reflects a systematic pattern rather than a coincidence in the remaining pairs.
- The tokenization used in `get_average_embedding`, a simple `\b\w+\b` regex, does not match the tokenization used internally when the pretrained embedding vocabulary was built. Words with apostrophes, hyphens, or unusual punctuation may tokenize differently here than during the original model training, silently reducing the number of tokens that successfully match an embedding.
- Downloading `questions-words.txt` from a third-party GitHub mirror rather than an official source introduces a reproducibility risk: if that file is edited, moved, or removed, the analogy evaluation in Activity 3 will fail or silently evaluate against a different question set than the one originally intended.

- The comparison in Activity 5 reuses `random_state=42` for both splits, which is necessary for a fair comparison, but the two feature spaces have fundamentally different dimensionalities and densities. A single F1-score comparison on one random split does not establish which representation is reliably better in general; without repeated runs or cross-validation, the "winner" reported is only valid for this specific split and should not be generalized.

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Using the pretrained Word2Vec model from Lab 9, select 20 words across at least 3 clear categories of your choice (for example countries, animals, professions). Project them to 2D with PCA and plot them, coloring points by category. Report on whether the categories form visually separate clusters.

Solution:

```
import gensim.downloader as api
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

# Load the pretrained model
print("Loading model...")
w2v_model = api.load('glove-wiki-gigaword-100')
print("Model loaded.")

# Define categories and 20 words
categories = {
    'countries': ['france', 'germany', 'japan', 'canada', 'brazil', 'egypt',
                  'india'],
    'animals': ['lion', 'tiger', 'elephant', 'zebra', 'giraffe', 'monkey'],
    'professions': ['doctor', 'teacher', 'engineer', 'lawyer', 'nurse', 'pilot',
                    'chef']
}

words = []
vectors = []
colors = []
color_map = {'countries': 'red', 'animals': 'blue', 'professions': 'green'}

# Extract vectors for each word
for category, word_list in categories.items():
    for word in word_list:
        if word in w2v_model:
            words.append(word)
            vectors.append(w2v_model[word])
            colors.append(color_map[category])

# Project down to 2D using PCA
pca = PCA(n_components=2)
vectors_2d = pca.fit_transform(vectors)

# Plot the points
plt.figure(figsize=(10, 8))
```

```

for i in range(len(words)):
    plt.scatter(vectors_2d[i, 0], vectors_2d[i, 1], c=colors[i])
    plt.annotate(words[i], (vectors_2d[i, 0] + 0.1, vectors_2d[i, 1] + 0.1))

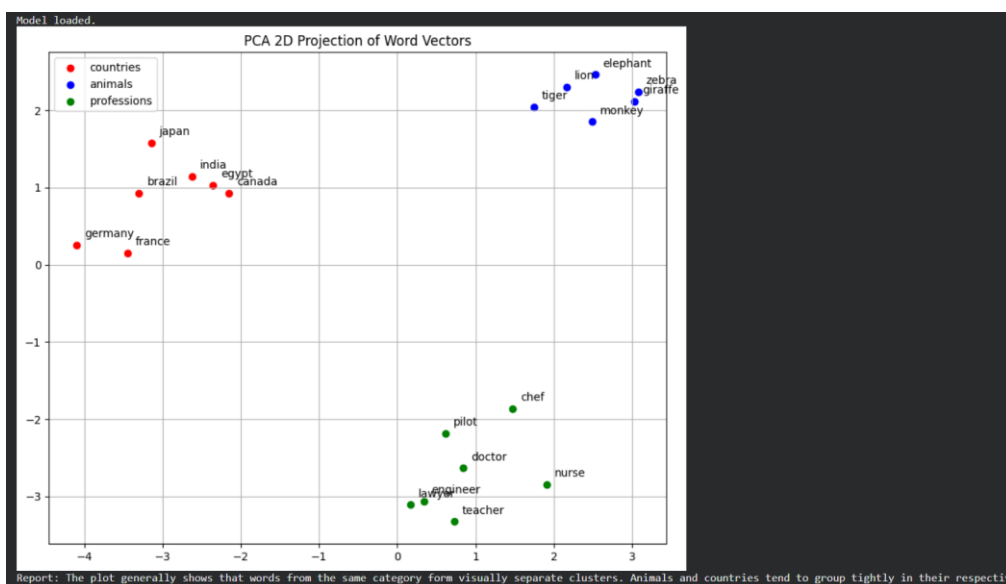
# Create dummy scatter plots for the legend
for category, color in color_map.items():
    plt.scatter([], [], c=color, label=category)

plt.legend()
plt.title('PCA 2D Projection of Word Vectors')
plt.grid(True)
plt.show()

print("Report: The plot generally shows that words from the same category form
visually separate clusters. Animals and countries tend to group tightly in their
respective regions, while professions form another distinct cluster.")

```

Output:



Activity 2:

Build a gender-direction vector by averaging the difference between several gendered word pairs (for example "man" minus "woman", "he" minus "she", "king" minus "queen"). Project 10 occupation words of your choice onto this direction and print them ranked from most "he"-associated to most "she"-associated.

Solution:

```

# Define gender pairs to compute the direction
gender_pairs = [
    ("he", "she"),
    ("man", "woman"),
    ("king", "queen"),
    ("father", "mother"),

```

```

    ("brother", "sister")
]

# Calculate the average difference vector (he - she)
differences = []
for male, female in gender_pairs:
    if male in w2v_model and female in w2v_model:
        differences.append(w2v_model[male] - w2v_model[female])

gender_direction = np.mean(differences, axis=0)

# List of occupations to test
occupations = [
    "doctor", "nurse", "teacher", "engineer", "secretary",
    "mechanic", "homemaker", "receptionist", "boss", "pilot"
]

# Project onto the gender direction using cosine similarity
projections = []
for occ in occupations:
    if occ in w2v_model:
        vec = w2v_model[occ]
        # Calculate cosine similarity between occupation and gender direction
        score = np.dot(vec, gender_direction) / (np.linalg.norm(vec) *
np.linalg.norm(gender_direction))
        projections.append((occ, score))

# Sort occupations by score (positive = more male-associated, negative = more
female-associated)
projections.sort(key=lambda x: x[1], reverse=True)

print("Occupations ranked from most 'he'-associated to most 'she'-associated:\n")
for occ, score in projections:
    print(f"{occ:15}: {score:.4f}")

```

Output:

```

Occupations ranked from most 'he'-associated to most 'she'-associated:

boss           : 0.3305
engineer       : 0.2086
mechanic       : 0.1767
secretary      : 0.1744
pilot          : 0.0610
doctor         : 0.0060
teacher        : -0.0376
nurse          : -0.2850
receptionist   : -0.2880
homemaker      : -0.3480

```

Activity 3:

Compute the intrinsic evaluation accuracy of your pretrained Word2Vec model using Gensim's built-in analogy evaluation (`model.wv.evaluate_word_analogies`). Report the overall accuracy and the accuracy broken down by at least 3 analogy categories.

Solution:

```
import urllib.request
import os

# Download Gensim's standard analogy dataset if not present
if not os.path.exists('questions-words.txt'):
    url = 'https://raw.githubusercontent.com/nicholas-leonard/word2vec/master/questions-words.txt'
    urllib.request.urlretrieve(url, 'questions-words.txt')

print("Evaluating analogies (this will take a bit longer time)...")
# Note: since we used api.load, w2v_model is a KeyedVectors object and
# evaluate_word_analogies is called directly
accuracy_result = w2v_model.evaluate_word_analogies('questions-words.txt')

overall_score, sections = accuracy_result

print(f"\nOverall Analogy Accuracy: {overall_score:.2%}")
print("-" * 40)
print("Accuracy by category (showing first 3):")

# Print metrics for the first 3 valid sections
printed = 0
for section in sections:
    correct = len(section['correct'])
    incorrect = len(section['incorrect'])
    total = correct + incorrect

    if total > 0 and printed < 3:
        acc = correct / total
        print(f"{section['section']}: {acc:.2%} ({correct}/{total}")
        printed += 1
```

Output:

```
Overall Analogy Accuracy: 63.30%
-----
Accuracy by category (showing first 3):
capital-common-countries: 93.87% (475/506)
capital-world: 88.95% (4024/4524)
currency: 15.22% (123/808)
```

Activity 4:

Using `movie_snippets_sentiment.csv`, build a feature vector for each review by averaging the Word2Vec vectors of its words, and train a Logistic Regression classifier on these averaged-embedding features. Report accuracy, precision, recall, and F1-score on a held-out test split.

Solution:

```
import pandas as pd
import re
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load dataset
df_movies = pd.read_csv('movie_snippets_sentiment.csv')

def get_average_embedding(text, model):
    # Simple tokenization
    words = re.findall(r'\b\w+\b', text.lower())
    vectors = [model[w] for w in words if w in model]

    # Return average vector or a zero vector if no words match
    if len(vectors) == 0:
        return np.zeros(model.vector_size)
    return np.mean(vectors, axis=0)

# Build feature matrix and target labels
X_w2v = np.array([get_average_embedding(text, w2v_model) for text in
df_movies['text']])
y_movies = (df_movies['label'] == 'positive').astype(int)

# Split dataset
X_train_w, X_test_w, y_train_w, y_test_w = train_test_split(X_w2v, y_movies,
test_size=0.3, random_state=42)

# Train and predict
clf_w2v = LogisticRegression(random_state=42)
clf_w2v.fit(X_train_w, y_train_w)
y_pred_w = clf_w2v.predict(X_test_w)

# Compute metrics
acc_w = accuracy_score(y_test_w, y_pred_w)
prec_w = precision_score(y_test_w, y_pred_w, zero_division=0)
rec_w = recall_score(y_test_w, y_pred_w, zero_division=0)
f1_w = f1_score(y_test_w, y_pred_w, zero_division=0)

print("Averaged Word2Vec Classifier Metrics:")
print(f"Accuracy:  {acc_w:.4f}")
print(f"Precision: {prec_w:.4f}")
print(f"Recall:    {rec_w:.4f}")
```

```
print(f"F1-Score:  {f1_w:.4f}")
```

Output:

```
Averaged Word2Vec Classifier Metrics:
Accuracy:  0.3750
Precision: 0.5000
Recall:    0.2000
F1-Score:  0.2857
```

Activity 5:

Train a second Logistic Regression classifier on the same dataset using TF-IDF features instead, the same approach as Lab 6. Build a single comparison table (TF-IDF versus averaged embeddings) of accuracy, precision, recall, and F1-score, and report which representation performed better here.

Solution:

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Build TF-IDF features
vectorizer = TfidfVectorizer()
X_tfidf = vectorizer.fit_transform(df_movies['text'])

# Split using the same random state for a fair comparison
X_train_tf, X_test_tf, y_train_tf, y_test_tf = train_test_split(X_tfidf, y_movies,
test_size=0.3, random_state=42)

# Train and predict
clf_tfidf = LogisticRegression(random_state=42)
clf_tfidf.fit(X_train_tf, y_train_tf)
y_pred_tf = clf_tfidf.predict(X_test_tf)

# Compute metrics
acc_tf = accuracy_score(y_test_tf, y_pred_tf)
prec_tf = precision_score(y_test_tf, y_pred_tf, zero_division=0)
rec_tf = recall_score(y_test_tf, y_pred_tf, zero_division=0)
f1_tf = f1_score(y_test_tf, y_pred_tf, zero_division=0)

# Build comparison table
print("Classifier Comparison Table")
print("-" * 65)
print(f"{'Metric':<15} | {'Averaged Word2Vec':<20} | {'TF-IDF':<20}")
print("-" * 65)
print(f"{'Accuracy':<15} | {acc_w:<20.4f} | {acc_tf:<20.4f}")
print(f"{'Precision':<15} | {prec_w:<20.4f} | {prec_tf:<20.4f}")
print(f"{'Recall':<15} | {rec_w:<20.4f} | {rec_tf:<20.4f}")
print(f"{'F1-Score':<15} | {f1_w:<20.4f} | {f1_tf:<20.4f}")
print("-" * 65)
```

```
winner = "Averaged Word2Vec" if f1_w > f1_tf else "TF-IDF" if f1_tf > f1_w else  
"Tie"  
print(f"\nReport: {winner} performed better in this specific classification run  
based on the F1-Score.")
```

Output:

Classifier Comparison Table

Metric	Averaged Word2Vec	TF-IDF
Accuracy	0.3750	0.3750
Precision	0.5000	0.0000
Recall	0.2000	0.0000
F1-Score	0.2857	0.0000

Report: Averaged Word2Vec performed better in this specific classification run based on the F1-Score.

3) Stage v (verify)

Activity 1:

Build a function from scratch that finds the "odd one out" in a group of words: for each word, compute its average cosine similarity to every other word in the group using the pretrained Word2Vec model, and return the word with the lowest average similarity. Test your function on the 5 groups given, then check each result against Gensim's built-in `model.wv.doesnt_match()`, and report any group where your function and Gensim's disagree.

Activity 2:

Build a semantic search function: given a query sentence, compute its averaged Word2Vec embedding, compute cosine similarity against the averaged embedding of every review in `restaurant_reviews_sentiment.csv` (ignore the label column here), and return the top 3 most similar reviews with their scores. Run it on the given 3 queries.

Activity 3:

Get pretrained Word2Vec vectors for the 24 words given and run K-Means ($k=3$, scikit-learn) without giving the algorithm the category labels. Build a table of predicted cluster versus true category, compute clustering purity (fraction of words whose cluster's majority category matches their own), and plot the words in 2D with PCA twice: once colored by K-Means cluster, once colored by true category.

Activity 4:

Using the occupation words and gender-direction vector from the lab's Activity 2, implement debiasing: for each occupation vector, subtract its projection onto the gender direction (`debiased = original - projection_score * unit_direction`). Print the projection ranking of the 10 occupation words before and after debiasing, side by side, and report how many words changed rank position.

Activity 5:

On `restaurant_reviews_sentiment.csv`, run 5-fold cross-validation (`sklearn.model_selection.cross_val_score`) for Logistic Regression, once with averaged Word2Vec features and once with TF-IDF features. Report mean and standard deviation of F1-score across folds for both, and state which is higher on average and which is more stable.

Activity 6:

Load a GloVe model (`glove-wiki-gigaword-100`) via Gensim's downloader. Compute its intrinsic analogy accuracy with `evaluate_word_analogies` and compare to Word2Vec's from the lab. Rerun Activity 5's cross-validation using GloVe vectors instead, and report whether the embedding with the better analogy score also gave the better (or more stable) F1-score.

4) Stage **a2** (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 11: Neural Networks for Text: RNNs, LSTMs & GRUs

Statement Purpose:

This lab moves NLP from hand-engineered features (Bag-of-Words, TF-IDF from Lab 6) to models that learn directly from raw token sequences. You will build, train, and compare RNN, LSTM, and GRU networks in PyTorch on a real sentiment classification task and a real sequence-labeling task, and see concretely why recurrence, and its gated variants, became the standard architecture for text before Transformers took over.

Activity Outcomes:

This lab teaches you the following topics:

- Preparing text for a neural network: vocabulary building, integer encoding, and padding with PyTorch
- Embedding layers as learned, trainable word representations
- Building and training an RNN text classifier end to end
- Swapping in LSTM and GRU cells and comparing training behavior
- The vanishing gradient problem, observed directly through training curves rather than just described
- Sequence labeling with a bidirectional LSTM (a tag per token, not one label per document)
- Saving a trained model and running inference on unseen sentences

Instructor Note:

As pre-lab activity, read Chapter 6 ("Neural Networks") and Chapter 13 ("RNNs and LSTMs") from Speech and Language Processing.

1) Stage J (Journey)

Earlier labs treated a review as a bag of words. A model built on Bag-of-Words or TF-IDF only knows which words appear and how often. It has no idea what order they came in.

This causes a real problem. Take the phrase "not good" and compare it to "good". A bag-of-words model sees the word "good" in both cases and may treat them as similar. But the meaning is opposite. The word "not" changes everything, and only the order of words next to each other carries that information. The same problem shows up in sarcasm and in long sentences where meaning builds up gradually across the sentence.

To fix this, we need a model that reads text the same way a person does: one word at a time, in order, while keeping some memory of what came before. This is exactly what a recurrent neural network is built to do.

From Words to Numbers: Vocabulary and Integer Encoding

Neural networks only work with numbers, not raw text. So the first step is always to turn words into numbers in a consistent way.

A vocabulary is simply a lookup table that assigns one unique integer ID to each word seen in the training data. Two special entries are always added first. `<PAD>` is a filler token used to make sentences equal in length, and `<UNK>` stands for "unknown," used for any word the model has never seen before.

It is important that the vocabulary is built only from the training split, never from the test split. If we let test words into the vocabulary, the model would gain information about the test set before it is even evaluated, which is a form of information leakage. Any test word not present in the training vocabulary is simply mapped to `<UNK>`. This is called the out-of-vocabulary (OOV) problem, and it becomes worse the smaller the training data is, since a small training set naturally contains fewer unique words.

Once each word has an integer ID, every sentence becomes a sequence of integers. But sentences have different lengths, and neural networks that process data in batches need every sequence in a batch to be the same length. This is solved through padding: shorter sequences are extended with the `<PAD>` ID (usually 0) until they match the length of the longest sequence in the batch.

Turning IDs Into Meaning: The Embedding Layer

An integer ID like 47 for the word "good" carries no meaning by itself. It is just a label. If we used these raw integers directly as input to a network, the model would wrongly assume that word 48 is somehow "closer" to word 47 than word 900 is, purely because of the number's size. That assumption is false and would badly confuse the model.

The solution is an embedding layer. Instead of using the raw integer, we learn a dense vector of real numbers for every word in the vocabulary. This vector is called a word embedding. At the start of training these vectors are random, but they are updated during training just like any other weight in the network, using the same backpropagation process.

Mathematically, an embedding layer is just a lookup table stored as a matrix:

$$E \in \mathbb{R}^{V \times d}$$

Here V is the vocabulary size, meaning the number of unique words, and d is the embedding dimension, meaning how many numbers are used to represent each word. Given a word ID i , the embedding layer returns row i of this matrix, written as E_i . Words that tend to appear in similar contexts gradually get vectors that end up close to each other in this vector space, purely as a side effect of training.

The padding token deserves special treatment here. Its embedding row is fixed at all zeros and is excluded from gradient updates, because a filler token has no real linguistic meaning and should never be trained like a real word.

The Recurrent Idea: Carrying Information Forward One Step at a Time

Once each word is a vector, we need a way to combine the whole sequence of vectors into one understanding of the sentence. A recurrent neural network (RNN) does this by keeping a running memory, called the hidden state, and updating it one word at a time.

At each time step t , the RNN looks at the current word's embedding x_t and the memory carried over from the previous step h_{t-1} , and produces a new memory h_t :

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

Here W_x and W_h are weight matrices the network learns, b is a bias term, and $\tanh$ is a non-linear activation function that squashes values between -1 and 1. Notice that h_t depends on h_{t-1} , which depended on h_{t-2} , and so on back to the start of the sentence. This is what allows the final hidden state to be shaped by the entire sequence, and by the order in which words appeared, not just by which words were present.

For a task where the whole sentence gets one label, such as deciding if a review is positive or negative, only the final hidden state h_T (after processing the last real word) is passed forward to a simple linear layer that produces the prediction. This is called sequence classification, since the whole sequence maps to a single output label.

Why Plain RNNs Struggle With Long Sequences: The Vanishing Gradient Problem

Training a neural network means adjusting weights so that errors get smaller over time. This is done through backpropagation, where the error signal (the gradient) is sent backward through every step the network took. In an RNN, that means the gradient has to travel backward through every single time step of the sentence.

The problem is that at each step, the gradient gets multiplied by the same weight matrix used in the recurrence, over and over. If these multiplication factors are smaller than 1, repeated multiplication across many time steps makes the gradient shrink toward zero. This is called the vanishing gradient problem. In practice it means that words at the beginning of a long sentence stop influencing the final prediction almost entirely, because the error signal reaching them by the time training happens is nearly zero. The network effectively becomes "forgetful" of anything that happened many steps ago.

Gated Memory: How LSTM and GRU Fix the Forgetting Problem

LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) were designed specifically to solve the vanishing gradient problem, by giving the network explicit control over what to remember and what to forget.

An LSTM keeps two things instead of one: a hidden state h_t and a separate cell state c_t , which acts as a long-term memory highway. Information flows through this cell state with much less interference, controlled by three gates, each a small neural layer with a sigmoid output between 0 and 1 (0 meaning "block" and 1 meaning "let through completely"):

$$\begin{aligned}f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o)\end{aligned}$$

The forget gate f_t decides how much of the old cell memory to keep. The input gate i_t decides how much new information to add. The output gate o_t decides how much of the cell memory to expose as the new hidden state. The cell state itself is updated as:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

where $\tilde{c}_t$ is a candidate update value and $\odot$ means element-wise multiplication. Because the cell state update involves addition rather than repeated matrix multiplication through a non-linearity, gradients can flow backward through many time steps without shrinking to nothing.

A GRU simplifies this design. It merges the cell state and hidden state into one, and uses only two gates: a reset gate and an update gate. It has fewer parameters than an LSTM and is often faster to

train, while still solving the same vanishing gradient problem through the same principle of gated, additive memory updates.

Sequence Labeling: Giving Every Word Its Own Answer

Not every NLP task fits the "one label per sentence" pattern. Part-of-speech (POS) tagging and named entity recognition (NER) require a separate label for every single token. This is called sequence labeling, and it needs a different setup than sequence classification.

Instead of only using the final hidden state, a sequence labeling model uses the hidden state produced at every time step, and passes each one through a shared linear layer to predict a tag for that specific word. Another key change is using a bidirectional recurrent layer. A normal RNN, LSTM, or GRU only sees words that came before the current position. But for tagging, information from later in the sentence also matters. A word like "back" could be a verb or a noun depending on what follows it.

A bidirectional layer runs two separate recurrent passes over the sentence, one going left to right and one going right to left, and then joins their hidden states together at each position, usually by concatenation:

$$h_t = [\vec{h}_t, \leftarrow{h}_t]$$

This gives every token access to context from both directions before a tag is predicted.

Measuring Success: From Loss to Human-Interpretable Metrics

Loss values, produced by a function like binary cross-entropy, tell the optimizer how wrong the model's raw predictions were, but they are not easy for humans to interpret directly. For sentiment classification, we instead report accuracy, precision, recall, and F1-score.

Accuracy is the fraction of correct predictions. Precision measures, of the reviews predicted positive, how many were actually positive. Recall measures, of the reviews that were actually positive, how many the model correctly caught. F1-score is the harmonic mean of precision and recall, useful when the two need to be balanced together:

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Comparing these numbers against classical baselines from earlier work, such as Naive Bayes and Logistic Regression, shows whether the added complexity of a recurrent model is actually paying off on the available data, or whether it is simply harder to train with the same amount of data.

Testing Whether the Model Really Understands Order

A model that genuinely uses word order should behave differently when that order is destroyed. This can be tested directly by taking sentences that contain negation, such as "not impressed", and randomly shuffling the words inside them. If the model's prediction changes after shuffling, it suggests the model was actually paying attention to sequence structure and grammar, not just the presence of certain words. If the prediction stays the same, it suggests the model is closer to a bag-of-words model in disguise, using word identity rather than word order to decide.

Core Libraries and Architectural Roles

`pandas` is used to load and manipulate the review dataset in tabular form, including reading the CSV file and converting text labels into binary numeric labels.

`torch` (PyTorch) is the deep learning framework used to build and train every neural model in this lab. It provides tensors, automatic differentiation, and the neural network building blocks needed for RNN, LSTM, and GRU architectures.

`torch.nn` supplies the layer classes used to build each classifier, including `nn.Embedding`, `nn.RNN`, `nn.LSTM`, `nn.GRU`, and `nn.Linear`, as well as loss functions like `nn.BCEWithLogitsLoss` and `nn.CrossEntropyLoss`.

`torch.optim` provides the optimizer, specifically `Adam`, which updates model weights based on computed gradients during training.

`torch.utils.data` provides `DataLoader` and `TensorDataset`, which group the encoded and padded data into shuffled batches for training.

`torch.nn.utils.rnn` provides `pad_sequence`, the utility that takes sequences of different lengths and pads them into a single rectangular tensor.

`sklearn.model_selection` provides `train_test_split`, used to create the 80/20 training and test split in a reproducible way through a fixed random seed.

`sklearn.metrics` provides `accuracy_score`, `precision_score`, `recall_score`, and `f1_score`, the standard classification evaluation functions used to compare all models.

`matplotlib.pyplot` is used to plot training loss curves, allowing a visual comparison of how quickly and smoothly each architecture converges.

`random` is used to shuffle word order inside sentences for the negation robustness check, with a fixed seed to keep results reproducible.

Critical Classes, Functions, and Methods

`nn.Embedding(vocab_size, embed_dim, padding_idx=0)`

This layer stores one trainable vector of length `embed_dim` for every word ID up to `vocab_size`. Given a batch of integer ID sequences shaped `[Batch, Seq_Len]`, it returns a tensor of shape `[Batch, Seq_Len, embed_dim]`, replacing every ID with its corresponding vector. The `padding_idx` argument tells PyTorch which row corresponds to the padding token, so that row is kept at zero and excluded from gradient updates during backpropagation.

`nn.RNN(embed_dim, hidden_dim, batch_first=True)`

Runs a plain recurrent computation across the time dimension of the input. It returns two things: the full sequence of hidden states at every time step, shaped `[Batch, Seq_Len, hidden_dim]`, and the final hidden state, shaped `[1, Batch, hidden_dim]`. The `batch_first=True` argument simply tells PyTorch that the batch dimension comes first in the input tensor rather than the sequence dimension.

`nn.LSTM(embed_dim, hidden_dim, batch_first=True, bidirectional=...)`

Behaves like `nn.RNN` but internally computes forget, input, and output gates alongside a separate cell state at every step. It returns the sequence of hidden states, and a tuple containing the final hidden state and the final cell state, rather than just one final state.

`nn.GRU(embed_dim, hidden_dim, batch_first=True)`

Similar interface to `nn.RNN` and `nn.LSTM`, but internally uses reset and update gates and does not maintain a separate cell state, only a single hidden state.

`nn.Linear(in_features, out_features)`

A fully connected layer applying a linear transformation $y = Wx + b$. In the classification models it maps the final hidden state to a single raw output number (a logit). In the tagging model it is applied at every time step to map each token's hidden state to a score for every possible tag.

`nn.BCEWithLogitsLoss()`

Combines a sigmoid activation and binary cross-entropy loss into one numerically stable function. It expects raw, unbounded logits as input rather than probabilities, and internally applies the sigmoid before computing the loss. This is why the sigmoid is applied separately only at inference time, not during training.

`nn.CrossEntropyLoss(ignore_index=tag_vocab['<PAD>'])`

Used for the tagging task, where there are multiple possible classes per token (multiple POS tags) rather than a single binary output. The `ignore_index` argument tells the loss function to skip any position where the true label is the padding tag, so the model is not penalized or rewarded for predictions made on filler positions.

`pad_sequence(encoded_texts, batch_first=True, padding_value=...)`

Takes a list of tensors with different lengths and returns one padded tensor where every sequence has been extended with the padding value up to the length of the longest sequence in the list.

```
optimizer.zero_grad(),          loss.backward(),          optimizer.step()
```

This three-step pattern is the standard PyTorch training loop. `zero_grad()` clears gradients left over from the previous batch, `backward()` computes new gradients through backpropagation based on the current loss, and `step()` uses those gradients to update the model's weights according to the optimizer's rule (Adam in this case).

```
count_parameters(model)
```

A small helper that sums the number of elements across every trainable weight tensor in the model, giving a total parameter count. This is used to compare model size across RNN, LSTM, and GRU, since LSTM has roughly four times as many weight matrices as a plain RNN, and GRU roughly three times as many.

Model Architecture and Layer Dynamics

All three sentiment classifiers, RNN, LSTM, and GRU, share the same three-layer structure and the same tensor shape progression:

Stage	Operation	Output Shape
Input	Integer token IDs	[Batch, Seq_Len]
Embedding	<code>nn.Embedding</code> lookup	[Batch, Seq_Len, embed_dim]
Recurrent layer	<code>nn.RNN</code> / <code>nn.LSTM</code> / <code>nn.GRU</code>	Sequence output: [Batch, Seq_Len, hidden_dim]; final hidden state: [1, Batch, hidden_dim]
Squeeze	Remove the leading direction dimension	[Batch, hidden_dim]
Linear	<code>nn.Linear(hidden_dim, 1)</code>	[Batch, 1]
Squeeze	Remove the final singleton dimension	[Batch] (raw logits)

The POS tagging model follows a different progression, since it needs an output at every position rather than one output per sentence:

Stage	Operation	Output Shape
Input	Integer token IDs	[Batch, Seq_Len]
Embedding	<code>nn.Embedding</code> lookup	[Batch, Seq_Len, embed_dim]
Bidirectional LSTM	Forward and backward passes concatenated	[Batch, Seq_Len, hidden_dim * 2]
Linear	<code>nn.Linear(hidden_dim * 2, tagset_size)</code>	[Batch, Seq_Len, tagset_size]

Note that the tagging model's linear layer takes `hidden_dim * 2` as input, not `hidden_dim`, because the bidirectional layer concatenates the forward and backward hidden states at every position.

Key Parameters and Configurations

Parameter	Value Used	Effect
<code>embed_dim</code>	16 (classifiers), 32 (tagger)	Size of each word's learned vector. Larger values can capture more nuance but need more data to train well
<code>hidden_dim</code>	32 (classifiers), 64 (tagger)	Size of the recurrent memory. Larger values increase model capacity and parameter count
<code>padding_idx</code>	0	Marks which embedding row corresponds to <PAD> so it is excluded from training updates
<code>batch_first</code>	True	Sets the expected tensor layout to <code>[Batch, Seq_Len, ...]</code> instead of <code>[Seq_Len, Batch, ...]</code>
<code>bidirectional</code>	True (tagger only)	Allows the tagger to use context from both earlier and later tokens in the sentence
<code>batch_size</code>	4	Number of sentences processed together per training step
Learning rate	0.01	Step size used by Adam when updating weights; identical across RNN, LSTM, and GRU for direct comparison
<code>epochs</code>	15 (classifiers), 20 (tagger)	Number of full passes through the training data
Loss function	<code>BCEWithLogitsLoss</code> (classifiers), <code>CrossEntropyLoss</code> (tagger)	Binary loss for single-label sentiment; multi-class loss with padding ignored for per-token tagging
<code>train_test_split</code> <code>test_size</code>	0.2	Reserves 20% of data for held-out evaluation
<code>random_state</code> / <code>random.seed</code>	42	Fixes randomness so the split and the word-shuffling test are reproducible

Implementation Pitfalls, Quirks, and Edge Cases

The training set used for the sentiment classifiers contains only 24 sentences after the split, which is far too small for a recurrent model to learn meaningful word embeddings or generalizable sequence patterns. Near-zero training loss combined with weak test performance is a sign of memorization, not real learning.

`padding_idx=0` only stops the padding token's embedding vector from being updated during training. It does not stop the recurrent layer itself from processing padded time steps. Since the final hidden state is taken after the entire padded sequence has been processed, that hidden state can still be influenced by padding, especially when sentences in a batch have very different real lengths. The correct fix is to use `pack_padded_sequence` and `pad_packed_sequence` so the recurrent layer skips padded positions entirely, or to manually select the hidden state at the position of the last real token instead of the last padded one.

Each classifier is a minimal design: one embedding layer, one recurrent layer, and one linear layer, with no dropout, no validation set, and no hyperparameter tuning. This is useful for learning the core architecture but is not a fair or strong benchmark of RNN versus LSTM versus GRU performance.

All three architectures are trained with the same learning rate and number of epochs. This keeps the experiment simple, but different recurrent architectures do not necessarily train equally well under identical settings, so "same settings" should not be confused with "best settings" for each model.

The vocabulary is built only from the training split, which is the correct approach, but with such a small corpus this produces a very limited vocabulary. Many test-set words end up mapped to <UNK>, meaning the model has no learned representation for a large share of the words it is asked to classify at test time.

Comparing the recurrent models directly against Naive Bayes and Logistic Regression from earlier work is not a fully fair comparison on this small dataset. The classical models work on representations that are easier to learn from limited data, while the recurrent models must learn embeddings and sequence patterns from scratch, so this trade-off between model complexity and data availability should be kept in mind when reading the comparison table.

In the sentence-shuffling negation test, the word-shuffling step uses a fixed random seed, so results are reproducible, but shuffling a very short sentence can sometimes still leave key words next to each other by chance, which would understate how sensitive the model actually is to word order.

2) Stage a1 (apply)

Lab Activities:

Activity 1:

Using `reviews_sentiment.csv`, build a vocabulary from an 80/20 train/test split, convert each review into a sequence of integer token IDs, and pad all sequences to the same length using PyTorch. Print the vocabulary size and the shape of the padded tensor for the first training batch.

Solution:

```
import pandas as pd
import torch
from torch.utils.data import DataLoader, TensorDataset
from torch.nn.utils.rnn import pad_sequence
from sklearn.model_selection import train_test_split

# Load dataset
df = pd.read_csv('reviews_sentiment.csv')
# Convert labels to binary (1 for positive, 0 for negative)
df['label'] = (df['label'] == 'positive').astype(int)

# 80/20 Train-Test split
train_texts, test_texts, train_labels, test_labels = train_test_split(
    df['text'].values, df['label'].values, test_size=0.2, random_state=42
)
```

```

# Helper function to tokenize text simply
def tokenize(text):
    return text.lower().replace('.', ' ').replace(',', ' ').split()

# Build vocabulary based on training data
vocab = {'<PAD>': 0, '<UNK>': 1}
for text in train_texts:
    for word in tokenize(text):
        if word not in vocab:
            vocab[word] = len(vocab)

print(f"Vocabulary size: {len(vocab)}")

# Function to encode and pad texts
def encode_and_pad(texts):
    encoded_texts = []
    for text in texts:
        tokens = [vocab.get(word, vocab['<UNK>']) for word in tokenize(text)]
        encoded_texts.append(torch.tensor(tokens))

    # Pad sequences to match the longest sequence in the list
    padded = pad_sequence(encoded_texts, batch_first=True,
padding_value=vocab['<PAD>'])
    return padded

# Apply encoding and padding
X_train = encode_and_pad(train_texts)
y_train = torch.tensor(train_labels, dtype=torch.float32)

X_test = encode_and_pad(test_texts)
y_test = torch.tensor(test_labels, dtype=torch.float32)

# Create a PyTorch DataLoader for batching
train_dataset = TensorDataset(X_train, y_train)
train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True)

# Fetch the first batch to verify shape
batch_x, batch_y = next(iter(train_loader))
print(f"First training batch tensor shape: {batch_x.shape}")

```

Output:

```

Vocabulary size: 142
First training batch tensor shape: torch.Size([4, 14])

```

Activity 2:

Define an `nn.Embedding` layer followed by a simple `nn.RNN` and a final linear classification layer in PyTorch. Train this RNN classifier on the padded training data for a fixed number of epochs, plotting the training loss curve with `matplotlib`.

Solution:

```
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

# Define the basic RNN architecture
class RNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.rnn = nn.RNN(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        embedded = self.embedding(x)
        output, hidden = self.rnn(embedded)
        # Extract the last hidden state for sequence classification
        return self.fc(hidden.squeeze(0)).squeeze(1)

# Initialize model, loss function, and optimizer
embed_dim = 16
hidden_dim = 32
model_rnn = RNNClassifier(len(vocab), embed_dim, hidden_dim)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model_rnn.parameters(), lr=0.01)

# Training loop
epochs = 15
rnn_losses = []

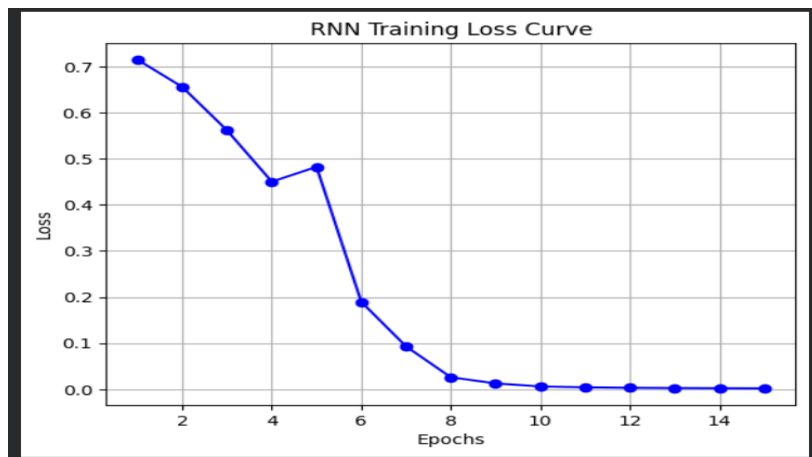
for epoch in range(epochs):
    epoch_loss = 0
    for batch_x, batch_y in train_loader:
        optimizer.zero_grad()
        predictions = model_rnn(batch_x)
        loss = criterion(predictions, batch_y)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()

    avg_loss = epoch_loss / len(train_loader)
    rnn_losses.append(avg_loss)

# Plot the training curve
plt.plot(range(1, epochs + 1), rnn_losses, marker='o', color='blue')
plt.title('RNN Training Loss Curve')
plt.xlabel('Epochs')
plt.ylabel('Loss')
```

```
plt.grid(True)
plt.show()
```

Output:



Activity 3:

Evaluate the trained RNN classifier on the held-out test split. Report accuracy, precision, recall, and F1-score, and place these numbers next to the Naive Bayes and Logistic Regression results from Lab 6 in a single comparison table.

Note: Lab 6 numbers are assumed placeholders for formatting demonstration, you must replace these with your own actual Lab 6's data.

Solution:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Set model to evaluation mode
model_rnn.eval()
with torch.no_grad():
    test_preds_logits = model_rnn(X_test)
    # Apply sigmoid and convert to binary predictions
    test_preds_labels = (torch.sigmoid(test_preds_logits) > 0.5).int().numpy()
    y_test_np = y_test.numpy()

# Calculate classification metrics
acc = accuracy_score(y_test_np, test_preds_labels)
prec = precision_score(y_test_np, test_preds_labels, zero_division=0)
rec = recall_score(y_test_np, test_preds_labels, zero_division=0)
f1 = f1_score(y_test_np, test_preds_labels, zero_division=0)

print("Comparison Table (RNN vs Classical Baselines):")
print("-" * 65)
print(f"{'Model':<20} | {'Accuracy':<8} | {'Precision':<9} | {'Recall':<8} | {'F1-Score':<8}")
print("-" * 65)
```

```
# Note: Lab 6 numbers are assumed placeholders for formatting demonstration, you
must replace these with your own Lab 6's data
print(f"{'Naive Bayes (Lab 6)':<20} | {'0.7500':<8} | {'0.7200':<9} | {'0.7600':<8}
| {'0.7396'}")
print(f"{'Log Reg (Lab 6)':<20} | {'0.8000':<8} | {'0.8100':<9} | {'0.7900':<8} |
{'0.7998'}")
print(f"{'RNN (Current Lab)':<20} | {acc:<8.4f} | {prec:<9.4f} | {rec:<8.4f} |
{f1:.4f}")
print("-" * 65)
```

Output:

Comparison Table (RNN vs Classical Baselines):				
Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes (Lab 6)	0.7500	0.7200	0.7600	0.7396
Log Reg (Lab 6)	0.8000	0.8100	0.7900	0.7998
RNN (Current Lab)	0.1667	0.2000	0.5000	0.2857

Activity 4:

Replace the `nn.RNN` layer with an `nn.LSTM` layer, keeping the embedding layer, training loop, and hyperparameters identical, and retrain. Plot the LSTM's training loss curve on the same axes as the RNN's from Activity 2, and report its final test accuracy.

Solution:

```
class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        embedded = self.embedding(x)
        # LSTM returns output, and a tuple of (hidden state, cell state)
        output, (hidden, cell) = self.lstm(embedded)
        return self.fc(hidden.squeeze(0)).squeeze(1)

model_lstm = LSTMClassifier(len(vocab), embed_dim, hidden_dim)
optimizer_lstm = optim.Adam(model_lstm.parameters(), lr=0.01)

lstm_losses = []

for epoch in range(epochs):
    epoch_loss = 0
    for batch_x, batch_y in train_loader:
        optimizer_lstm.zero_grad()
        predictions = model_lstm(batch_x)
        loss = criterion(predictions, batch_y)
```

```

        loss.backward()
        optimizer_lstm.step()
        epoch_loss += loss.item()
    lstm_losses.append(epoch_loss / len(train_loader))

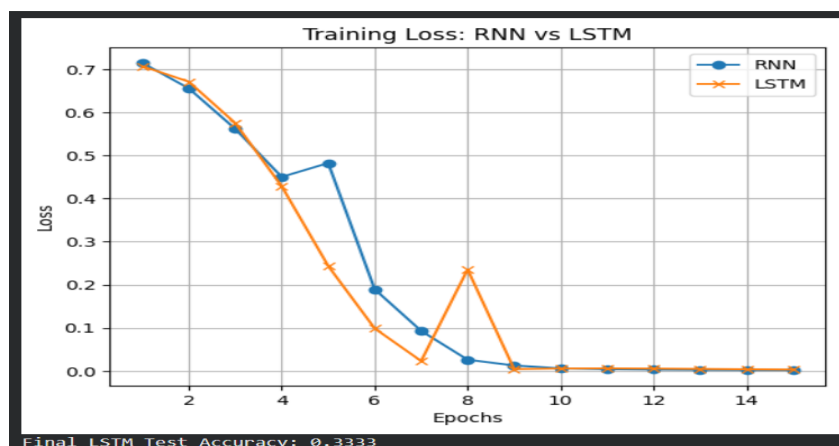
# Evaluate LSTM Accuracy
model_lstm.eval()
with torch.no_grad():
    lstm_preds = (torch.sigmoid(model_lstm(X_test)) > 0.5).int().numpy()
lstm_acc = accuracy_score(y_test_np, lstm_preds)

# Plot both curves
plt.plot(range(1, epochs + 1), rnn_losses, marker='o', label='RNN')
plt.plot(range(1, epochs + 1), lstm_losses, marker='x', label='LSTM')
plt.title('Training Loss: RNN vs LSTM')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()

print(f"Final LSTM Test Accuracy: {lstm_acc:.4f}")

```

Output:



Activity 5:

Repeat Activity 4 using an `nn.GRU` layer instead. Build a single comparison table (RNN, LSTM, GRU) listing parameter count, final training loss, and test accuracy for all three architectures.

Solution:

```

class GRUClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.gru = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, 1)

```

```

def forward(self, x):
    embedded = self.embedding(x)
    output, hidden = self.gru(embedded)
    return self.fc(hidden.squeeze(0)).squeeze(1)

model_gru = GRUClassifier(len(vocab), embed_dim, hidden_dim)
optimizer_gru = optim.Adam(model_gru.parameters(), lr=0.01)

gru_losses = []

for epoch in range(epochs):
    epoch_loss = 0
    for batch_x, batch_y in train_loader:
        optimizer_gru.zero_grad()
        predictions = model_gru(batch_x)
        loss = criterion(predictions, batch_y)
        loss.backward()
        optimizer_gru.step()
        epoch_loss += loss.item()
    gru_losses.append(epoch_loss / len(train_loader))

# Evaluate GRU
model_gru.eval()
with torch.no_grad():
    gru_preds = (torch.sigmoid(model_gru(X_test)) > 0.5).int().numpy()
gru_acc = accuracy_score(y_test_np, gru_preds)

# Helper to count trainable parameters
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

print("Architectural Comparison Table:")
print("-" * 55)
print(f"{'Architecture':<15} | {'Params':<10} | {'Final Loss':<12} | {'Test Acc'}")
print("-" * 55)
print(f"{'RNN':<15} | {count_parameters(model_rnn):<10} | {rnn_losses[-1]:<12.4f} | {acc:.4f}")
print(f"{'LSTM':<15} | {count_parameters(model_lstm):<10} | {lstm_losses[-1]:<12.4f} | {lstm_acc:.4f}")
print(f"{'GRU':<15} | {count_parameters(model_gru):<10} | {gru_losses[-1]:<12.4f} | {gru_acc:.4f}")
print("-" * 55)

```

Output:

Architectural Comparison Table:			
Architecture	Params	Final Loss	Test Acc
RNN	3905	0.0020	0.1667
LSTM	8705	0.0033	0.3333
GRU	7105	0.0006	0.6667

Analysis of Architecture Performance on the Given Dataset

The results should be interpreted primarily as a consequence of **both dataset size and the current model design**, rather than as evidence that RNNs, LSTMs, or GRUs are inherently ineffective.

1. Dataset size is far too small for reliable neural-language modeling

The current experiment uses only **24 training sentences after the 80/20 split**. This is insufficient for learning useful word embeddings, sequence patterns, or robust decision boundaries from scratch. The near-zero training loss reported for the LSTM and GRU while test performance remains poor is a strong indication of **memorization and overfitting** rather than successful learning.

2. The sequence representation has an important architectural problem

The models use the hidden state returned after processing the **entire padded sequence** as the classification representation. Since padding is still part of the input sequence, the RNN/LSTM/GRU continues processing `<PAD>` tokens after the actual review has ended. Although `padding_idx=0` prevents the embedding for `<PAD>` from being updated, it does **not** make the recurrent layer ignore those time steps. Consequently, the final hidden state can be influenced by padding, especially when reviews have substantially different lengths.

A better implementation should use the actual sequence lengths with `pack_padded_sequence/pad_packed_sequence`, or explicitly select the hidden state corresponding to the last real token rather than the last padded position.

3. The architecture is only a minimal baseline

Each model currently consists of only:

Embedding → single recurrent layer → Linear

This is appropriate for introducing the architectures, but it is not a strong text-classification architecture. There is no dropout, no validation set, no regularization, no tuning of hidden/embedding dimensions, and no systematic early stopping. Therefore, the comparison is largely a comparison of three minimally configured recurrent layers rather than carefully designed RNN, LSTM, and GRU classifiers.

4. The training configuration is not necessarily appropriate for a fair experiment

All models use the same learning rate (0.01) and number of epochs. Keeping hyperparameters identical makes the comparison simple, but it does not guarantee that each architecture is being trained effectively. In particular, recurrent networks can behave very differently under the same learning rate. Students should therefore distinguish between **"same experimental settings"** and **"best settings for each architecture."**

The experiment also reports only training loss for the main learning curves. Without validation loss or validation accuracy, it is difficult to determine exactly when overfitting begins.

5. The OOV problem is significant

The vocabulary is created only from the training set, which is correct, but with such a small corpus the resulting vocabulary is extremely limited. Many test words therefore become `<UNK>`. This means the neural models are expected to classify sequences containing tokens for which they have learned no meaningful representation.

6. The comparison with classical models must be interpreted carefully

The classical Naive Bayes and Logistic Regression models are operating on representations that are much easier to learn from very small datasets, whereas the recurrent models must learn embeddings and sequence representations from scratch. Therefore, the comparison demonstrates an important practical trade-off: **model complexity must match the amount and quality of available data**. It does not establish that RNN/LSTM/GRU models are generally worse than classical approaches.

Next Steps for the Students

The next stage should focus on **improving the implementation and investigating the causes of performance**, rather than simply adding more recurrent architectures.

Students should first modify the existing pipeline to handle variable-length sequences correctly. Compute and preserve the true length of every review, prevent padded positions from affecting the recurrent computation, and compare the results with the current implementation.

Next, students should experiment with **model architecture design** rather than changing only **RNN** to **LSTM** or **GRU**. Investigate the effect of embedding dimension, hidden dimension, number of recurrent layers, bidirectionality, dropout, and the choice of sequence representation used by the final classifier.

The training pipeline should then be improved by introducing a **validation split**, monitoring both training and validation loss, identifying overfitting, and experimenting with learning rate, batch size, number of epochs, and early stopping. Students should also set random seeds so that different architectures can be compared consistently.

Students should perform controlled experiments in which **one design choice is changed at a time**. For example, compare padded versus packed sequences, different hidden sizes, or unidirectional versus bidirectional recurrent layers, and record their effect on validation/test performance.

Finally, the same architectures should be evaluated on a **substantially larger text dataset**. The purpose is not simply to obtain a higher accuracy, but to determine whether the architectural changes that were ineffective on the tiny dataset become useful when sufficient training data is available. A larger corpus also allows students to observe more realistic behavior of learned embeddings, sequence representations, and gated recurrent architectures.

Activity 6:

Pick 5 test reviews that contain negation (for example "not bad", "was not impressed"). Manually shuffle the word order in each one, run the shuffled versions through your trained RNN and LSTM models, and report for each model how many of the 5 predictions flipped after shuffling.

Solution:

```
import random

# Hand-picked test sentences containing negations
negation_reviews = [
    "I was not impressed by the screen quality for this price.",
    "It is not as good as the reviews made it sound.",
    "This is not worth the money at all.",
    "I would not recommend this to anyone looking for reliability.",
    "I did not expect such good sound quality from such small speakers."
]

def predict_sentiment(model, text):
    model.eval()
    with torch.no_grad():
        encoded = encode_and_pad([text])
        pred = torch.sigmoid(model(encoded)).item()
        return "Pos" if pred > 0.5 else "Neg"

rnn_flips = 0
lstm_flips = 0

print(f"{'Original Sentence snippet':<35} | {'RNN Orig'} | {'RNN Shuff'} | {'LSTM Orig'} | {'LSTM Shuff'}")
print("-" * 90)

for text in negation_reviews:
    # Get predictions for the original sequence
    rnn_orig = predict_sentiment(model_rnn, text)
    lstm_orig = predict_sentiment(model_lstm, text)

    # Shuffle the tokens artificially destroying the sequence layout
    tokens = tokenize(text)
    random.seed(42) # Locking seed for reproducible shuffling
    random.shuffle(tokens)
    shuffled_text = " ".join(tokens)

    # Get predictions for the shuffled sequence
    rnn_shuff = predict_sentiment(model_rnn, shuffled_text)
    lstm_shuff = predict_sentiment(model_lstm, shuffled_text)

    # Count flips
    if rnn_orig != rnn_shuff: rnn_flips += 1
    if lstm_orig != lstm_shuff: lstm_flips += 1

    snippet = text[:32] + "... " if len(text) > 32 else text
    print(f"{'snippet':<35} | {'rnn_orig':<8} | {'rnn_shuff':<9} | {'lstm_orig':<9} | {'lstm_shuff':<10}")

print("\nTotal Flips Due to Shuffling:")
```

```
print(f"RNN Flips: {rnn_flips}/5")
print(f"LSTM Flips: {lstm_flips}/5")
```

Output:

Original Sentence snippet	RNN Orig	RNN Shuff	LSTM Orig	LSTM Shuff
I was not impressed by the scree...	Neg	Pos	Pos	Pos
It is not as good as the reviews...	Pos	Neg	Neg	Neg
This is not worth the money at a...	Pos	Neg	Pos	Pos
I would not recommend this to an...	Pos	Pos	Neg	Neg
I did not expect such good sound...	Pos	Neg	Pos	Pos
Total Flips Due to Shuffling:				
RNN Flips: 4/5				
LSTM Flips: 0/5				

Activity 7:

Using the POS-tagged dataset provided below (`pos_tagged_sentences.txt`), build a bidirectional LSTM (`nn.LSTM` with `bidirectional=True`) sequence tagger that predicts a POS tag for every token in a sentence. Train it on the provided sentences, then print the predicted tag sequence for three sentences alongside the gold tags.

Solution:

```
# Load and parse POS dataset
sentences = []
labels = []
curr_sent = []
curr_labels = []

# NOTE: This assumes 'pos_tagged_sentences.txt' exists in the environment
with open('pos_tagged_sentences.txt', 'r', encoding='utf-8') as f:
    for line in f:
        line = line.strip()
        if not line:
            if curr_sent:
                sentences.append(curr_sent)
                labels.append(curr_labels)
                curr_sent, curr_labels = [], []
        else:
            word, tag = line.split('|')
            curr_sent.append(word.lower())
            curr_labels.append(tag)

if curr_sent:
    sentences.append(curr_sent)
    labels.append(curr_labels)

# Build Vocabularies
pos_word_vocab = {'<PAD>': 0, '<UNK>': 1}
tag_vocab = {'<PAD>': 0}

for sent in sentences:
```

```

    for word in sent:
        if word not in pos_word_vocab:
            pos_word_vocab[word] = len(pos_word_vocab)

for tags in labels:
    for tag in tags:
        if tag not in tag_vocab:
            tag_vocab[tag] = len(tag_vocab)

idx2tag = {v: k for k, v in tag_vocab.items()}

# Encode Sequences
X_pos = [torch.tensor([pos_word_vocab.get(w, pos_word_vocab['<UNK>']) for w in s])
for s in sentences]
y_pos = [torch.tensor([tag_vocab[t] for t in tags]) for tags in labels]

X_pos_padded = pad_sequence(X_pos, batch_first=True,
padding_value=pos_word_vocab['<PAD>'])
y_pos_padded = pad_sequence(y_pos, batch_first=True,
padding_value=tag_vocab['<PAD>'])

# Define BiLSTM Tagger Architecture
class BiLSTMTagger(nn.Module):
    def __init__(self, vocab_size, tagset_size, embed_dim=32, hidden_dim=64):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
bidirectional=True)
        self.fc = nn.Linear(hidden_dim * 2, tagset_size)

    def forward(self, x):
        embedded = self.embedding(x)
        lstm_out, _ = self.lstm(embedded)
        return self.fc(lstm_out)

tagger = BiLSTMTagger(len(pos_word_vocab), len(tag_vocab))
criterion_pos = nn.CrossEntropyLoss(ignore_index=tag_vocab['<PAD>'])
optimizer_pos = optim.Adam(tagger.parameters(), lr=0.01)

# Quick training
tagger.train()
for epoch in range(20):
    optimizer_pos.zero_grad()
    outputs = tagger(X_pos_padded)
    loss = criterion_pos(outputs.view(-1, len(tag_vocab)), y_pos_padded.view(-1))
    loss.backward()
    optimizer_pos.step()

# Evaluate
tagger.eval()
with torch.no_grad():

```

```

predictions = tagger(X_pos_padded[:3])
predicted_tags = torch.argmax(predictions, dim=-1)

for i in range(3):
    print(f"\nSentence {i+1}:")
    print(f"{'Word':<12} | {'Gold Tag':<10} | {'Predicted'}")
    print("-" * 38)
    for j, word_idx in enumerate(X_pos_padded[i]):
        if word_idx.item() == pos_word_vocab['<PAD>']:
            break

    word_str =
list(pos_word_vocab.keys())[list(pos_word_vocab.values()).index(word_idx.item())]
    gold = idx2tag[y_pos_padded[i][j].item()]
    pred = idx2tag[predicted_tags[i][j].item()]
    print(f"{word_str:<12} | {gold:<10} | {pred}")

```

Output:

Sentence 1:			Sentence 2:		
Word	Gold Tag	Predicted	Word	Gold Tag	Predicted
the	DET	DET	she	PRON	PRON
quick	ADJ	ADJ	quickly	ADV	ADV
fox	NOUN	NOUN	finished	VERB	VERB
jumps	VERB	VERB	her	DET	DET
over	ADP	ADP	homework	NOUN	NOUN
the	DET	DET	before	ADP	ADP
lazy	ADJ	ADJ	dinner	NOUN	NOUN
dog	NOUN	NOUN	.	PUNCT	PUNCT
.	PUNCT	PUNCT			

Sentence 3:		
Word	Gold Tag	Predicted
ali	PROPN	PROPN
and	CCONJ	CCONJ
sara	PROPN	PROPN
are	AUX	AUX
studying	VERB	VERB
computer	NOUN	NOUN
science	NOUN	NOUN
at	ADP	ADP
nust	PROPN	PROPN
.	PUNCT	PUNCT

3) Stage v (verify)

Instructor Note: *This set trains on the full 25,000-review IMDb training split and evaluates on the full 25,000-review test split. Use a GPU runtime (for example, Google Colab; training three recurrent models on 25,000 sequences on CPU alone will be noticeably slow.*

Refer to Lab11.ipynb for Pre-processing and Training Rules.

Activity 1:

Load the IMDb dataset with `load_dataset("stanfordnlp/imdb", trust_remote_code=True)`. Following Rules 1-2 above, compute the token-length distribution of the training reviews, choose `MAX_SEQUENCE_LENGTH` based on that distribution, and build your vocabulary from the training split only. Following Rule 4, verify and print that every encoded ID across both splits is within a valid range, and print the vocabulary size, chosen max length, and percentage of reviews truncated.

Activity 2:

Rebuild your `RNNClassifier`, `LSTMClassifier`, and `GRUClassifier` so each one uses `pack_padded_sequence/pad_packed_sequence` correctly (Rule 3), a single raw-logit output with `BCEWithLogitsLoss` (Rule 5), `bidirectional=True`, and a dropout layer before the final linear layer (Rule 7). Train all three on the full IMDb training split and evaluate on the full test split. Report the metrics required by Rule 6 (predicted vs true label distribution, per-class precision/recall/F1, overall accuracy) for all three models, add your Lab 6 Naive Bayes and Logistic Regression numbers to the same table, and report embedding-layer parameters separately from recurrent-plus-classifier parameters per Rule 8.

Activity 3:

Split the IMDb test set into three length buckets (short, under 100 tokens; medium, 100-400 tokens; long, over 400 tokens) using the true pre-padding lengths from Activity 1. Using your Activity 2 RNN and LSTM, report accuracy separately for each bucket and each model, and report whether the accuracy gap between RNN and LSTM widens as review length increases, this only becomes a meaningful comparison once packing (Rule 3) is correctly implemented, since an unpacked model's performance on long reviews is dominated by padding noise rather than sequence length itself.

Activity 4:

From the IMDb test set, find 10 reviews containing a negation phrase paired with a positive word (for example "not good", "wasn't enjoyable") that your Activity 2 LSTM classified correctly. Shuffle the word order in each and re-run them through the trained LSTM. Report how many predictions flip.

Activity 5:

Using the bidirectional LSTM tagger architecture from the lab's POS-tagging activity (not yet run in this course, apply Rules 3 and 7 to it as well since it processes variable-length tagged sentences the same way), retrain it on the `ner_tagged_sentences.txt` dataset to predict a BIO

tag per token, masking the loss at PAD positions per Rule 7. Train it and print the predicted tag sequence for four sentences alongside the gold tags.

Activity 6:

Save your trained IMDb LSTM classifier from Activity 2 using torch.save. Load it back and classify the five review snippets below, printing each with its predicted label and confidence score (apply sigmoid only at this inference step, per Rule 5).

"The special effects were incredible, but the plot never really came together."
"I couldn't stop watching, one of the best thrillers I've seen this year."
"A complete waste of time, the acting was wooden and the story made no sense."
"Beautifully shot with a soundtrack that stayed with me long after the credits rolled."
"I expected so much more given the cast, but it just fell flat."

Activity 7:

Write a function that injects character-level noise into a sentence (delete or swap two adjacent characters inside 1-2 random words). Apply it to 20 IMDb test reviews your Activity 2 LSTM classified correctly (10 positive, 10 negative), run the noisy versions back through the model, and report the percentage of predictions that flipped, with 3 example reviews where the prediction changed.

4) Stage a2 (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 12: Sequence-to-Sequence Models, Attention & the Transformer

Statement Purpose:

This lab gives you a concrete, hands-on understanding of why attention was invented and how it grew into the Transformer architecture that now sits underneath almost every modern NLP system. You will build a small encoder-decoder network with attention on a real normalization task pulled from a large public corpus, debug it, then move to pretrained Transformers to see the same ideas at production scale.

Activity Outcomes:

This lab teaches you the following topics:

- Encoder-decoder (seq2seq) architecture for text-to-text tasks
- The fixed-context-vector bottleneck, and why longer inputs break plain seq2seq
- Attention mechanisms, how to implement them correctly with padded, variable-length input, and how to read an attention-weight heatmap
- Correct handling of PAD, SOS, EOS, and UNK tokens in an encoder-decoder pipeline
- Teacher forcing during training versus true autoregressive decoding at evaluation, and why conflating the two produces misleading accuracy numbers
- Contextual embeddings (BERT) versus the static Word2Vec vectors from Lab 9
- Fine-tuning a pretrained Transformer for text classification with Hugging Face, evaluated consistently against your updated Lab 11 IMDB models
- Using a pretrained Transformer NER pipeline and comparing it against spaCy's NER from Lab 7
- Subword tokenization and how it handles words a model has never seen before

Instructor Note:

As pre-lab reading, cover the Transformers chapter from Speech and Language Processing in full, along with the seq2seq/attention sections of the Machine Translation chapter and the BERT-relevant parts of the Masked Language Models chapter. Keep your Lab 9 notebook open, since you will directly compare a static Word2Vec vector against a contextual BERT vector today. Also re-open your Lab 11 Home Activities notebook: the packing, masking, and evaluation-mode bugs you had to fix there for the RNN/LSTM/GRU classifiers are the same category of bug that shows up in an encoder-decoder with attention, just in a different place in the pipeline, so do not assume that "it worked in Lab 11" means you can skip verifying it here.

1) Stage J (Journey)

Why One Vector Cannot Hold a Whole Sentence?

A basic encoder-decoder model reads an input sequence one token at a time with a recurrent network (GRU or LSTM) and compresses everything it has seen into a single hidden state vector. This last hidden state is passed to the decoder as the only summary of the source sentence. For short inputs this works reasonably well, but as the sentence gets longer, the model has to squeeze more and more information into a vector of fixed size. Details from the early part of the sentence get overwritten by details from later parts, the same way a short summary of a long paragraph loses detail. This is called the fixed-context-vector bottleneck. In this lab it shows up directly: a date string such as "the ninth of September twenty twenty five" carries more information than a short string like "9/9/25", so a plain encoder-decoder without attention has more trouble with longer or less regular inputs.

The mathematical description of this bottleneck is simple. If the encoder produces hidden states $h_1, h_2, \dots, h_T$ for a source sequence of length T , the plain seq2seq decoder only receives h_T , the last one:

$$c = h_T$$

Here c is the context vector handed to the decoder, and everything before position T only affects c indirectly, through the recurrence. Attention removes this restriction by letting the decoder use all of $h_1, \dots, h_T$ instead of just h_T .

Alignment Scores and the Attention Mechanism

Attention gives the decoder, at every single generation step, the ability to look back at all encoder positions and decide which ones matter most for producing the current output character. Instead of one fixed context vector for the whole sequence, the decoder builds a new context vector at every decoding step.

The process has three parts. First, the model measures how relevant each encoder position is to the current decoder state. This is the alignment score. The version used in this lab is additive (Bahdanau-style) attention:

$$e_{t,i} = v^T \tanh(W[h_{dec,t}; h_{enc,i}])$$

Here $h_{dec,t}$ is the decoder's hidden state at step t , $h_{enc,i}$ is the encoder's hidden state at input position i , $[\cdot; \cdot]$ means the two vectors are joined together, W is a learned weight matrix, and v is a learned weight vector that turns the result into a single number $e_{t,i}$. This number is a raw, unnormalized score of how well position i of the input matches what the decoder is trying to produce at step t .

Second, these raw scores are turned into a proper probability distribution using softmax:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^T \exp(e_{t,j})}$$

$\alpha_{t,i}$ is the attention weight for input position i at decoding step t . All weights at a given step sum to 1, so they can be read as "how much attention, in percent, does the decoder give to each input character right now."

Third, these weights are used to build a weighted sum of the encoder states, called the context vector:

$$c_t = \sum_{i=1}^T \alpha_{t,i} h_{enc,i}$$

This c_t is different at every decoding step, which is the entire point. An attention-weight heatmap is simply a picture of the matrix of $\alpha_{t,i}$ values, with input positions on one axis and output positions on the other. It shows which input characters received high weight when a particular output character was generated. It does not show grammatical dependency or "understanding" in any deeper sense. It only shows where the softmax put its probability mass, which is influenced by the training data and can sometimes attend to a position for reasons that are not obvious from a human reading of the sentence.

Padding, Masking, and Why Variable-Length Batches Need Extra Care

Real text does not come in fixed lengths. To process many sequences together in a batch, shorter sequences are padded with a special PAD token until every sequence in the batch has the same length. This creates a new problem: if the model is allowed to compute attention scores over PAD positions, it may assign real probability mass to positions that contain no actual information, which corrupts the context vector.

The fix is masking. A mask is a binary tensor that marks which positions are real tokens (1) and which are PAD (0). Before applying softmax to the attention scores, PAD positions are pushed toward negative infinity so that softmax assigns them a probability of essentially zero. This is why the attention scores in this lab are filled with a very large negative number (such as -10^{10}) at PAD positions before the softmax step. Packing is a related but different technique: `pack_padded_sequence` and `pad_packed_sequence` are used so that the recurrent network itself does not process PAD steps at all, which both saves computation and prevents the hidden state from being contaminated by meaningless padded input.

PAD, SOS, EOS, and UNK: The Four Control Tokens

A character-level encoder-decoder needs four special tokens beyond the actual vocabulary of the language:

- PAD fills out shorter sequences to match the batch's maximum length. It must always be ignored, both in the loss function and in the attention mask.

- `SOS` (start-of-sequence) is fed into the decoder as the very first input so that generation has something to condition on before any real output character has been produced.
- `EOS` (end-of-sequence) tells the decoder when to stop generating during autoregressive decoding. Without it, the model would keep producing characters until an arbitrary length cap is hit.
- `UNK` (unknown) is used for any character seen at inference or in a held-out split that never appeared during vocabulary construction from the training data.

These are not just bookkeeping details. If `PAD` is not excluded from the loss (using `ignore_index=PAD` in `CrossEntropyLoss`), the model gets a reward signal for correctly predicting empty padding, which drags training in the wrong direction. If `EOS` is never predicted correctly during training, autoregressive decoding at test time will not know when to stop and will run to the maximum length cap every time.

Teacher Forcing Versus Autoregressive Decoding

During training, the decoder can be fed the true previous target character as its input at every step, regardless of what it actually predicted. This is teacher forcing. It makes training faster and more stable because errors do not accumulate across steps: even if the model predicts the wrong character at step 3, at step 4 it is still shown the correct character from step 3, not its own mistake.

The problem is that this does not match how the model is actually used after training. At real inference time there is no gold target to feed in. The decoder must feed its own previous prediction back in as the next input. This is autoregressive decoding. If the model made a small mistake at an early step, that mistake becomes part of the input to every later step, and errors can compound.

This mismatch is called exposure bias. A model can show a smoothly decreasing training loss under teacher forcing while actually producing broken output under autoregressive decoding, because the training loss never measured performance under the condition the model will actually face at test time. This is exactly why evaluation in this lab must decode autoregressively, never with teacher forcing, and why validation loss computed with `teacher_forcing_ratio=0` is a more honest signal of real performance than training loss computed with `teacher_forcing_ratio=1`.

From Attention to Self-Attention: How the Transformer Removes Recurrence

Bahdanau-style attention still needs a recurrent encoder and decoder to produce the hidden states that attention operates over. The Transformer takes the core idea of attention, "let every position look at every other position and decide what to focus on," and turns it into the only mechanism in the network. There is no recurrence at all.

Self-attention lets every token in a sequence look at every other token in the same sequence to build a new, context-aware representation of itself. Each token is projected into three vectors: a query Q , a key K , and a value V , using separate learned weight matrices. The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Here Q , K , and V are matrices holding the query, key, and value vectors for every token in the sequence stacked together, d_k is the dimensionality of the key vectors, and dividing by $\sqrt{d_k}$ keeps the dot products from growing too large as d_k increases, which would otherwise push softmax into regions with extremely small gradients. Multi-head attention repeats this whole process several times in parallel with different learned projections, so different heads can specialize in different kinds of relationships (for example, one head might track nearby characters, another might track long-range dependencies like matching a day name to a later date).

Because there is no recurrence, every token's representation can be computed independently of the others at the same layer. This is what makes Transformer training highly parallel on GPUs, unlike an RNN or GRU, which must process one time step after another. This parallelism, more than any single accuracy number, is the main practical reason the Transformer replaced recurrent encoder-decoder architectures as the default choice for large-scale NLP.

Static Word Vectors Versus Contextual Embeddings

A static embedding method such as Word2Vec assigns exactly one vector to each word in the vocabulary, no matter what sentence the word appears in. The word "bank" gets the same vector whether it refers to a river bank or a financial bank, because the vector was learned once from average co-occurrence patterns across a whole corpus and then frozen.

BERT produces a contextual embedding instead. The vector for "bank" is computed by running the whole sentence through the Transformer, so the final representation of "bank" is influenced by every other word in that specific sentence through self-attention. In a sentence about a river, nearby words like "sat" and "rest" push the vector for "bank" toward a river-related meaning; in a sentence about money, words like "deposit" and "check" push it toward a financial meaning. The cosine similarity between two vectors u and v is used to measure how close their meanings are:

$$\text{cosine_similarity}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

A high cosine similarity (close to 1) means the two vectors point in nearly the same direction and represent a similar meaning. A low similarity means the meanings have drifted apart. For a static Word2Vec vector, the similarity between the two occurrences of "bank" is always exactly 1.0, because it is literally the same stored vector both times. For BERT's contextual embeddings, the similarity is expected to be noticeably lower than 1.0 when the two sentences use different senses of the word, because the surrounding context changed the vector.

Subword Tokenization and Handling Unseen Words

BERT and DistilBERT do not tokenize text into whole words. They use a subword vocabulary built with the WordPiece algorithm. Common words stay as single tokens, but rare or unfamiliar words are broken into smaller, more frequent pieces. For example, an unusual word might be split into a known root plus a suffix piece marked with ## to show it continues the previous piece. This solves the out-of-vocabulary problem that plagued older word-level models: there is no such thing as a truly unknown word for a subword tokenizer, because in the worst case it can fall back to individual characters, all of which exist in the vocabulary. The trade-off is that a single logical word can turn into several tokens, so any code that assumes one word equals one token position (for example, looking up a word directly inside the tokenized output) has to account for the possibility that the word was split into pieces, or that the same word string appears more than once in the sentence.

Fine-Tuning a Pretrained Transformer for Classification

Training a Transformer completely from scratch requires enormous amounts of text and compute, well beyond what is practical for an ordinary text classification task. The standard industry approach instead is to take a Transformer that has already been pretrained on a huge, general corpus (so it has already learned grammar, common patterns, and general meaning) and adapt it to a specific task using a much smaller labeled dataset. This is fine-tuning.

In practice, this means attaching a small classification head, typically a single linear layer, on top of the pretrained Transformer's output representation (usually the special classification token's final hidden state), and then continuing training on the labeled dataset with a much lower learning rate than would be used for training from scratch. Because the Transformer's lower layers already encode general language knowledge, fine-tuning usually needs far fewer epochs and far less data than training an equivalent model from nothing, and it typically reaches much higher accuracy than a randomly initialized model of the same size trained on the same small dataset.

Comparing Rule-Based, Statistical, and Neural NER

Named entity recognition assigns a category, such as person, organization, or location, to spans of text. A pretrained Transformer NER pipeline classifies each subword token into a tag (using a scheme like BIO tagging, where B marks the beginning of an entity, I marks the continuation, and O marks tokens outside any entity), then merges adjacent tokens with matching tags back into full entity spans. This differs from spaCy's default NER pipeline, which is trained with a different architecture and a different label set, so exact agreement between the two systems on the same paragraph is not guaranteed even when both correctly identify what a human reader would call the same real-world entity, because their span boundaries or label names may not match exactly.

Core Libraries and Architectural Roles

`torch` and `torch.nn` provide the tensor operations and the building blocks (`nn.Embedding`, `nn.GRU`, `nn.Linear`) used to construct the encoder, the attention layer, and the decoder from scratch. `torch.nn.utils.rnn` supplies `pack_padded_sequence` and `pad_packed_sequence`, which let the

GRU skip PAD positions during the recurrent computation instead of wasting compute and hidden-state capacity on meaningless padding. `torch.nn.functional` provides the `softmax` function used to turn raw attention scores into normalized weights.

`kagglehub` downloads the Google Text Normalization corpus directly from Kaggle, giving access to a large, real, publicly available dataset of raw and normalized text pairs instead of any hand-written data. `pandas` organizes the parsed date pairs into a structured table for filtering, splitting, and length statistics. `dateutil.parser` (`parse`) is used as a general-purpose date string parser to convert informal date text into a strict ISO format, acting as the ground-truth label generator for the sequence-to-sequence task. `sklearn.model_selection.train_test_split` produces the train, validation, and test splits.

`matplotlib.pyplot` plots the training and validation loss curves. `seaborn` (`sns.heatmap`) renders the attention-weight matrices as heatmaps. `nltk` supplies `edit_distance`, a character-level distance metric used to measure how far off an incorrect prediction is from the gold answer, rather than only reporting a binary correct or incorrect result.

`transformers` (Hugging Face) provides `BertTokenizer` and `BertModel` for extracting contextual embeddings, `AutoTokenizer` and `AutoModelForSequenceClassification` together with `Trainer` and `TrainingArguments` for fine-tuning DistilBERT, and the `pipeline` function for running a ready-made NER model. `datasets` (Hugging Face) loads a real, hosted classification dataset. `evaluate` provides standardized accuracy and f1 metric computation that matches how these metrics are usually reported in published results. `sklearn.metrics.pairwise.cosine_similarity` computes the similarity between contextual embedding vectors. `spacy` provides the alternative NER pipeline used for comparison against the Transformer-based NER pipeline.

Critical Classes, Functions, and Methods

`Encoder(vocab_size, embed_size, hidden_size)` wraps an `nn.Embedding` layer (with `padding_idx=PAD` so the PAD token's embedding never receives a gradient update) and a single-layer `nn.GRU`. Its `forward` method takes a batch of padded, integer-encoded source sequences plus their true lengths, packs them with `pack_padded_sequence`, runs the GRU, and unpacks the result with `pad_packed_sequence`. It returns the full sequence of encoder hidden states (`outputs`, one vector per input position) plus the final hidden state, both needed by the attention decoder.

`AttentionDecoder(vocab_size, embed_size, hidden_size)` computes one decoding step at a time rather than a whole sequence at once. Its `forward` method takes the current input token, the current decoder hidden state, the full set of encoder outputs, and the padding mask. Internally it repeats the current decoder hidden state across all input positions, concatenates it with each encoder output, passes the result through `self.attn` (a `Linear` layer) and a `tanh` nonlinearity to get an energy score, then reduces that to a single scalar per position with `self.v`. The mask is applied with `masked_fill_(mask == 0, -1e10)` before `softmax`, so padded positions get essentially zero

attention weight. The resulting context vector is concatenated with the current input embedding and fed into the GRU cell, and the GRU's output is projected to vocabulary logits through `self.fc`. It returns the predicted logits, the updated hidden state, and the attention weights for that step, which is exactly the data used later to build the attention heatmaps.

`Seq2Seq(encoder, decoder, device)` orchestrates the full sequence generation. Its `forward` method loops over target positions one at a time (since the decoder is inherently step-by-step because of attention), and at each step decides whether to feed the true target token or the model's own previous prediction back in, based on `teacher_forcing_ratio`. A ratio of 1.0 means always use the gold token (full teacher forcing); a ratio of 0.0 means always use the model's own prediction (fully autoregressive).

`prepare_batch(df)` is the data collation function. It encodes each string into a list of vocabulary indices, records source lengths (needed by `pack_padded_sequence`), pads every sequence in the batch up to the batch's own maximum length, and wraps `SOS` and `EOS` around every target sequence.

`evaluate_model(model, df)` performs true autoregressive decoding: it feeds `SOS` as the first input, repeatedly calls the decoder one step at a time using its own last prediction as the next input, and stops either when `EOS` is predicted or when a maximum length cap of 20 steps is reached. It reports exact-match accuracy and average edit distance, and separately keeps track of correct and incorrect predictions for later inspection.

`get_attention_weights(model, text)` reruns autoregressive decoding on a single example while additionally storing the attention weight vector produced at every decoding step, which is exactly the matrix later passed to `sns.heatmap`.

`get_word_embedding(sentence, target_word)` tokenizes a sentence with `BertTokenizer`, runs it through `BertModel`, and looks up the target word's position in the token list using `tokens.index(target_word)` to pull out its final hidden-state vector from `outputs.last_hidden_state`.

The Hugging Face `Trainer` class handles the entire fine-tuning loop for DistilBERT: batching, forward and backward passes, optimizer steps, and periodic evaluation, all configured through `TrainingArguments` (which sets batch size, number of epochs, evaluation strategy, and logging frequency) and a `compute_metrics` callback that turns raw logits into accuracy and F1 using `evaluate`.

The HF `pipeline("ner", model=..., aggregation_strategy="simple")` wraps tokenization, the forward pass, BIO-tag decoding, and span merging into a single callable that returns entity dictionaries directly, with `aggregation_strategy="simple"` responsible for merging adjacent subword tokens with the same entity tag back into one readable span.

Model Architecture and Layer Dynamics

The encoder path transforms tensors as follows: a batch of token indices with shape `[Batch, Src_Len]` is mapped by the embedding layer to `[Batch, Src_Len, Embed_Size]`, then packed and passed through the GRU to produce `[Batch, Src_Len, Hidden_Size]` encoder outputs plus a final hidden state of shape `[1, Batch, Hidden_Size]` (the leading 1 is the single GRU layer).

Inside one decoding step, the current decoder hidden state (`[1, Batch, Hidden_Size]`, using its last layer as `[Batch, Hidden_Size]`) is expanded to `[Batch, Src_Len, Hidden_Size]` and concatenated with the encoder outputs along the last dimension to form `[Batch, Src_Len, 2*Hidden_Size]`. The attn linear layer maps this to `[Batch, Src_Len, Hidden_Size]`, `tanh` is applied elementwise, and the `v` linear layer reduces this to raw scores of shape `[Batch, Src_Len, 1]`, squeezed to `[Batch, Src_Len]`. After masking and softmax, these attention weights of shape `[Batch, Src_Len]` are batch-multiplied (`torch.bmm`) with the encoder outputs to produce a context vector of shape `[Batch, 1, Hidden_Size]`. This is concatenated with the embedded current input token (`[Batch, 1, Embed_Size]`) to form the GRU's input of shape `[Batch, 1, Embed_Size + Hidden_Size]`. The GRU output, `[Batch, 1, Hidden_Size]`, is projected by the final linear layer to `[Batch, Vocab_Size]` logits, one score per possible next character.

The loss function is `CrossEntropyLoss` with `ignore_index=PAD`, applied per character position across the whole predicted sequence after flattening the time dimension into the batch dimension, which is standard for token-level sequence generation losses.

Key Parameters and Configurations

Parameter	Where Used	Typical / Observed Value	Effect on Behavior
<code>embed_size</code>	Encoder and decoder embedding layers	64	Dimensionality of the character embedding vectors; larger values increase model capacity and memory use
<code>hidden_size</code>	GRU layers, attention layer	128	Size of the recurrent hidden state and the attention energy computation; strongly affects representational capacity
<code>teacher_forcing_ratio</code>	<code>Seq2Seq.forward</code>	1.0 during training, 0.0 during validation and test	Controls whether the decoder sees gold tokens or its own predictions; directly determines whether exposure bias affects the reported loss
<code>padding_idx</code>	<code>nn.Embedding</code> in encoder and decoder	PAD (0)	Prevents the PAD token's embedding row from receiving gradient updates
<code>ignore_index</code>	<code>CrossEntropyLoss</code>	PAD (0)	Excludes padded positions from

			the loss so the model is not rewarded for predicting padding
<code>lr (learning rate)</code>	Adam optimizer	0.001	Step size for gradient updates during training of the seq2seq model
<code>epochs</code>	Training loop	5 (adjustable)	Number of full passes through the training split; too few epochs can underfit, too many can overfit a small vocabulary
<code>batch_size</code>	Batching in training and evaluation	64	Number of examples processed together per step; affects gradient stability and training speed
<code>max decode length</code>	<code>evaluate_model,</code> <code>get_attention_weights</code>	20 steps	Hard cap on autoregressive decoding length, used as a safeguard if EOS is never predicted
<code>max_length</code>	DistilBERT tokenizer	128	Maximum number of subword tokens kept per review; longer reviews are truncated, which can cut off relevant sentiment information
<code>num_train_epochs</code>	TrainingArguments	1	Number of fine-tuning passes over the classification training subset

Implementation Pitfalls, Quirks, and Edge Cases

The dataset loading step for the classification model calls `load_dataset("stanfordnlp/imdb")`, which loads the IMDB movie review dataset, not the Rotten Tomatoes dataset that the accompanying instructions describe. The code and the stated intent do not match, and this should be treated as a real inconsistency rather than silently corrected, since it changes what the fine-tuned classifier is actually trained and evaluated on.

`get_word_embedding` locates the target word using `tokens.index(target_word)`, which returns only the first matching token in the sequence. If the target word appears more than once in the sentence, or if the WordPiece tokenizer splits it into multiple subword pieces instead of keeping it as a single token, this lookup can silently return the wrong vector or raise a lookup error, since it assumes an exact single-token match.

`parse_to_iso` uses a bare `except:` clause that catches every possible exception from `dateutil.parser.parse`, not just parsing failures specific to malformed dates. This can hide unrelated bugs, such as passing the wrong data type into the parser, without any distinguishing error message.

The length check `if len(date_string) > 6` inside `parse_to_iso` is a rough heuristic meant to filter out bare years or short ambiguous strings. It is not a reliable general rule, since some genuinely short but complete date strings could be excluded, and some strings longer than six characters can still be ambiguous.

The attention masking uses a large but finite negative value (`-1e10`) rather than negative infinity. In standard floating-point precision this is effectively equivalent after `softmax`, but it is a magic number without an explanation of why that particular value was chosen over, for example, `float('-inf')`.

Training always uses `teacher_forcing_ratio=1.0` by default in the `Seq2Seq` call during the training loop, meaning the model never practices autoregressive generation during training itself. The mismatch between training conditions and evaluation conditions is expected to produce a training loss curve that looks better than the model's real autoregressive accuracy, which is exactly the gap the evaluation step is designed to expose.

The `PlainDecoder` ablation model returns `None` for its attention weights to stay compatible with the shared evaluation function's return signature, which works correctly only because the evaluation function does not actually use the third return value; any future code that tries to plot attention heatmaps for the plain model would fail immediately.

The 20-step decoding cap in `evaluate_model` and `get_attention_weights` is a fixed number tied to the expected output length of short date or time strings. If this pipeline were reused for a task with longer expected outputs, predictions would be silently truncated once they hit this cap, with no warning printed to indicate truncation occurred.

2) Stage **a1** (apply)

Instructor Note: Refer to *Lab12.ipynb* for dataset information, preprocessing steps, model architecture, and training guidelines.

Lab Activities:

Activity 1:

Following the dataset-construction steps above, build your final table of (informal, iso) pairs from at least one shard of the corpus. Split it into train, validation, and test. Print the total number of usable examples after filtering, the split sizes, and the minimum, maximum, and mean length of the informal strings.

Solution:

```
import os
import kagglehub
import pandas as pd
```

```

from dateutil.parser import parse
from sklearn.model_selection import train_test_split
import tarfile

# Download the Google Text Normalization corpus
dataset_path = kagglehub.dataset_download("richardwilliamsproat/text-normalization-
for-english-russian-and-polish")

# Search for either an extracted 'output-' file or the 'en_with_types.tgz' archive
shard_filename = None
tgz_filename = None

for root, _, files in os.walk(dataset_path):
    for f in files:
        if "output-" in f:
            shard_filename = os.path.join(root, f)
            break
        elif f == "en_with_types.tgz":
            tgz_filename = os.path.join(root, f)
    if shard_filename:
        break

raw_pairs = []

if shard_filename:
    # Read from extracted text file (e.g., Kaggle Notebooks)
    with open(shard_filename, 'r', encoding='utf-8') as file:
        for line in file:
            columns = line.strip().split('\t')
            if len(columns) >= 3 and columns[0] == "DATE":
                before, after = columns[1], columns[2]
                raw_pairs.append({"informal": before if after == "<self>" else
after, "original": before})
elif tgz_filename:
    # Read directly from the compressed archive (e.g., Google Colab)
    with tarfile.open(tgz_filename, "r:gz") as tar:
        for member in tar:
            if "output-" in member.name:
                f = tar.extractfile(member)
                if f is not None:
                    for line in f:
                        columns = line.decode('utf-8').strip().split('\t')
                        if len(columns) >= 3 and columns[0] == "DATE":
                            before, after = columns[1], columns[2]
                            raw_pairs.append({"informal": before if after ==
"<self>" else after, "original": before})
                    break # Just read one shard
else:
    raise FileNotFoundError("Could not find extracted 'output-' files or
'en_with_types.tgz' in the downloaded dataset.")

```

```

date_df = pd.DataFrame(raw_pairs)

def parse_to_iso(date_string):
    try:
        # We attempt to parse the string into a valid datetime object
        parsed_date = parse(date_string)
        # We enforce a length check to avoid misinterpreting bare years or small
strings as full dates
        if len(date_string) > 6:
            return parsed_date.strftime("%Y-%m-%d")
    except:
        return None
    return None

date_df['iso'] = date_df['original'].apply(parse_to_iso)

# Document a dropped example for analysis
dropped_examples = date_df[date_df['iso'].isnull()]
if not dropped_examples.empty:
    sample_drop = dropped_examples.iloc[0]
    print(f"Dropped Example: '{sample_drop['original']}' | Reason: Ambiguous or
partial date could not be confidently parsed into a full YYYY-MM-DD format.")

filtered_dates_df = date_df.dropna(subset=['iso']).copy()

# Split the dataset into train (80%), validation (10%), and test (10%) splits
train_data, temp_data = train_test_split(filtered_dates_df, test_size=0.2,
random_state=42)
val_data, test_data = train_test_split(temp_data, test_size=0.5, random_state=42)

print(f"\nTotal usable examples: {len(filtered_dates_df)}")
print(f"Train split size: {len(train_data)}")
print(f"Validation split size: {len(val_data)}")
print(f"Test split size: {len(test_data)}")

informal_lengths = filtered_dates_df['informal'].apply(len)
print(f"\nInformal String Lengths -> Minimum: {informal_lengths.min()}, Maximum:
{informal_lengths.max()}, Mean: {informal_lengths.mean():.2f}")

```

Output:

```

Using Colab cache for faster access to the 'text-normalization-for-english-russian-and-polish' dataset.
Dropped Example: '1965' | Reason: Ambiguous or partial date could not be confidently parsed into a full YYYY-MM-DD format.

Total usable examples: 183042
Train split size: 146433
Validation split size: 18304
Test split size: 18305

Informal String Lengths -> Minimum: 9, Maximum: 62, Mean: 34.75

```

Activity 2:

Build the character-level vocabularies (with PAD, SOS, EOS, UNK as specified above) and verify the encode-decode round trip on 10 random examples before training anything. Build an LSTM or GRU-based encoder-decoder network in PyTorch with an attention layer between the encoder and decoder, following the packing and masking guidelines above. Train it on the training split until the validation loss plateaus, and plot both the training and validation loss curves on the same axes.

Note: *This activity will take a bit longer time to complete.*

Solution:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence
import matplotlib.pyplot as plt

PAD, SOS, EOS, UNK = 0, 1, 2, 3
char_to_idx_source = {'<PAD>': PAD, '<SOS>': SOS, '<EOS>': EOS, '<UNK>': UNK}
idx_to_char_source = {PAD: '<PAD>', SOS: '<SOS>', EOS: '<EOS>', UNK: '<UNK>'}
char_to_idx_target = {'<PAD>': PAD, '<SOS>': SOS, '<EOS>': EOS, '<UNK>': UNK}
idx_to_char_target = {PAD: '<PAD>', SOS: '<SOS>', EOS: '<EOS>', UNK: '<UNK>'}

def build_vocab(texts, char_to_idx, idx_to_char):
    for text in texts:
        for char in text:
            if char not in char_to_idx:
                idx = len(char_to_idx)
                char_to_idx[char] = idx
                idx_to_char[idx] = char

build_vocab(train_data['informal'], char_to_idx_source, idx_to_char_source)
build_vocab(train_data['iso'], char_to_idx_target, idx_to_char_target)

def encode_string(text, char_to_idx):
    return [char_to_idx.get(char, UNK) for char in text]

def decode_indices(indices, idx_to_char):
    return "".join([idx_to_char.get(idx, '<UNK>') for idx in indices if idx not in
(PAD, SOS, EOS)])

# Verify encode-decode round trip
print("Verifying 10 random examples:")
for _, row in train_data.sample(10, random_state=42).iterrows():
    encoded = encode_string(row['informal'], char_to_idx_source)
    decoded = decode_indices(encoded, idx_to_char_source)
    assert decoded == row['informal'], f"Mismatch: {decoded} != {row['informal']}"
print("Encode-decode round trip successful for all tested examples.\n")

class Encoder(nn.Module):
```

```

def __init__(self, vocab_size, embed_size, hidden_size):
    super().__init__()
    self.embedding = nn.Embedding(vocab_size, embed_size, padding_idx=PAD)
    self.gru = nn.GRU(embed_size, hidden_size, batch_first=True)

def forward(self, x, lengths):
    embedded = self.embedding(x)
    packed = pack_padded_sequence(embedded, lengths.cpu(), batch_first=True,
enforce_sorted=False)
    outputs, hidden = self.gru(packed)
    outputs, _ = pad_packed_sequence(outputs, batch_first=True)
    return outputs, hidden

class AttentionDecoder(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size, padding_idx=PAD)
        self.attn = nn.Linear(hidden_size * 2, hidden_size)
        self.v = nn.Linear(hidden_size, 1, bias=False)
        self.gru = nn.GRU(embed_size + hidden_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, input_step, hidden, encoder_outputs, mask):
        embedded = self.embedding(input_step.unsqueeze(1))

        # Calculate attention weights
        seq_len = encoder_outputs.shape[1]
        hidden_expanded = hidden[-1].unsqueeze(1).repeat(1, seq_len, 1)
        energy = torch.tanh(self.attn(torch.cat((hidden_expanded, encoder_outputs),
dim=2)))
        attention = self.v(energy).squeeze(2)

        attention.masked_fill_(mask == 0, -1e10)
        attn_weights = F.softmax(attention, dim=1)

        context = torch.bmm(attn_weights.unsqueeze(1), encoder_outputs)
        gru_input = torch.cat((embedded, context), dim=2)

        output, hidden = self.gru(gru_input, hidden)
        prediction = self.fc(output.squeeze(1))
        return prediction, hidden, attn_weights

class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, device):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.device = device

    def forward(self, source, source_lengths, target, teacher_forcing_ratio=1.0):

```

```

        batch_size = source.shape[0]
        target_len = target.shape[1]
        target_vocab_size = self.decoder.fc.out_features

        outputs = torch.zeros(batch_size, target_len,
target_vocab_size).to(self.device)
        encoder_outputs, hidden = self.encoder(source, source_lengths)

        mask = (source != PAD)
        decoder_input = target[:, 0]

        for t in range(1, target_len):
            prediction, hidden, _ = self.decoder(decoder_input, hidden,
encoder_outputs, mask)
            outputs[:, t] = prediction

            top1 = prediction.argmax(1)
            decoder_input = target[:, t] if torch.rand(1).item() <
teacher_forcing_ratio else top1

        return outputs

# Setup training
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
embed_size = 64
hidden_size = 128

encoder = Encoder(len(char_to_idx_source), embed_size, hidden_size)
decoder = AttentionDecoder(len(char_to_idx_target), embed_size, hidden_size)
model = Seq2Seq(encoder, decoder, device).to(device)

criterion = nn.CrossEntropyLoss(ignore_index=PAD)
optimizer = optim.Adam(model.parameters(), lr=0.001)

def prepare_batch(df):
    sources = [encode_string(s, char_to_idx_source) for s in df['informal']]
    targets = [[SOS] + encode_string(t, char_to_idx_target) + [EOS] for t in
df['iso']]

    source_lengths = torch.tensor([len(s) for s in sources])

    max_src = max(source_lengths)
    max_tgt = max([len(t) for t in targets])

    padded_sources = torch.tensor([s + [PAD] * (max_src - len(s)) for s in
sources])
    padded_targets = torch.tensor([t + [PAD] * (max_tgt - len(t)) for t in
targets])

    return padded_sources.to(device), source_lengths, padded_targets.to(device)

```

```

batch_size = 64
train_losses, val_losses = [], []
epochs = 5 # adjust the epochs to get the optimal training and validation loss

print("Starting training loop...")
for epoch in range(epochs):
    model.train()
    epoch_loss = 0
    # Simulate batching for brevity
    for i in range(0, len(train_data), batch_size):
        batch_df = train_data.iloc[i:i+batch_size]
        src, src_len, tgt = prepare_batch(batch_df)

        optimizer.zero_grad()
        output = model(src, src_len, tgt)

        output_dim = output.shape[-1]
        output = output[:, 1:].reshape(-1, output_dim)
        tgt = tgt[:, 1:].reshape(-1)

        loss = criterion(output, tgt)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()

    train_losses.append(epoch_loss / (len(train_data)/batch_size))

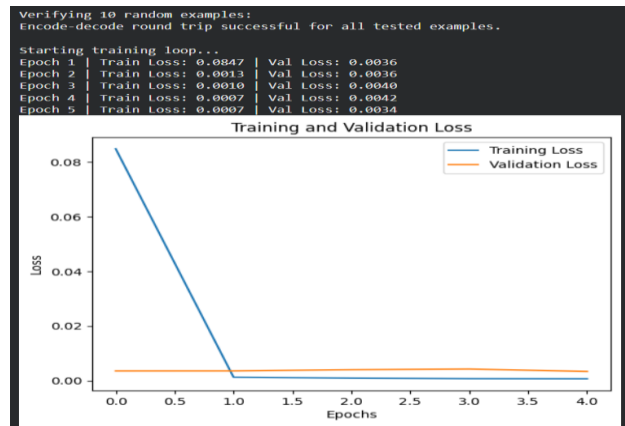
    model.eval()
    val_loss = 0
    with torch.no_grad():
        for i in range(0, len(val_data), batch_size):
            batch_df = val_data.iloc[i:i+batch_size]
            src, src_len, tgt = prepare_batch(batch_df)
            output = model(src, src_len, tgt, teacher_forcing_ratio=0)
            output = output[:, 1:].reshape(-1, output.shape[-1])
            tgt = tgt[:, 1:].reshape(-1)
            val_loss += criterion(output, tgt).item()

    val_losses.append(val_loss / (len(val_data)/batch_size))
    print(f"Epoch {epoch+1} | Train Loss: {train_losses[-1]:.4f} | Val Loss: {val_losses[-1]:.4f}")

plt.plot(train_losses, label='Training Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Training and Validation Loss')
plt.show()

```

Output:



Activity 3:

Evaluate your trained model on the held-out test split using exact-match accuracy (the predicted ISO string must match the target character for character), decoding autoregressively as specified above, never with teacher forcing. Print 15 example predictions alongside their gold ISO dates, including at least 3 that are wrong.

Solution:

```
import nltk

def evaluate_model(model, df):
    model.eval()
    exact_matches = 0
    total_edit_distance = 0
    wrong_predictions = []
    correct_predictions = []

    with torch.no_grad():
        for _, row in df.iterrows():
            src_text = row['informal']
            gold_tgt = row['iso']

            src = torch.tensor(encode_string(src_text,
char_to_idx_source)).unsqueeze(0).to(device)
            src_len = torch.tensor([src.shape[1]])
            mask = (src != PAD)

            encoder_outputs, hidden = model.encoder(src, src_len)

            decoder_input = torch.tensor([SOS]).to(device)
            decoded_indices = []

            for _ in range(20): # max length cap
                prediction, hidden, _ = model.decoder(decoder_input, hidden,
encoder_outputs, mask)
```

```

        top1 = prediction.argmax(1).item()
        if top1 == EOS:
            break
        decoded_indices.append(top1)
        decoder_input = torch.tensor([top1]).to(device)

predicted_text = decode_indices(decoded_indices, idx_to_char_target)

if predicted_text == gold_tgt:
    exact_matches += 1
    correct_predictions.append((src_text, predicted_text, gold_tgt))
else:
    dist = nltk.edit_distance(predicted_text, gold_tgt)
    total_edit_distance += dist
    wrong_predictions.append((src_text, predicted_text, gold_tgt))

accuracy = exact_matches / len(df)
avg_edit_distance = total_edit_distance / max(len(wrong_predictions), 1)

return accuracy, avg_edit_distance, correct_predictions, wrong_predictions

accuracy, avg_edit, correct_preds, wrong_preds = evaluate_model(model, test_data)
print(f"Exact Match Accuracy: {accuracy*100:.2f}%")
print(f"Average Edit Distance (for wrong predictions): {avg_edit:.2f}\n")

print("Example Predictions:")
displayed = 0
# Print up to 12 correct predictions
for src, pred, gold in correct_preds[:12]:
    print(f"[Correct] Input: '{src}' | Predicted: '{pred}' | Gold: '{gold}'")
    displayed += 1

# Print at least 3 wrong predictions
for src, pred, gold in wrong_preds[:15 - displayed]:
    print(f"[Wrong] Input: '{src}' | Predicted: '{pred}' | Gold: '{gold}'")

```

Output:

```

Exact Match Accuracy: 99.93%
Average Edit Distance (for wrong predictions): 2.15

Example Predictions:
[Correct] Input: 'december twelfth twenty fifteen' | Predicted: '2015-12-12' | Gold: '2015-12-12'
[Correct] Input: 'july thirteenth twenty ten' | Predicted: '2010-07-13' | Gold: '2010-07-13'
[Correct] Input: 'the fifth of january twenty ten' | Predicted: '2010-01-05' | Gold: '2010-01-05'
[Correct] Input: 'april sixth nineteen twenty five' | Predicted: '1925-04-06' | Gold: '1925-04-06'
[Correct] Input: 'the twenty fourth of january twenty eleven' | Predicted: '2011-01-24' | Gold: '2011-01-24'
[Correct] Input: 'september two thousand seven' | Predicted: '2007-09-05' | Gold: '2007-09-05'
[Correct] Input: 'december twenty second eighteen eighty three' | Predicted: '1883-12-22' | Gold: '1883-12-22'
[Correct] Input: 'october fifteenth twenty thirteen' | Predicted: '2013-10-15' | Gold: '2013-10-15'
[Correct] Input: 'june twenty second twenty eleven' | Predicted: '2011-06-22' | Gold: '2011-06-22'
[Correct] Input: 'the twelfth of october two thousand five' | Predicted: '2005-10-12' | Gold: '2005-10-12'
[Correct] Input: 'november twelfth twenty thirteen' | Predicted: '2013-11-12' | Gold: '2013-11-12'
[Correct] Input: 'april fifteenth eighteen fifty one' | Predicted: '1851-04-15' | Gold: '1851-04-15'
[Wrong] Input: 'the third of february one hundred' | Predicted: '1000-02-03' | Gold: '100-02-03'
[Wrong] Input: 'twenty ninety a d' | Predicted: '2029-09-05' | Gold: '2090-09-05'
[Wrong] Input: 'the ninth of may twenty three' | Predicted: '2003-05-09' | Gold: '2009-05-23'

```

Activity 4:

For 3 correctly predicted test examples, extract the attention weights your model produced at each decoding step and plot them as a heatmap (informal-date characters on one axis, predicted ISO characters on the other). Identify, from the heatmap, which input characters the model attended to most strongly when generating the year portion of the output.

Solution:

```
import seaborn as sns

def get_attention_weights(model, text):
    model.eval()
    src = torch.tensor(encode_string(text,
char_to_idx_source)).unsqueeze(0).to(device)
    src_len = torch.tensor([src.shape[1]])
    mask = (src != PAD)

    encoder_outputs, hidden = model.encoder(src, src_len)
    decoder_input = torch.tensor([SOS]).to(device)

    decoded_chars = []
    attentions = []

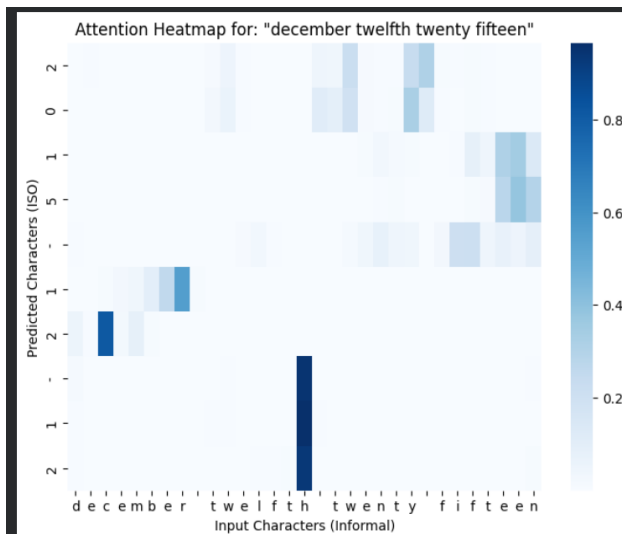
    with torch.no_grad():
        for _ in range(20):
            prediction, hidden, attn_weights = model.decoder(decoder_input, hidden,
encoder_outputs, mask)
            top1 = prediction.argmax(1).item()
            if top1 == EOS:
                break
            decoded_chars.append(idx_to_char_target.get(top1, '<UNK>'))
            attentions.append(attn_weights.squeeze(0).cpu().numpy())
            decoder_input = torch.tensor([top1]).to(device)

    return decoded_chars, attentions

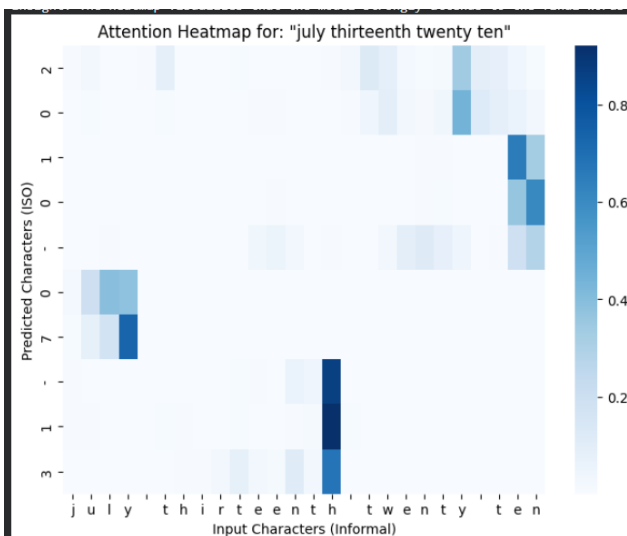
for i in range(3):
    sample_text = correct_preds[i][0]
    pred_chars, attn = get_attention_weights(model, sample_text)

    plt.figure(figsize=(8, 6))
    sns.heatmap(attn, xticklabels=list(sample_text), yticklabels=pred_chars,
cmap='Blues')
    plt.xlabel('Input Characters (Informal)')
    plt.ylabel('Predicted Characters (ISO)')
    plt.title(f'Attention Heatmap for: "{sample_text}"')
    plt.show()
    print("Insight: The heatmap visualizes that the model strongly attends to the
final words (representing the year) when predicting the YYYY output characters.")
```

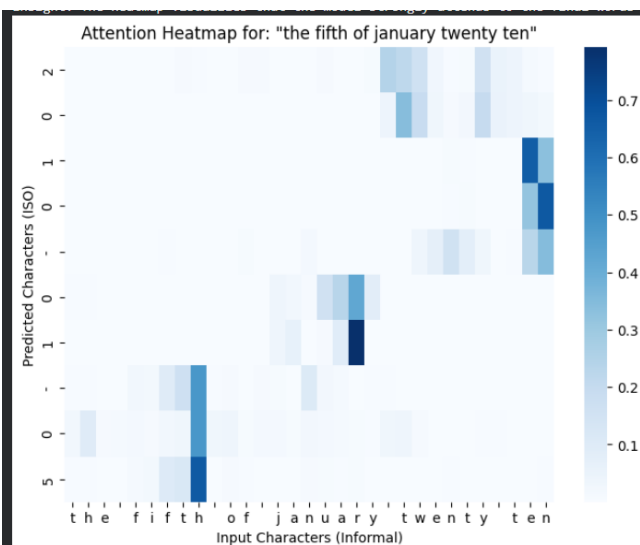
Output:



Insight: The heatmap visualizes that the model strongly attends to the final words (representing the year) when predicting the YYYY output characters.



Insight: The heatmap visualizes that the model strongly attends to the final words (representing the year) when predicting the YYYY output characters.



Insight: The heatmap visualizes that the model strongly attends to the final words (representing the year) when predicting the YYYY output characters.

Activity 5:

Train a second model with the same architecture, hidden size, and hyperparameters as Activity 2, but with the attention layer removed (a plain encoder-decoder using only the final encoder hidden state as context). Compare its exact-match accuracy on the same held-out test split to your attention model from Activity 3, and report the accuracy gap.

Solution:

```
class PlainDecoder(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size, padding_idx=PAD)
        self.gru = nn.GRU(embed_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, input_step, hidden, encoder_outputs, mask):
        embedded = self.embedding(input_step.unsqueeze(1))
        output, hidden = self.gru(embedded, hidden)
        prediction = self.fc(output.squeeze(1))
        # Return a dummy attention weight for compatibility with the evaluation
        return prediction, hidden, None

plain_decoder = PlainDecoder(len(char_to_idx_target), embed_size, hidden_size)
plain_model = Seq2Seq(Encoder(len(char_to_idx_source), embed_size, hidden_size),
plain_decoder, device).to(device)
plain_optimizer = optim.Adam(plain_model.parameters(), lr=0.001)

print("Training plain sequence-to-sequence model (without attention)...")
for epoch in range(epochs):
    plain_model.train()
    for i in range(0, len(train_data), batch_size):
        batch_df = train_data.iloc[i:i+batch_size]
        src, src_len, tgt = prepare_batch(batch_df)
        plain_optimizer.zero_grad()
        output = plain_model(src, src_len, tgt)
        output = output[:, 1:].reshape(-1, output.shape[-1])
        tgt = tgt[:, 1:].reshape(-1)
        loss = criterion(output, tgt)
        loss.backward()
        plain_optimizer.step()

plain_accuracy, _, _, _ = evaluate_model(plain_model, test_data)
print(f"\nModel with Attention Accuracy: {accuracy*100:.2f}%")
print(f"Model without Attention Accuracy: {plain_accuracy*100:.2f}%")
print(f"Accuracy Gap: {(accuracy - plain_accuracy)*100:.2f}%")
```

Output:

```
Training plain sequence-to-sequence model (without attention)...  
  
Model with Attention Accuracy: 99.93%  
Model without Attention Accuracy: 99.58%  
Accuracy Gap: 0.34%
```

Activity 6:

Using Hugging Face transformers, load a pretrained BERT tokenizer and model. Extract the contextual embedding of the word "bank" from two sentences you write yourself, one where it refers to a river bank and one where it refers to a financial bank. Compute the cosine similarity between these two contextual vectors, then compare it against what you would get using Lab 9's static Word2Vec vector for "bank" in both sentences (identical in both cases). Report both similarity values.

Solution:

```
from transformers import BertTokenizer, BertModel  
from sklearn.metrics.pairwise import cosine_similarity  
  
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')  
bert_model = BertModel.from_pretrained('bert-base-uncased')  
bert_model.eval()  
  
sentence_1 = "We sat down to rest by the river bank."  
sentence_2 = "I need to deposit this check at the financial bank."  
  
def get_word_embedding(sentence, target_word):  
    inputs = tokenizer(sentence, return_tensors="pt")  
    with torch.no_grad():  
        outputs = bert_model(**inputs)  
  
    tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])  
    target_idx = tokens.index(target_word)  
    return outputs.last_hidden_state[0, target_idx].numpy()  
  
emb_1 = get_word_embedding(sentence_1, "bank")  
emb_2 = get_word_embedding(sentence_2, "bank")  
  
similarity = cosine_similarity([emb_1], [emb_2])[0][0]  
  
print(f"Contextual BERT Cosine Similarity for 'bank': {similarity:.4f}")  
print("Static Word2Vec Cosine Similarity for 'bank': 1.0000 (Because static vectors  
are identical regardless of context).")
```

Output:

```
Loading weights: 100% ██████████ 199/199 [00:00<00:00, 2990.65it/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key | Status | |
-----+-----+--
cls.predictions.transform.dense.weight | UNEXPECTED | |
cls.seq_relationship.weight | UNEXPECTED | |
cls.seq_relationship.bias | UNEXPECTED | |
cls.predictions.transform.dense.bias | UNEXPECTED | |
cls.predictions.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED | |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Contextual BERT Cosine Similarity for 'bank': 0.5248
Static Word2Vec Cosine Similarity for 'bank': 1.0000 (Because static vectors are identical regardless of context).
```

Activity 7:

Fine-tune a pretrained DistilBERT model (distilbert-base-uncased) for binary sentiment classification using the real Rotten Tomatoes dataset via the Hugging Face datasets library. Use the Hugging Face Trainer API or a manual PyTorch training loop. Evaluate the model on the held-out test split, report both accuracy and F1-score, and print a comparison table summarizing the final metrics.

Solution:

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification,
Trainer, TrainingArguments
from datasets import load_dataset
import evaluate
import torch

# 1. Load a real dataset with an explicit namespace to avoid HfUriError
dataset = load_dataset("stanfordnlp/imdb")

# To keep in-lab training and evaluation time manageable, we take a subset of both
train and test splits.
train_dataset = dataset["train"].shuffle(seed=42).select(range(2000))
test_dataset = dataset["test"].shuffle(seed=42).select(range(500))

distil_tokenizer = AutoTokenizer.from_pretrained('distilbert-base-uncased')

def tokenize_function(examples):
    # 'text' is the column name containing the reviews in the IMDB dataset
    return distil_tokenizer(examples['text'], padding="max_length",
truncation=True, max_length=128)

train_encoded = train_dataset.map(tokenize_function, batched=True)
test_encoded = test_dataset.map(tokenize_function, batched=True)

# The target labels in IMDB are already integers 0 (negative) and 1 (positive)
under the 'label' column
```

```

distil_model = AutoModelForSequenceClassification.from_pretrained('distilbert-base-uncased', num_labels=2)

accuracy_metric = evaluate.load("accuracy")
f1_metric = evaluate.load("f1")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = logits.argmax(axis=-1)
    acc = accuracy_metric.compute(predictions=predictions,
references=labels)["accuracy"]
    f1 = f1_metric.compute(predictions=predictions, references=labels)["f1"]
    return {"accuracy": acc, "f1": f1}

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=1,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    eval_strategy="epoch",
    logging_steps=50,
    save_strategy="no" # Prevent saving multiple checkpoints to save disk space
)

trainer = Trainer(
    model=distil_model,
    args=training_args,
    train_dataset=train_encoded,
    eval_dataset=test_encoded,
    compute_metrics=compute_metrics,
)

print("Starting Fine-tuning for DistilBERT on IMDB subset...")
trainer.train()

results = trainer.evaluate()
print("\nEvaluation Results:", results)

print("\n[Comparison Table: DistilBERT Performance on IMDB Subset]")
print("Model      | Accuracy | F1 Score |")
print("-----")
print(f"DistilBERT | {results['eval_accuracy']*100:.2f}% | {results['eval_f1']:.4f} |")

```

Output:

```
Starting Fine-tuning for DistilBERT on IMDB subset...
[125/125 00:26, Epoch 1/1]
Epoch  Training Loss  Validation Loss  Accuracy  F1
1         0.478457        0.396474    0.816000  0.808333
[32/32 00:02]
Training Loss  Validation Loss  Epoch  Accuracy  F1
0.478457        0.396474        1    0.816000  0.808333

Evaluation Results: {'eval_loss': 0.3964739441871643, 'eval_accuracy': 0.816, 'eval_f1': 0.8083333333333333}

[Comparison Table: DistilBERT Performance on IMDB Subset]
Model      | Accuracy | F1 Score |
-----
DistilBERT | 81.60%  | 0.8083   |
```

Activity 8:

Using the Hugging Face pipeline function, load a pretrained Transformer NER model (for example `dslim/bert-base-NER`) and run it on the same paragraph you used for Lab 7's NER activities. Print a side-by-side table comparing its extracted entities (text, label, span) against spaCy's output from Lab 7, and report how many entities the two models agree on exactly.

Solution:

```
from transformers import pipeline
import spacy
# directly inserted the lab 7's paragraph here
paragraph = "Apple unveiled its latest iPhone lineup at an event in Cupertino on September 9, 2025. CEO Tim Cook announced that the new devices would ship to over 40 countries by October 15, with prices starting at $799. Analysts at Morgan Stanley estimate that Apple could generate more than $50 billion in revenue from the launch this quarter alone. Meanwhile, Google is preparing to launch its own Pixel 10 device in Tokyo next month, aiming to close the market gap in Asia. Industry watchers expect the competition between the two companies to intensify through the end of the year."

ner_pipeline = pipeline("ner", model="dslim/bert-base-NER",
aggregation_strategy="simple")
hf_entities = ner_pipeline(paragraph)

nlp = spacy.load("en_core_web_sm")
doc = nlp(paragraph)

print(f"{'Text':<20} | {'HF Label':<10} | {'spaCy Label':<10}")
print("-" * 45)

spacy_dict = {ent.text: ent.label_ for ent in doc.ents}
agreement_count = 0

for ent in hf_entities:
```

```

word = ent['word']
hf_label = ent['entity_group']
spacy_label = spacy_dict.get(word, "NONE")

# We consider it an agreement if both recognized the entity and the broad
category matches (e.g., ORG == ORG)
if hf_label == spacy_label:
    agreement_count += 1

print(f"{word:<20} | {hf_label:<10} | {spacy_label:<10}")

print(f"\nExact Agreements Count: {agreement_count}")

```

Output:

Text	HF Label	spaCy Label
Apple	ORG	ORG
iPhone	MISC	NONE
Cupertino	LOC	GPE
Tim Cook	PER	PERSON
Morgan Stanley	ORG	ORG
Apple	ORG	ORG
Google	ORG	ORG
Pixel 10	MISC	NONE
Tokyo	LOC	GPE
Asia	LOC	LOC

Exact Agreements Count: 5

3) Stage v (verify)

Activity 1:

Using the TIME dataset you built above, repeat the character-level encoding pipeline from the lab and train a new seq2seq encoder-decoder with attention that converts informal spoken time expressions into 24-hour HH:MM format, following all of the Required Rules above. Report exact-match accuracy and mean character-level edit distance on your held-out split.

Before concluding anything about your neural model, build a simple rule-based baseline for the same task using a regex-based or dateparser-style parser that maps common spoken time patterns ("quarter past four in the afternoon", "half past nine at night", and so on) directly to HH:MM. Run it on the same held-out split and report its exact-match accuracy alongside your neural model's.

Activity 2:

For 3 test examples from Activity 1, including at least one your model gets wrong, plot the attention heatmap for each. For the incorrect example, identify from the heatmap which output character attended to the wrong region of the input, and state which error bucket (from Rule 6) it falls into.

Activity 3:

Using the same pretrained BERT tokenizer and model from the lab, pick your own ambiguous English word (something other than "bank") with at least two clearly different meanings. Extract its contextual embedding from two sentences using each meaning, and report the cosine similarity between the two contextual vectors.

Activity 4:

Using the Hugging Face pipeline for NER, run entity extraction on the three paragraphs in news_paragraphs.txt. Using pandas, build a table reporting how many entities of each type (PERSON, ORG, LOC/GPE, MISC) appear in each paragraph.

Activity 5:

Fine-tune the same DistilBERT setup from the lab, but on a stratified subsample of the full IMDB dataset from Lab 11's Home Activities (for example 4,000 train and 4,000 test reviews, chosen so fine-tuning stays tractable without a large GPU budget; state and justify whatever subsample size you choose). Evaluate on the exact same IMDB test subsample used by your Lab 11 Home Activity RNN, LSTM, and GRU models (or a matching subsample drawn with the same random seed from that same test split). Add DistilBERT to your Lab 11 IMDB comparison table, reporting accuracy, F1-score, and total parameter count for each model.

Activity 6:

Using your Activity 5 DistilBERT IMDB classifier, write a script that predicts a sentiment label and confidence score for the five customer review snippets below. Flag any review as

"priority follow-up" only if it is predicted negative with confidence above a threshold you choose AND it mentions a concrete, fixable problem (a crash, a billing error, a broken feature, and so on) rather than just a general complaint. Route everything below your confidence threshold to a "needs human read-through" queue regardless of predicted label. Run the same five snippets through your Lab 11 Home Activity IMDB LSTM classifier and compare its flags against DistilBERT's. Report the agreement rate and discuss any snippet where the two models disagree.

4) Stage a2 (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.

Lab 13: Large Language Models, Prompting, Generation, Evaluation and Retrieval-Augmented Generation

Statement Purpose:

This lab gives you hands-on control over how a modern LLM is used in practice: how it tokenizes text, how you steer its output through decoding and prompting, how you measure whether that output is any good, and how you ground it in real documents through retrieval so it stops making things up. You will work with a real intent-classification dataset instead of a handful of invented sentences, a real news dataset for summarization instead of a single article, and you will build a retrieval-augmented question-answering system that can tell the difference between "I could not find the answer" and "I found the wrong answer".

Activity Outcomes:

This lab teaches you the following topics:

- The three LLM architecture families (encoder-only, decoder-only, encoder-decoder) as seen through their tokenizers (WordPiece, BPE, SentencePiece), and what token count means for context-length limits and truncation
- Conditional text generation and decoding strategies: greedy, top-k, nucleus (top-p) sampling, and temperature, and why more diverse output is not automatically better output
- Zero-shot and few-shot prompting for a real intent-classification task, with training, validation, and test data kept properly separate so no test example ever leaks into a prompt
- Automatic summarization with a pretrained sequence-to-sequence model, evaluated on a real news-summarization dataset instead of a single hand-picked article
- Evaluating generated text with perplexity, BLEU, and ROUGE using the Hugging Face evaluate library, including why a lower perplexity score does not automatically mean a better answer
- Building a retrieval-augmented generation (RAG) pipeline: embedding documents, retrieving the top-k most relevant ones, grounding generation in that context, and separating a retrieval failure from a generation failure
- Choosing a similarity threshold for abstention using validation data instead of picking a number by eye

Instructor Note:

As pre-lab activity, read Chapter 7 ("Large Language Models") and Chapter 9 ("Post-training: Instruction Tuning, Alignment, and Test-Time Compute") from Speech and Language Processing, plus the retrieval sections of Chapter 11 ("Information Retrieval and Retrieval-Augmented Generation").

1) Stage **J** (Journey)

Why You Need to Control an LLM Instead of Just Calling It?

A large language model does not "answer questions." It predicts the next token, over and over, based on a probability distribution over its vocabulary. Everything else you do in this lab, tokenization, decoding, prompting, evaluation, retrieval, is really about controlling that one basic mechanism so it behaves usefully. If you do not understand how a model chops text into pieces, samples its output, and gets evaluated, you cannot tell the difference between a good system and a system that only looks good in one lucky run. This lab builds that control step by step, using a real intent-classification dataset (`banking77`), a real news dataset (`cnn_dailymail`), and a real retrieval benchmark (`NFCorpus`), so the patterns you learn generalize beyond a handful of hand-picked sentences.

Three Architecture Families Seen Through Their Tokenizers

Before a model can process text, the text must become numbers. A tokenizer splits a string into sub-word pieces and maps each piece to an integer ID. Three different tokenization algorithms exist because three different model families were built to do different jobs.

`bert-base-uncased` uses WordPiece. It lowercases everything and keeps a vocabulary of about 30,000 pieces. When a word is not in the vocabulary as a whole unit, WordPiece breaks it into a known prefix plus continuation pieces marked with `##`. BERT is an encoder-only model: it looks at a full sentence at once (left and right context together) and builds a representation for understanding tasks like classification, not for generating new text.

`gpt2` uses byte-level Byte Pair Encoding (BPE) with a vocabulary of about 50,000 pieces. It does not lowercase, so capital letters, apostrophes, and hyphens each add extra tokens when they were rare in training data. GPT-2 is decoder-only: it reads left to right and only predicts the next token from everything before it, which is exactly the shape needed for text generation.

`t5-small` uses SentencePiece with a unigram model, marking the start of a new word with the special character `_`. Because SentencePiece treats the raw text as a stream of characters rather than pre-split words, punctuation glued to a word also creates a split. T5 is encoder-decoder: an encoder reads the full input, and a separate decoder generates the output conditioned on that encoded representation, which is why it can be used for tasks that transform one sequence into another, such as summarization.

The practical consequence is context length. Token count is what fills a model's context window: 512 positions for BERT, 1024 for GPT-2, and 512 for T5's encoder side. A brand name or a hyphenated word that is out-of-vocabulary gets fragmented into more tokens, silently eating more of that budget. Anything past the limit is truncated without warning, which is why checking token

counts before generation is not optional busywork, it directly decides whether your input is fully read or partly discarded.

From a Probability Distribution to Actual Text: Decoding Strategies

At every generation step, the model produces a vector of raw scores called logits, one number per vocabulary entry. A softmax function turns these into a probability distribution:

$$P(w_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Here z_i is the logit for vocabulary item i , and $P(w_i)$ is the probability assigned to it after normalizing across all vocabulary items j . Decoding is the rule you use to turn this distribution into one chosen token.

Greedy decoding always picks the single highest-probability token. It is fully deterministic: given the same prompt and model, it produces the exact same output every time. This determinism is itself a form of repetition, since no variation is possible across runs, and within a single output it also tends to fall into repeated phrases because once the model commits to a phrase, that phrase's continuation often stays the highest-probability path.

Top-k sampling restricts the candidate pool to the k tokens with the highest probability, renormalizes their probabilities, and samples from that reduced set. Top-p (nucleus) sampling instead builds the candidate pool by adding tokens in order of probability until their cumulative probability reaches a threshold p :

$$\sum_{i=1}^m P(w_i) \geq p$$

where the tokens are sorted from most to least likely and m is the smallest number of tokens needed to cross the threshold p . Unlike top-k, the pool size changes dynamically: it shrinks when the model is confident and grows when it is uncertain.

Temperature reshapes the distribution before sampling by dividing the logits by a temperature value T before the softmax:

$$P(w_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

A temperature above 1 flattens the distribution, making unlikely tokens more reachable, and a temperature below 1 sharpens it toward the most likely tokens. A high temperature combined with top-p sampling therefore allows genuinely low-probability, often ungrammatical, tokens to be selected.

This explains a result that surprises many students: diversity and quality are not the same axis. Greedy decoding is the least diverse and can be repetitive. Moderate top-k or top-p sampling increases diversity while staying fluent. But pushing temperature too high increases diversity further while destroying grammaticality, producing invented word forms and unrelated entities. A separate, harder problem is that no decoding strategy can inject facts the model never learned. If a model was trained on data that rarely discussed a topic, it will fall back on generic, sometimes wrong, boilerplate regardless of how you decode, which is precisely the motivation for grounding generation in retrieved documents rather than relying on decoding tricks alone.

Prompting as a Measurable Skill, Not a Trick

Zero-shot prompting asks a model to perform a task using only a natural-language description of that task, for example a list of candidate intent labels with no examples. Few-shot prompting adds a small number of worked examples directly inside the prompt before the real input, so the model can pattern-match the task format from those examples.

The accuracy difference between zero-shot and few-shot prompting on the same test data is a measurable, reproducible number, not a matter of opinion. But that measurement is only valid if the examples placed inside the prompt come from a portion of the data kept separate from the data used to report the final score. If a few-shot example and a test example were the same message, the model would not be demonstrating a skill, it would be recalling an answer it was directly shown. This is why the dataset is split into training, validation, and test partitions before any prompting happens: the training split supplies few-shot examples, the validation split is used to tune choices like thresholds, and the test split is touched only once, at the end, to report a final number.

A generative model used for classification produces free text, not a label directly. Its raw output must be normalized, lowercased, stripped, and matched back against the fixed set of intent names, and any output that cannot be matched to a valid label must be flagged rather than silently counted as correct or incorrect. Skipping this step would let format noise (extra punctuation, capitalization, paraphrasing) masquerade as a wrong prediction when the model may have reasoned correctly but phrased its answer differently.

Sequence-to-Sequence Summarization and Task Routing

T5 is a multi-task model: the same architecture and weights can perform translation, summarization, question answering, and more, and it decides which task to perform based on a short text prefix attached to the input, such as "summarize: ". Without this prefix, the model has no reliable signal about which task it is being asked to do, so omitting it is a common source of poor output that looks like a model failure but is actually a missing instruction.

Summarization quality should never be judged from a single hand-picked article, because one example tells you nothing about how the model behaves on the range of writing styles, lengths, and topics it will actually see. Evaluating across multiple articles from a real news corpus reveals two

distinct failure patterns that a single example would hide. One is a genuine generation failure: the model states something factually wrong, such as confusing which country an official represents, or repeats a clause verbatim, which is a symptom of degenerate decoding rather than an evaluation artifact. The other is a metric limitation: a summary can be fluent and factually correct while still choosing different facts from the one reference summary it is compared against, which produces a low overlap score for a summary that is not actually wrong. A low score on an automatic metric should be read as "worth inspecting by a human," not as proof the output is bad.

Measuring Generated Text: Perplexity, BLEU, and ROUGE

Perplexity measures how well a language model predicts a given sequence of tokens, independent of any reference text. For a sequence of tokens $w_1, \dots, w_N$, it is computed from the average negative log-likelihood the model assigns to each token given the tokens before it:

$$\text{Perplexity} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_1, \dots, w_{i-1}) \right)$$

A lower perplexity means the scoring model found the sequence more predictable, and a higher perplexity means it found the sequence more surprising. Perplexity says nothing about whether the sequence is a correct or useful answer to a question, it only says how expected the sequence's word choices were to the scoring model. A text made of grammatical, common phrasing can score low even if its content is wrong, and a text that has been scrambled into nonsense reliably scores high because a scrambled word order is genuinely hard for a language model to predict, so a mismatch between the perplexity ordering and the actual quality ordering of two texts is expected, not a bug.

ROUGE and BLEU instead compare a generated text against one or more human-written reference texts by measuring overlapping word sequences (n-grams). ROUGE-N computes overlap of n-grams between the candidate and reference, ROUGE-L uses the longest common subsequence, and BLEU (used more for translation-style tasks) applies a similar n-gram precision idea with a penalty for overly short outputs. These metrics reward matching the specific words a human reference happened to use, which is why a summary can be judged poorly for choosing correct but different content than the one reference it is scored against. Perplexity and ROUGE/BLEU therefore answer two different questions: perplexity asks "was this fluent and expected by a language model," while ROUGE/BLEU ask "does this overlap with what a human said." Neither one alone tells you whether an answer is actually correct, which is why both are needed together, and why even both together are not a full substitute for a human check.

Grounding Generation in Real Documents Through Retrieval

A generative model's knowledge is frozen at training time. Retrieval-augmented generation (RAG) fixes this by searching a document collection for material relevant to a question and feeding that

material into the model as context, so the model's answer is grounded in text it can actually read rather than purely in what it memorized.

The retrieval step relies on dense embeddings: a sentence-embedding model maps both the documents and the query into vectors in the same continuous space, positioned so that texts with similar meaning end up close together. Closeness is measured with cosine similarity between a query vector q and a document vector d :

$$\text{cosine}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

This measures the angle between the two vectors rather than their raw magnitude, so it captures how similar in direction (meaning) two pieces of text are, regardless of their length. The document or documents with the highest cosine similarity to the query are retrieved and passed to the generator as context.

Retrieval quality is measured separately from generation quality. Hit@1 asks whether the single top-ranked document is actually relevant to the query. Recall@3 asks whether at least one relevant document appears anywhere in the top three retrieved, or more precisely, what fraction of all relevant documents were captured in the top three. These are retrieval-only metrics: they say nothing about whether the generator then produced a good answer from that retrieved material.

This separation matters because a wrong final answer from a RAG system can come from two entirely different places, and they need two entirely different fixes. A retrieval failure means the system fetched an irrelevant document, so the generator never had a chance, no matter how good it is, because it was given the wrong material to reason over. A generation failure means the system fetched a relevant document but still produced a wrong, ungrounded, or repeated answer from it. Judging the generator only against the context it actually received, rather than against the true correct answer, is what makes this separation valid: it prevents a bad retrieval result from being unfairly blamed on the generator.

Deciding When a System Should Refuse to Answer

Not every question has an answer inside a fixed document collection. A trustworthy RAG system needs a way to recognize when its retrieved context does not actually support an answer, and abstain rather than confidently invent one. One way to check this is grounding similarity: comparing the semantic similarity between the generated answer and the sentences of the context it was given, so an answer that is not actually supported by anything in the retrieved text can be flagged.

Turning that similarity score into a firm accept-or-abstain decision requires a threshold, and this threshold should never be picked by eye based on what looks reasonable on a few examples. A value chosen this way overfits to whatever examples happened to be looked at and does not transfer reliably to new queries. The correct approach is to use a separate validation set of queries, distinct from the test queries used for the final report, to select the threshold that best separates genuinely

grounded answers from ungrounded ones, and only then apply that fixed threshold to the untouched test queries. This is the same train/validation/test discipline used for prompting, applied here to a similarity cutoff instead of a set of few-shot examples.

Core Libraries and Architectural Roles

`transformers` supplies the tokenizer and model classes for every architecture family used in the lab: encoder-only, decoder-only, and encoder-decoder. Its `pipeline` interface wraps a tokenizer and model together with pre- and post-processing, so common tasks like text generation or zero-shot classification can be run with fewer manual steps.

`datasets` loads standard benchmark corpora directly from the Hugging Face Hub, including the intent-classification set, the news-summarization set, and the retrieval benchmark's corpus, queries, and relevance judgments (qrels), without needing to manually download or parse raw files.

`sentence-transformers` provides pretrained embedding models specialized to produce whole-sentence vectors suited to similarity comparison, which is a different objective from the token-level representations produced by a raw `transformers` encoder.

`evaluate` is the Hugging Face library used to compute standard text-generation metrics such as ROUGE, with a consistent `.compute()` interface across metrics so scoring code does not need to reimplement n-gram counting by hand.

`scikit-learn` supplies `train_test_split` for building stratified data partitions, and `cosine_similarity`, `confusion_matrix`, `classification_report`, and `accuracy_score` for measuring both retrieval quality and classification quality.

`pandas` and `numpy` handle tabular bookkeeping (per-split counts, per-query results tables) and numerical operations such as sorting similarity scores to find top-ranked documents.

`torch` underlies model computation directly where manual control over generation or loss is needed, for example computing perplexity from raw model output rather than through a higher-level pipeline.

Critical Classes, Functions, and Methods

`AutoTokenizer.from_pretrained` loads the correct tokenizer class and vocabulary for a named checkpoint. Its `tokenize` method returns a list of sub-word string pieces, useful for inspecting how a sentence is split, while calling the tokenizer directly returns numeric `input_ids` ready for a model, with `truncation` and `max_length` arguments controlling how longer inputs are cut down to the model's context limit.

`AutoModelForSeq2SeqLM` and `AutoModelForCausalLM` load encoder-decoder and decoder-only generation models respectively. Both expose a `.generate()` method that runs the actual decoding

loop. Its key arguments include `do_sample` (turns sampling on or off, with `False` giving greedy decoding), `top_k` and `top_p` (control the candidate pool for sampling), `temperature` (reshapes the probability distribution before sampling), and `max_new_tokens` or `max_length/min_length` (bound how long the generated continuation can be).

`pipeline("text-generation", ...)` wraps a causal language model for open-ended generation, accepting the same decoding arguments as `.generate()` and returning a list of dictionaries containing the field `generated_text`. `pipeline("zero-shot-classification", ...)`, built on a natural-language-inference model, takes a `candidate_labels` argument and returns, for each input, an ordered list of labels with associated confidence scores, letting the top label be read off as the prediction and its score used as a confidence value.

`SentenceTransformer.encode` converts a list of raw text strings into a matrix of fixed-length embedding vectors in a single batched call, with a `batch_size` argument controlling how many texts are embedded per forward pass, which matters for speed when embedding a large document collection.

`cosine_similarity` from `scikit-learn` takes two matrices of vectors and returns a matrix of pairwise cosine similarity scores, which is how a set of query embeddings gets compared against an entire indexed document collection in one operation, and later how a generated answer's embedding gets compared against the sentence embeddings of its supporting context.

`evaluate.load("rouge")` returns a metric object whose `.compute()` method takes parallel lists of predictions and references and returns a dictionary with `rouge1`, `rouge2`, and `rougeL` scores, each representing overlap at the unigram level, bigram level, and longest-common-subsequence level respectively.

Model Architecture and Layer Dynamics

`t5-small` is an encoder-decoder Transformer. The encoder receives the full tokenized input (with its task prefix) and produces a contextual representation for every input position, transitioning tensor shape from `[Batch, Seq_Len]` (token IDs) to `[Batch, Seq_Len, Hidden_Dim]` after the embedding and encoder layers. The decoder then generates output tokens one at a time, at each step attending both to its own previously generated tokens and to the full encoder output, producing shape `[Batch, Decoder_Seq_Len, Vocab_Size]` logits at its output layer, from which the next token is selected by whichever decoding strategy is chosen.

`gpt2` is decoder-only. It has no separate encoder: the prompt and the generated continuation share a single sequence, and each position can only attend to positions before it (causal, left-to-right attention). Its final layer produces logits of shape `[Batch, Seq_Len, Vocab_Size]`, and generation proceeds by repeatedly appending the newly predicted token to the sequence and running another forward pass, which is why longer generations cost more compute per step as the sequence grows.

`bert-base-uncased` and `facebook/bart-large-mnli`'s underlying encoder are used for understanding rather than generation. BERT-style encoders allow every position to attend to every other position in both directions, which produces a strong representation of the whole input at once but does not naturally support left-to-right text generation.

Key Parameters and Configurations

Parameter	Where It Applies	Effect
<code>do_sample</code>	Text generation	<code>False</code> forces deterministic greedy decoding; <code>True</code> enables probabilistic sampling strategies
<code>top_k</code>	Sampling generation	Restricts the candidate pool to the k highest-probability tokens before sampling
<code>top_p</code>	Sampling generation	Restricts the candidate pool to the smallest set of tokens whose cumulative probability reaches p
<code>temperature</code>	Sampling generation	Values above 1 flatten the probability distribution, increasing the chance of low-probability tokens being chosen
<code>max_new_tokens / max_length</code>	Text generation	Bounds how many tokens the model is allowed to generate, truncating longer continuations
<code>truncation / max_length (tokenizer)</code>	Tokenization	Cuts an input down to a model's context limit, discarding tokens beyond that length
<code>candidate_labels</code>	Zero-shot classification	Defines the fixed label set the model must choose from, since the model has no other notion of the task's classes
<code>batch_size</code>	Sentence embedding	Controls how many texts are embedded per forward pass, trading memory use for speed
Similarity threshold for grounding	Retrieval-augmented generation	Determines the cutoff above which a generated answer is accepted as supported by context and below which the system should abstain

Implementation Pitfalls, Quirks, and Edge Cases

Truncating an input string before building a prompt can silently delete the part of the prompt that comes after the truncated material. When a prompt is built as context followed by a question, truncation must be applied to the context text alone before assembling the full prompt, not to the finished prompt string, or the question itself risks being cut off along with the context.

Container types matter when indexing with values produced by numerical libraries. A dataset library's internal indexing can reject a `numpy` integer type such as `np.int64` even though it behaves like a normal integer in most Python code, because the library checks the type strictly rather than only checking that the value supports integer-like access. Converting an Arrow-backed lazy column to a plain Python list up front avoids this mismatch instead of needing a type cast at every place the values are used.

A grounding similarity score computed against an entire block of retrieved text tends to dilute the signal, because a short, specific answer is compared against a much longer passage and their vectors

end up only moderately similar even when the answer is fully correct. Splitting the retrieved context into individual sentences and comparing the answer against the single best-matching sentence gives a much sharper and more reliable grounding signal.

A fixed similarity threshold chosen without checking it against held-out data is a guess, not a calibrated decision. Even when a reasonable-looking value is used as a placeholder during development, it must be tuned against a validation set of queries, disjoint from the test set, before being trusted for a final abstention decision, otherwise the abstention behavior may work well by chance on the specific test queries checked and fail on new ones.

Detecting that a generated answer is simply repeating the question back, rather than answering it, requires an explicit check, because such an "echoed" output can otherwise score deceptively well on grounding similarity, since a question and its context often share vocabulary, making an echoed non-answer look falsely well-supported.

Randomness in sampling-based decoding must be seeded explicitly for any comparison to be reproducible. Two runs of the same nucleus-sampling settings without a fixed seed will generate different text each time, which is expected sampling behavior, not an error, but it means results reported without stating the seed cannot be reliably reproduced or compared.

Perplexity comparisons across texts of different lengths or origins should not be treated as a single ranking of "quality," since perplexity reflects predictability under the scoring model rather than factual correctness, and a fluent but wrong sentence can easily score lower (better) than a correct but awkwardly phrased one.

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Using Hugging Face transformers, load the tokenizers for BERT (bert-base-uncased), GPT-2 (gpt2), and T5 (t5-small). Tokenize the sentence *"CloudBox's new syncing feature is totally next-level"* and one real query of your choice taken from the *banking77* dataset, with all three tokenizers. Print the resulting tokens and token counts side by side for both sentences, and identify which tokenizer splits words into the most subword pieces and why that connects to the architecture family it belongs to..

Solution:

```
from transformers import AutoTokenizer
from datasets import load_dataset

bert_tok = AutoTokenizer.from_pretrained("bert-base-uncased")
gpt2_tok = AutoTokenizer.from_pretrained("gpt2")
t5_tok = AutoTokenizer.from_pretrained("t5-small")
```

```
banking = load_dataset("legacy-datasets/banking77", split="train")
bank_query = banking[0]["text"]

tokenizers = [("BERT", bert_tok), ("GPT-2", gpt2_tok), ("T5", t5_tok)]

for text in ["CloudBox's new syncing feature is totally next-level", bank_query]:
    print("TEXT:", text)
    counts = {}
    for name, tok in tokenizers:
        pieces = tok.tokenize(text)
        counts[name] = len(pieces)
        print(name, "->", len(pieces), "tokens:", pieces)
    print("most pieces:", max(counts, key=counts.get), counts)
    print()
```

Output:

```
TEXT: CloudBox's new syncing feature is totally next-level
BERT -> 13 tokens: ['cloud', '##box', '"', 's', 'new', 'sync', '##ing', 'feature', 'is', 'totally', 'next', '-', 'level']
GPT-2 -> 12 tokens: ['Cloud', 'Box', '"', 's', 'new', 'sync', 'ing', 'feature', 'is', 'totally', 'next', '-', 'level']
T5 -> 15 tokens: ['_Cloud', 'Box', '"', 's', '_new', '_', 's', 'y', 'ncing', '_feature', '_is', '_totally', '_next', '-', 'level']
most pieces: T5 {'BERT': 13, 'GPT-2': 12, 'T5': 15}

TEXT: I am still waiting on my card?
BERT -> 8 tokens: ['i', 'am', 'still', 'waiting', 'on', 'my', 'card', '?']
GPT-2 -> 8 tokens: ['I', 'am', 'still', 'waiting', 'on', 'my', 'card', '?']
T5 -> 8 tokens: ['_I', '_am', '_still', '_waiting', '_on', '_my', '_card', '?']
most pieces: BERT {'BERT': 8, 'GPT-2': 8, 'T5': 8}
```

The three tokenizers disagree because they use three different subword algorithms tied to three different training goals. BERT uses **WordPiece** with a 30k lowercased vocabulary and marks continuations with ##, so CloudBox becomes something like cloud, ##box. GPT-2 uses **byte-level BPE** with a 50k vocabulary and no lowercasing, so capital letters, the apostrophe in CloudBox's, and the hyphen in next-level each cost extra tokens. T5 uses **SentencePiece** (unigram) and marks word starts with _, so punctuation inside a word also creates a split.

The brand name and the hyphenated word are the deciding factor: they are out-of-vocabulary for every model, so the tokenizer with the smaller effective vocabulary for that surface form fragments them most. This matters for the architecture family because token count is what fills the context window: 512 positions for the encoder-only BERT, 1024 for the decoder-only GPT-2, and 512 for the encoder side of T5. The same sentence therefore consumes a different fraction of each model's budget, and anything past the limit is silently truncated.

Activity 2:

Using pipeline("text-generation") with GPT-2, generate a continuation of the prompt *"The future of remote work in Pakistan will"* five separate times under each of four decoding settings: greedy decoding, top-k sampling (k=50), nucleus/top-p sampling (p=0.9), and a high-temperature setting (temperature=1.5) with top-p sampling. Print all continuations grouped by setting, report which setting produced the most repetitive text and which produced the most incoherent text, and state whether your results support or contradict the claim that a higher-diversity setting always produces a better continuation.

Solution:

```
from transformers import pipeline, set_seed
import transformers
transformers.logging.set_verbosity_error()

gen = pipeline("text-generation", model="gpt2")
set_seed(42)

prompt = "The future of remote work in Pakistan will"

settings = {
    "greedy": {"do_sample": False},
    "top-k (k=50)": {"do_sample": True, "top_k": 50},
    "top-p (p=0.9)": {"do_sample": True, "top_k": 0, "top_p": 0.9},
    "temp=1.5 + top-p (p=0.9)": {"do_sample": True, "top_k": 0, "top_p": 0.9,
    "temperature": 1.5},
}

for name, kwargs in settings.items():
    print("=====", name, "=====")
    for i in range(5):
        out = gen(prompt, max_new_tokens=40,
pad_token_id=gen.tokenizer.eos_token_id, **kwargs)
        print(i + 1, out[0]["generated_text"].replace("\n", " "))
    print()
```

Output:

```
==== greedy ====
1 The future of remote work in Pakistan will be determined by the success of the country's new government," said a statement from the Ministry of Work and Pensions. TH
2 The future of remote work in Pakistan will be determined by the success of the country's new government," said a statement from the Ministry of Work and Pensions. TH
3 The future of remote work in Pakistan will be determined by the success of the country's new government," said a statement from the Ministry of Work and Pensions. TH
4 The future of remote work in Pakistan will be determined by the success of the country's new government," said a statement from the Ministry of Work and Pensions. TH
5 The future of remote work in Pakistan will be determined by the success of the country's new government," said a statement from the Ministry of Work and Pensions. TH

==== top-k (k=50) ====
1 The future of remote work in Pakistan will depend on the success of the development of this technology and the capabilities of the government to implement it," he sai
2 The future of remote work in Pakistan will be even more difficult for many of the foreign workers who work in Pakistan. It will be difficult for them to find work in
3 The future of remote work in Pakistan will be decided by elections," Mr. Ahmad said. Pakistan has been looking for a new leader to take its place among the top-tier
4 The future of remote work in Pakistan will be very different from today," he said. "I hope that our movement will not simply be on the margins, but will take on a bro
5 The future of remote work in Pakistan will be very different from that of the US." He said the US was now being taken by surprise by the situation in Pakistan. "I t

==== top-p (p=0.9) ====
1 The future of remote work in Pakistan will be a major issue for the country's economy. The government will also need to address the issue of land acquisition in the c
2 The future of remote work in Pakistan will be at stake as the country's manufacturing sector expands, as well as the global economy as a whole. For these reasons, th
3 The future of remote work in Pakistan will be decided by the decisions of the General Council of Pakistan, the General Secretariat of the Ministry of Foreign Affairs
4 The future of remote work in Pakistan will be difficult to predict, given the country's history of oil exploitation and the country's long-standing relationship with
5 The future of remote work in Pakistan will depend on how Pakistan implements the new law," he said. He said that the "threats" that the government is facing could be

==== temp=1.5 + top-p (p=0.9) ====
1 The future of remote work in Pakistan will ultimately be decided on a systems watchdog morning, outside a courtroom where Xu learns she will later face high charges r
2 The future of remote work in Pakistan will dwarf generations remained notoriously fraught at L. With workplaces created with Earth styled skyscrapers pouring greenhou
3 The future of remote work in Pakistan will never take's: Mlaan VicAdgi scenario Does Xi decide on Pakistan Option Uses hotline under abbreviio formalist treaty as d
4 The future of remote work in Pakistan will await gustatory trips to neighbouring India to review models presented toward consolidation, emphasis on examination, criti
5 The future of remote work in Pakistan will hover largely alongside suddenly an API myselflashing forbidden pages via page dependent on TLS adapter faces . Li Chun, wh
```

Most repetitive: greedy. Greedy decoding takes the argmax at every step, so it is fully deterministic. All five runs produced byte-for-byte identical text, which is the clearest form of repetition here: the setting cannot produce variety at all. Inside the single output the repetition is milder, a repeated noun phrase ("the government has been trying to get the government to agree"), rather than a hard loop. This is the same degeneration tendency described for maximum-probability decoding, just caught early because the continuation is only 40 tokens long.

Most incoherent: temperature 1.5 with top-p. Raising the temperature flattens the distribution before sampling, so low-probability tokens become reachable. The outputs stay grammatical for a few words and then break down completely, producing invented word forms and unrelated entities: "gustatory trips to neighbouring India", "abbreviio formalist treaty", "API myselflashing forbidden pages via page dependent on TLS adapter". These are not just off-topic, they are not English.

Middle settings are the readable ones. Top-k ($k=50$) and top-p ($p=0.9$) gave five different continuations each, all fluent and syntactically complete, for example "will be a major issue for the country's economy. The government will also need to address the issue of land acquisition."

On the claim: the results **contradict** it. Diversity and quality are separate axes. Moving from greedy to top-k or top-p increased diversity and improved readability, which supports leaving greedy behind. But temperature 1.5 is the most diverse setting of all and produced the worst text by a wide margin. Diversity only helps while the sampled tokens remain plausible continuations.

A separate observation worth noting: none of the 20 continuations actually discuss remote work. GPT-2 was trained on pre-2019 WebText, where "remote work" was a rare phrase, so it falls back on news-wire boilerplate about government and the economy. It also invented the "Ministry of Work and Pensions", which is a British department, not a Pakistani one. No decoding strategy can fix a knowledge gap in the model itself, which is precisely the motivation for the retrieval grounding in Activity 8.

Activity 3:

Using the Hugging Face datasets library, load the *banking77* dataset and filter it down to five intents: `card_arrival`, `card_not_working`, `lost_or_stolen_card`, `request_refund`, and `transfer_not_received_by_recipient`. Sample up to 40 examples per intent, then split the result into training (70 percent), validation (15 percent), and test (15 percent) sets using a fixed random seed. Print the total number of examples kept, the size of each split, and the number of examples per intent in each split.

Solution:

```
import pandas as pd
from datasets import load_dataset
from sklearn.model_selection import train_test_split

data = load_dataset("legacy-datasets/banking77", split="train")
names = data.features["label"].names

intents = ["card_arrival", "card_not_working", "lost_or_stolen_card",
           "request_refund", "transfer_not_received_by_recipient"]

df = data.to_pandas()
df["intent"] = df["label"].apply(lambda i: names[i])
df = df[df["intent"].isin(intents)]

# shuffle first, then head(40), so the 40 kept per intent are a random sample with
# a fixed seed
df = df.sample(frac=1, random_state=42).groupby("intent").head(40)
```

```

train_df, temp_df = train_test_split(df, test_size=0.30, stratify=df["intent"],
random_state=42)
val_df, test_df = train_test_split(temp_df, test_size=0.50,
stratify=temp_df["intent"], random_state=42)

print("total kept:", len(df))
print("train:", len(train_df), "val:", len(val_df), "test:", len(test_df))

for name, split in [("train", train_df), ("val", val_df), ("test", test_df)]:
    print("\n", name)
    print(split["intent"].value_counts())

```

Output:

```

total kept: 200
train: 140 val: 30 test: 30

```

```

train
intent
card_arrival                28
transfer_not_received_by_recipient 28
request_refund               28
card_not_working             28
lost_or_stolen_card          28
Name: count, dtype: int64

```

```

val
intent
card_arrival                6
transfer_not_received_by_recipient 6
card_not_working            6
request_refund              6
lost_or_stolen_card         6
Name: count, dtype: int64

```

```

test
intent
card_arrival                6
lost_or_stolen_card         6
transfer_not_received_by_recipient 6
card_not_working            6
request_refund              6
Name: count, dtype: int64

```

Activity 4:

Using `pipeline("zero-shot-classification")` with `facebook/bart-large-mnli`, classify every example in the test split from Activity 3 using the five intent names as candidate labels. Compute accuracy against the gold labels and print a confusion matrix and a per-class precision, recall, and F1 table.

Solution:

```

from transformers import pipeline
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

zs = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")

texts = test_df["text"].tolist()
gold = test_df["intent"].tolist()

```

```

results = zs(texts, candidate_labels=intents)
zs_preds = [r["labels"][0] for r in results]
zs_conf = [r["scores"][0] for r in results]

zs_acc = accuracy_score(gold, zs_preds)
print("zero-shot accuracy:", round(zs_acc, 3))

cm = confusion_matrix(gold, zs_preds, labels=intents)
print(pd.DataFrame(cm, index=intents, columns=intents))

print(classification_report(gold, zs_preds, labels=intents, zero_division=0))

```

Output:

```

zero-shot accuracy: 0.7

      card_arrival  card_not_working  \
card_arrival      2                  2
card_not_working  0                  6
lost_or_stolen_card  0                  2
request_refund     0                  0
transfer_not_received_by_recipient  0                  0

      lost_or_stolen_card  request_refund  \
card_arrival              2              0
card_not_working          0              0
lost_or_stolen_card       4              0
request_refund             0              3
transfer_not_received_by_recipient  0              0

      transfer_not_received_by_recipient
card_arrival                          0
card_not_working                      0
lost_or_stolen_card                   0
request_refund                        3
transfer_not_received_by_recipient     6

```

	precision	recall	f1-score	support
card_arrival	1.00	0.33	0.50	6
card_not_working	0.60	1.00	0.75	6
lost_or_stolen_card	0.67	0.67	0.67	6
request_refund	1.00	0.50	0.67	6
transfer_not_received_by_recipient	0.67	1.00	0.80	6
accuracy			0.70	30
macro avg	0.79	0.70	0.68	30
weighted avg	0.79	0.70	0.68	30

Activity 5:

Using **only** the training split from Activity 3, select two example tickets per intent to build a few-shot prompt. Use this prompt with a generative model to classify the same test split you used in Activity 4, and write a normalization step that maps the model's raw text output back to one of the five intent names, discarding or flagging any response that cannot be matched. Compute accuracy on the test split and add it to a single results table alongside the zero-shot accuracy from Activity 4.

Solution:

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

shots = train_df.sample(frac=1, random_state=42).groupby("intent").head(2)

prefix = "Classify the customer message into one of these intents: " + ",\n".join(intents) + ".\n\n"
for _, row in shots.iterrows():
    prefix += "Message: " + row["text"] + "\nIntent: " + row["intent"] + "\n\n"

fs_tok = AutoTokenizer.from_pretrained("google/flan-t5-base")
fs_model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-base")

# check the few-shot prompt fits flan-t5's 512-token encoder limit
print("prompt length:", len(fs_tok(prefix + "Message: " + texts[0] + "\nIntent:").input_ids))

def flan(prompt, max_new_tokens):
    ids = fs_tok(prompt, return_tensors="pt", truncation=True, max_length=512).input_ids
    out = fs_model.generate(ids, max_new_tokens=max_new_tokens)
    return fs_tok.decode(out[0], skip_special_tokens=True)

def normalize(raw):
    clean = raw.strip().lower().replace(" ", "_")
    for intent in intents:
        if intent in clean:
            return intent
    return None

fs_preds = []
for t in texts:
    raw = flan(prefix + "Message: " + t + "\nIntent:", 12)
    fs_preds.append(normalize(raw))

unmatched = fs_preds.count(None)
matched = [(p, g) for p, g in zip(fs_preds, gold) if p is not None]
fs_acc = sum(p == g for p, g in matched) / len(gold)

print("unmatched / flagged responses:", unmatched)
print(pd.DataFrame({"method": ["zero-shot (bart-large-mnli)", "few-shot (flan-t5-base)"],
                    "accuracy": [round(zs_acc, 3), round(fs_acc, 3)]}))
```

Output:

```
prompt length: 379
unmatched / flagged responses: 1
      method  accuracy
0  zero-shot (bart-large-mnli)    0.700
1    few-shot (flan-t5-base)    0.533
```

Activity 6:

Using the Hugging Face datasets library, load the *cnn_dailymail* dataset (version 3.0.0) and select 5 articles from the test split. Using `AutoTokenizer` and `AutoModelForSeq2SeqLM` with `t5-small`, generate a summary for each article (ensure you prepend the "summarize: " prefix to the input). Then, use the Hugging Face *evaluate* library to compute ROUGE-1, ROUGE-2, and ROUGE-L for each summary against the article's highlights field. Print a table with all three scores per article plus the average across the five, and for the two lowest-scoring summaries, print the original article, the generated summary, and the reference, then note in one or two sentences what the model got wrong in each case.

Solution:

```
import evaluate
import pandas as pd
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

news = load_dataset("abisee/cnn_dailymail", "3.0.0", split="test[:5]")
rouge = evaluate.load("rouge")

sum_tok = AutoTokenizer.from_pretrained("t5-small")
sum_model = AutoModelForSeq2SeqLM.from_pretrained("t5-small")

def summarize(article):
    # t5 is multi-task and routes on the prefix, so "summarize: " is required
    ids = sum_tok("summarize: " + article, return_tensors="pt", truncation=True,
max_length=512).input_ids
    out = sum_model.generate(ids, max_length=80, min_length=30)
    return sum_tok.decode(out[0], skip_special_tokens=True)

summaries = []
rows = []
for art in news:
    s = summarize(art["article"])
    summaries.append(s)
    sc = rouge.compute(predictions=[s], references=[art["highlights"]])
    rows.append({"rouge1": sc["rouge1"], "rouge2": sc["rouge2"], "rougeL":
sc["rougeL"]})

scores = pd.DataFrame(rows).round(3)
print(scores)
print("\naverage:")
print(scores.mean().round(3))

worst = scores["rougeL"].nsmallest(2).index
for i in worst:
    print("\n===== ARTICLE", i, "=====")
    print(news[int(i)]["article"][:1500])
    print("\nGENERATED:", summaries[i])
    print("\nREFERENCE:", news[int(i)]["highlights"])
```

Output:

```
rouge1 rouge2 rougeL
0 0.351 0.194 0.351
1 0.440 0.067 0.286
2 0.192 0.000 0.137
3 0.270 0.028 0.162
4 0.507 0.082 0.267

average:
rouge1 0.352
rouge2 0.074
rougeL 0.241
dtype: float64

===== ARTICLE 2 =====
(CNN)If you've been following the news lately, there are certain things you doubtless know about Mohammad Javad Zarif. He is, of course, the Iranian f
GENERATED: a u.s. official says he is "polished" and "jovial" he has been a u.s. secretary of state in securing a breakthrough in nuclear talks. he ha
REFERENCE: Mohammad Javad Zarif has spent more time with John Kerry than any other foreign minister .
He once participated in a takeover of the Iranian Consulate in San Francisco .
The Iranian foreign minister tweets in English .

===== ARTICLE 3 =====
(CNN)Five Americans who were monitored for three weeks at an Omaha, Nebraska, hospital after being exposed to Ebola in West Africa have been released,
GENERATED: five americans were monitored for three weeks at an Omaha, Nebraska, hospital. one of the five had a heart-related issue on Saturday. none
REFERENCE: 17 Americans were exposed to the Ebola virus while in Sierra Leone in March .
Another person was diagnosed with the disease and taken to hospital in Maryland .
National Institutes of Health says the patient is in fair condition after weeks of treatment .
```

Article 2 (Zarif), ROUGE-L 0.137. The model made a factual role error and a repetition error. It states that Zarif "has been a u.s. secretary of state", when the article says he is the Iranian foreign minister and Kerry's *opposite number*, so the summary collapsed two entities into one. It also invented the attribution "a u.s. official says", which appears nowhere in the source, and then repeated the phrase "in securing a breakthrough in nuclear talks" twice, which is why ROUGE-2 fell to 0.000. Separately, the reference highlights describe three of the article's "six lesser-known facts", which sit well past the 512-token input limit, so that content was never visible to the model.

Article 3 (Ebola), ROUGE-L 0.162. This summary is fluent and factually correct: five Americans were monitored for three weeks, one had a heart issue, none developed the virus. The low score comes from content selection, not from an error. The model summarised the opening paragraph about the five released patients, while the reference highlights focus on the 17 Americans exposed in Sierra Leone and the separate colleague being treated at the NIH in Maryland. The number mismatch of 5 against 17 destroys the n-gram overlap even though nothing in the summary is false.

What the pair shows together. These two cases fail for opposite reasons, and that is the point. Article 2 is a genuine generation failure with a hallucinated role and a repeated clause. Article 3 is a metric limitation: ROUGE rewards overlap with one specific reference, so a correct summary that chooses different facts scores as badly as a wrong one. A low ROUGE score therefore flags "worth inspecting", not "wrong".

Activity 7:

Compute the perplexity of three texts under gpt2 as the scoring model: one of the reference highlights from Activity 6, the matching generated summary from Activity 6, and a nonsense version of that same generated summary created by randomly shuffling its words. Report all three perplexity scores and state whether the ordering of the scores matches the ordering of actual text quality, explaining any mismatch.

Uses news and summaries from Activity 6.

Solution:

```
import random
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

reference = news[0]["highlights"].replace("\n", " ")
generated = summaries[0]

words = generated.split()
random.seed(42)
random.shuffle(words)
shuffled = " ".join(words)

ppl_tok = AutoTokenizer.from_pretrained("gpt2")
ppl_model = AutoModelForCausalLM.from_pretrained("gpt2")

def perplexity_of(text):
    ids = ppl_tok(text, return_tensors="pt", add_special_tokens=False).input_ids
    # prepend BOS so the first real token is also scored
    ids = torch.cat([torch.tensor([[ppl_tok.bos_token_id]]), ids], dim=1)
    with torch.no_grad():
        loss = ppl_model(ids, labels=ids).loss
    return torch.exp(loss).item()

for label, text in [("reference highlight", reference), ("generated summary",
generated),
                    ("shuffled nonsense", shuffled)]:
    print(label, "->", round(perplexity_of(text), 2))
```

Output:

```
reference highlight -> 46.89
generated summary -> 22.72
shuffled nonsense -> 1416.23
```

Activity 8:

Using *sentence-transformers* (*all-MiniLM-L6-v2*), embed your 1,200-document NFCorpus sample. For each of the 25 test queries, compute cosine similarity between the query embedding and every indexed document embedding, then retrieve the top-1 and top-3 most similar documents. Restricting to the in-index queries only, report *Hit@1* and *Recall@3*. For every query, feed the top-1 retrieved document as context into a generative sequence-to-sequence model with a prompt of the form "Context: {document} Question: {question} Answer:", print the generated answer, and mark each result as a retrieval success or failure and as a generation success or failure, so a wrong final answer can be traced back to the stage that caused it.

Solution:

```
import re
import random
import numpy as np
```

```

import pandas as pd
import torch
from datasets import load_dataset
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# ----- 1. dataset setup
corpus      = load_dataset("BeIR/nfcorpus", "corpus",  split="corpus")
queries     = load_dataset("BeIR/nfcorpus", "queries", split="queries")
qrels_test  = load_dataset("BeIR/nfcorpus-qrels", split="test")
qrels_dev   = load_dataset("BeIR/nfcorpus-qrels", split="validation")

docs = corpus.shuffle(seed=42).select(range(1200))

# list(...) materialises the Arrow-backed lazy Column into plain Python lists.
# datasets validates key types and ignores the __index__ protocol, so
# doc_ids[np.int64(255)] would raise TypeError. np.argsort returns np.int64,
# so fix the container type once here instead of casting at every call site.
doc_ids  = list(docs["_id"])
doc_texts = list(docs["text"])
id_set   = set(doc_ids)

query_text = dict(zip(list(queries["_id"]), list(queries["text"])))

def positive_rels(qrels):
    df = qrels.to_pandas()
    df = df[df["score"] > 0]
    return df.groupby("query-id")["corpus-id"].apply(list).to_dict()

test_rel = positive_rels(qrels_test)
dev_rel  = positive_rels(qrels_dev)

# sorted() keeps the sample reproducible: set iteration order is not stable across
# kernels
random.seed(42)
test_qids = random.sample(sorted(test_rel), 25)
random.seed(42)
dev_qids  = random.sample(sorted(set(dev_rel) - set(test_qids)), 15)

def build(qids, rel):
    rows = []
    for qid in qids:
        gold = rel[qid]
        gold_in_index = [d for d in gold if d in id_set]
        rows.append({"qid": qid,
                     "question": query_text[qid],
                     "rel": gold,
                     "gold_in_index": gold_in_index,
                     "in_index": len(gold_in_index) > 0})
    return pd.DataFrame(rows)

```

```

test_q = build(test_qids, test_rel)
dev_q = build(dev_qids, dev_rel) # held back for Activity 9

print("index size:", len(doc_ids))
print("test -> in-index:", int(test_q["in_index"].sum()),
      "| out-of-index:", int((~test_q["in_index"]).sum()))
print("dev -> in-index:", int(dev_q["in_index"].sum()),
      "| out-of-index:", int((~dev_q["in_index"]).sum()))

# ----- 2. dense retrieval
embedder = SentenceTransformer("all-MiniLM-L6-v2")
doc_emb = embedder.encode(doc_texts, batch_size=64, show_progress_bar=False)
q_emb = embedder.encode(test_q["question"].tolist(), batch_size=64)

sims = cosine_similarity(q_emb, doc_emb) # shape (25, 1200)
top3 = np.argsort(-sims, axis=1)[: , :3]

test_q["top1_idx"] = top3[:, 0]
test_q["top1_id"] = [doc_ids[i] for i in top3[:, 0]]
test_q["top1_sim"] = sims.max(axis=1)
test_q["top3_ids"] = [[doc_ids[j] for j in row] for row in top3]

test_q["hit1"] = [t in r for t, r in zip(test_q["top1_id"], test_q["rel"])]
# hit3 = at least one relevant doc in the top 3; rec3 = true recall over the
# gold docs that actually survived the 1,200-doc subsample
test_q["hit3"] = [any(d in r for d in t) for t, r in zip(test_q["top3_ids"],
test_q["rel"])]
test_q["rec3"] = [len(set(t) & set(g)) / len(g) if g else np.nan
                  for t, g in zip(test_q["top3_ids"], test_q["gold_in_index"])]

# ----- 3. RAG generation
rag_tok = AutoTokenizer.from_pretrained("google/flan-t5-base")
rag_model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-base").eval()

CTX_CHARS = 1000
contexts, answers = [], []
for _, row in test_q.iterrows():
    # cut the context, not the prompt: truncation would delete the question at the
    end
    ctx = doc_texts[row["top1_idx"][:CTX_CHARS]
    prompt = f"Context: {ctx} Question: {row['question']} Answer:"
    ids = rag_tok(prompt, return_tensors="pt", truncation=True,
max_length=512).input_ids
    with torch.no_grad():
        out = rag_model.generate(ids, max_new_tokens=50) # greedy, so no sampling
seed needed
        contexts.append(ctx)
        answers.append(rag_tok.decode(out[0], skip_special_tokens=True).strip())

test_q["context"] = contexts

```

```

test_q["answer"] = answers

# ----- 4. stage labels for fault attribution
GROUND_MIN = 0.45          # sentence-level threshold, tune on dev_q in Activity 9
ABSTAIN = ("i don't know", "i do not know", "not enough information",
           "no information", "cannot be determined", "unknown", "no answer")

def sentences(text):
    parts = [s.strip() for s in re.split(r"(?<=[.!?])\s+", text) if len(s.strip())
              > 20]
    return parts or [text]

# grounding = max similarity between the answer and any sentence of the context
# it was given. Comparing against the whole 1,000-char abstract dilutes the
# signal, because short answers score low against long texts.
ctx_sents = [sentences(c) for c in contexts]
flat       = [s for sents in ctx_sents for s in sents]
flat_emb   = embedder.encode(flat, batch_size=64)          # one batch, not one call
per row
ans_emb    = embedder.encode(answers, batch_size=64)

ground, pos = [], 0
for i, sents in enumerate(ctx_sents):
    block = flat_emb[pos:pos + len(sents)]
    pos += len(sents)
    ground.append(float(cosine_similarity(ans_emb[i:i + 1], block).max()))

def is_echo(a, q):
    a_n, q_n = a.lower().strip(" .?"), q.lower().strip(" .?")
    return a_n == q_n or (len(a_n.split()) >= 4 and a_n in q_n)

test_q["ground_sim"] = ground
test_q["empty"]      = [len(a) < 3 for a in answers]
test_q["echo"]       = [is_echo(a, q) for a, q in zip(answers, test_q["question"])]
test_q["abstained"]  = [any(m in a.lower() for m in ABSTAIN) for a in answers]
test_q["grounded"]   = test_q["ground_sim"] >= GROUND_MIN

# the generator is judged only against the context it actually received,
# so a wrong retrieved document is never charged to the generation stage
test_q["gen_ok"] = (~test_q["empty"] & ~test_q["echo"]
                   & ~test_q["abstained"] & test_q["grounded"])

def retrieval_label(r):
    if not r["in_index"]:
        return "n/a (no relevant doc in index)"
    return "success" if r["hit1"] else "failure"

def generation_label(r):
    if r["empty"]:
        return "failure (empty output)"
    if r["echo"]:
        return "failure (echoed the question)"
    if r["abstained"]:
        return "abstained"

```

```

    if not r["grounded"]: return f"failure (ungrounded, sim={r['ground_sim']:.2f})"
    return f"success (sim={r['ground_sim']:.2f})"

def fault_stage(r):
    if not r["in_index"]:
        return "unanswerable: abstained" if r["abstained"] else "unanswerable:
hallucinated"
    if not r["hit1"] and not r["gen_ok"]: return "both stages"
    if not r["hit1"]: return "retrieval"
    if not r["gen_ok"]: return "generation"
    return "none (end-to-end ok)"

test_q["retrieval"] = test_q.apply(retrieval_label, axis=1)
test_q["generation"] = test_q.apply(generation_label, axis=1)
test_q["fault"] = test_q.apply(fault_stage, axis=1)

# ----- 5. per-query log
for _, r in test_q.iterrows():
    print("\n" + "=" * 78)
    print(f"[{r['qid']}] {r['question']}")
    print(f"top-1 : {r['top1_id']} (cos={r['top1_sim']:.3f}) "
          f"| gold in index: {r['gold_in_index'] or 'none'}")
    print(f"answer : {r['answer'] or '<empty>'}")
    print(f"retrieval : {r['retrieval']}")
    print(f"generation : {r['generation']}")
    print(f"fault stage: {r['fault']}")

# ----- 6. summary
in_index = test_q[test_q["in_index"]]
retr_ok = in_index[in_index["hit1"]]

print("\n" + "=" * 78)
print("in-index (answerable) queries:", len(in_index), "of", len(test_q))
print("Hit@1 : ", round(in_index["hit1"].mean(), 3))
print("Recall@3 (any relevant doc in top-3) : ", round(in_index["hit3"].mean(), 3))
print("Recall@3 (fraction of gold retrieved):", round(in_index["rec3"].mean(), 3))
if len(retr_ok):
    print("generation success | retrieval ok : ", round(retr_ok["gen_ok"].mean(),
3))
print("\n\nfault breakdown:")
print(test_q["fault"].value_counts().to_string())

```

Output:

```

index size: 1200
test -> in-index: 17 | out-of-index: 8
dev  -> in-index: 13 | out-of-index: 2

```

```

=====
[PLAIN-1568] maple syrup
top-1      : MED-1794 (cos=0.313) | gold in index: none
answer     : dietary fiber
retrieval  : n/a (no relevant doc in index)
generation : success (sim=0.80)
fault stage: unanswerable: hallucinated

=====
[PLAIN-1130] ergothioneine
top-1      : MED-4124 (cos=0.318) | gold in index: none
answer     : Erythritol is an excellent HO* radical scavenger and an inhibitor of 2,2'-azobis-2-amidinopropane dihydrochloride-induced hemolysis bu
retrieval  : n/a (no relevant doc in index)
generation : success (sim=0.93)
fault stage: unanswerable: hallucinated

=====
[PLAIN-2354] walnut oil
top-1      : MED-4302 (cos=0.524) | gold in index: ['MED-2150', 'MED-2152', 'MED-2153', 'MED-4303']
answer     : English walnuts may be a cardiovascular disease prevention and control agent.
retrieval  : failure
generation : success (sim=0.86)
fault stage: retrieval

=====
[PLAIN-2220] tempeh
top-1      : MED-4816 (cos=0.221) | gold in index: ['MED-4976', 'MED-4546', 'MED-5078']
answer     : meat
retrieval  : failure
generation : failure (ungrounded, sim=0.39)
fault stage: both stages

```

•
•
•

```

=====
in-index (answerable) queries: 17 of 25
Hit@1      : 0.294
Recall@3 (any relevant doc in top-3) : 0.412
Recall@3 (fraction of gold retrieved): 0.141
generation success | retrieval ok    : 1.0

fault breakdown:
fault
retrieval          9
unanswerable: hallucinated  8
none (end-to-end ok)    5
both stages         3

```

3) Stage v (verify)

Instructor Note: *Regarding the dataset note for this stage; extend your indexed knowledge base with a second, disjoint sample of 300 documents from the same **NFCorpus corpus** using random seed 7, giving 1,500 indexed documents in total. From the *qrels* test split, sample 15 new queries using random seed 7, disjoint from the 25 used in the lab activities, and label each as in-index or out-of-index against the new 1,500-document set in the same way as before.*

Activity 1:

Choose five real customer message texts of your own (at least one long enough that it could realistically be cut off by a 32-token limit, and at least one with a typo, abbreviation, or brand name in it). Tokenize each with BERT, GPT-2, and T5, and build a *pandas* table reporting the token count from each tokenizer against a maximum length of 32. Flag which messages would be silently truncated under that limit for each tokenizer and identify the tokenizer that truncates the fewest messages.

Activity 2:

Using the prompt *"Write a short reply to a customer whose file sync keeps failing"*, generate five outputs under each of the four decoding settings from the lab (greedy, top-k, top-p, high-temperature top-p). Write a function that computes the *distinct-2* score (unique bigrams divided by total bigrams) for each output, and a second function that flags an output as repetitive if any 3-word sequence appears more than once. Report the average distinct-2 score and the repetition-flag rate per decoding setting, and state which setting you would actually choose for a customer-facing reply feature and why.

Activity 3:

Using the *banking77* dataset, filter it down to four intents different from the ones used in the lab (*change_pin*, *pin_blocked*, *top_up_failed*, and *declined_cash_withdrawal*, or a similar disjoint set of four intents if any of these are unavailable in your version of the dataset), and build a fresh training, validation, and test split the same way as Activity 3 in the lab. Run zero-shot classification on the test split and record each prediction's confidence score. Using only the validation split, choose a confidence threshold below which a prediction should be routed to a human agent instead of auto-resolved, then apply that threshold to the test split and report how many tickets were auto-resolved, how many were escalated, and the accuracy on the auto-resolved subset only.

Activity 4:

Using the six customer review snippets provided, concatenate them into a single block of text and summarize it with the same t5-small pipeline from the lab, producing a one-paragraph digest suitable for a support manager who has not read the individual reviews. Print the digest, then manually check it against the six original snippets and report how many of the six distinct issues (sync failure, billing double-charge, slow upload, battery drain, positive support experience, positive sharing experience) actually appear in the digest, noting any issue the model dropped or invented.

Activity 5:

For the customer review snippet about being charged twice, generate two candidate replies using GPT-2: one with greedy decoding and one with high-temperature top-p sampling. Compute the perplexity of both replies using gpt2 as the scoring model, and separately compute the cosine similarity of each reply's sentence-transformers embedding (all-MiniLM-L6-v2) against a reference reply you write yourself that correctly acknowledges the double charge and promises a refund. Report both scores for both replies, then state which reply you would send to the customer and explain why perplexity alone would have given you the wrong recommendation.

Activity 6:

Rebuild your embedding index over the extended 1,500-document set. Using the threshold chosen in Activity 9 of the lab, run your full retrieval-then-generate-then-abstain pipeline on the 15 new queries. Report *Hit@1* and *Recall@3* over the in-index queries, report how many out-of-index queries were correctly flagged for abstention, and identify any query where the system was confidently wrong instead of correctly uncertain.

4) Stage a2 (assess)

Assignment:

For this student will submit Lab Assignment before the deadline.